
XDrawMaker

Version 4.0

User's Guide

Nova Software Labs

© Copyright Nova Software Labs S.A.R.L. 1993 -2002.

All rights reserved. No part of this document may be stored in a retrieval system, transmitted, or otherwise copied or reproduced by any electronic, mechanical, photographic, or recording process without express written permission from Nova Software Labs S.A.R.L.

Published by:



Nova Software Labs

8, rue de Hanovre

75002 Paris, France

Tel: +33 1 58 18 61 61

Fax: +33 1 58 18 61 60

email: sales@NovaSoftwareLabs.com or sales@nsl.fr.

Web: <http://www.NovaSoftwareLabs.com> or <http://www.nsl.fr>.

Every precaution has been taken to ensure the accuracy of this document. Nova Software Labs S.A.R.L. assumes no responsibility for errors or omissions, or for damages resulting from the use of this document.

XFaceMaker, XDrawMaker, XnslDrw are registered trademarks of Nova Software Labs S.A.R.L. X Window System is a trademark of the X Consortium. UNIX is a registered trademark of Novell. OSF/Motif and Motif are trademarks of The Open Group, Inc. All trademarks are the property of their respective owners.

Document Number: XDMLRefMan v. 7.1.1.

Document date: November 29, 2002

Acknowledgments

The following people have contributed to the development of the XFaceMaker suite:

Alain Bastard, Genevieve Beaujard, Nicole Brion, Alan Char, Eric Durocher, Marc Durocher, France Geze, Eric Herbaut, Elizabeth Huffer, Ion Filotti, Jérôme Maloberti, Sara Sautin, Eric Touchard, Philippe Volle.

Many thanks to all of them and to several others.

Table of Contents

1. Introduction	1
1.1. XDrawMaker Architecture	2
1.2. About this Guide	3
1.3. Terminology	4
1.4. Typesetting Conventions	5
1.5. NSL Technical Support	5
2. Getting Started	7
2.1. Starting the program	8
2.2. The Main Window	9
2.3. Setting up Your Document	10
2.4. Setting up the Grid	12
2.5. Working with Objects	14
2.5.1. Using the Escape (osfCancel) Key	14
2.5.2. Drawing Objects	14
2.5.3. Naming Objects	16
2.5.4. Linking Objects	17
2.5.5. Selecting Objects	20
2.5.6. Grouping Objects	21
2.5.7. Moving Objects	21
2.6. Writing Scripts	22
2.7. Saving your work	28
2.8. Generating Files	29
2.9. Printing	30
2.10. Opening an Existing File	31
3. Drawing And Graphics	33
3.1. Drawing Objects	34
3.1.1. The Object Icons	35
3.1.2. The Select and Keep Tools	35
3.2. Geometric Objects	36
3.3. Grouping Objects	39
3.4. Creating Paths	40
3.4.1. The Tree	41

3.5. Adding Graphics	43
3.6. The Attribute Boxes.	47
3.6.1. The Graphic Attributes Box	47
3.6.2. The Geometry Box	51
3.6.3. The Arrow Box	52
3.6.4. The Dash Box	53
3.6.5. The Slot Box	53
3.6.6. The Expert Box	55
3.7. Entering Text	56
3.7.1. Using the Text Icon.	56
3.7.2. Setting Text Options	56
3.7.3. Changing Fonts	62
3.7.4. Using the Label	63
3.8. Inserting Images.	65
3.8.1. Color Allocation	66
4. Editing Techniques	67
4.1. Moving Objects	67
4.2. Resizing Objects	68
4.2.1. Adding and Removing Points	68
4.3. Rotating Objects	69
4.4. Scaling Objects	70
4.5. Cut, Copy, Paste, Duplicate, Delete.	71
4.6. Grids and View Options	72
4.6.1. Viewing Options	72
4.6.2. The Zoom Feature.	72
4.6.3. Grids	73
4.6.4. Aligning Objects on a Grid	75
4.7. Display options	76
5. Animating a Drawing	77
5.1. Writing scripts	78
5.2. Running a script.	82
6. The FACE Language	85
6.1. Syntax in FACE.	86

6.1.1. Keywords in FACE	86
6.1.2. Special Characters	86
6.1.3. Comments	86
6.1.4. Arithmetic Expressions	86
6.1.5. Include Files.	87
6.2. Basic Objects	88
6.2.1. Object Types	88
6.2.2. Special FACE Types	90
6.2.3. Constants	90
6.2.4. Variables	91
6.2.5. Special Variables	93
6.2.6. Defined Constants	94
6.2.7. Widgets	95
6.2.8. Widget Resources	97
6.2.9. X Properties	98
6.3. Type Casters	100
6.3.1. Declaring New Type Casters	100
6.3.2. Declaring Enumerated Resource Types	100
6.3.3. Explicit Type Conversion	101
6.4. Statements.	102
6.4.1. Assignments.	102
6.4.2. Control Structures	103
6.4.3. Function calls.	104
6.4.4. Named Scripts	104
6.5. FACE Functions	106
6.5.1. Scope of Variables in FACE Functions.	107
6.5.2. Declaration of Application Functions	108
6.6. Floating-Point Variables.	109
6.7. Using Arrays in FACE	111
6.7.1. Declaring Arrays	111
6.7.2. Allocating Arrays.	113
6.7.3. Accessing Array Elements.	114
6.7.4. Iterating Through Arrays	115
6.7.5. Getting the Size of an Array	115
6.7.6. Getting the Address of an Array	116

6.7.7. Clearing an Array	116
6.8. Structures	117
6.8.1. Declaring a Structure from a C application.	117
6.8.2. Declaring a Structure in a FACE script.	118
6.8.3. Declaring Structure Variables	118
6.8.4. Referencing Structure Fields	118
6.8.5. Allocating Structures	119
6.8.6. Arrays as Structure Elements	119
APPENDIX A: Built-in FACE Functions	121
1. General FACE Functions	121
2. FACE Type-casting Functions	128
APPENDIX B: Configuring XDrawMaker	131
1. Command Line Options	131
2. Environment Variables	132
APPENDIX C: The XDrawMaker Menus	135
APPENDIX D: XDrawMaker Functions	141
1. Arrow Functions	142
XnslDrawSetArrowDimension	143
XnslDrawSetArrowDirection	144
XnslDrawSetArrowStyle	145
2. Bitmap Table Functions	146
XnslDrawBitmapName	147
XnslDrawBitmapPixmap	148
XnslDrawBitmapStipple	149
XnslDrawClearBitmapTable	150
XnslDrawGetIndexBitmap	151
XnslDrawGetStipple	152
XnslDrawGetStipplePixmap	153
3. Color Table Functions	154
XnslDrawClearColorTable	155
XnslDrawColorName	156
XnslDrawColorPixel	157
XnslDrawGetColor	158
4. Object Creation Functions	159
XnslDrawCreateArc	160
XnslDrawCreateDrawObject	161

XnslDrawCreateEllipse	162
XnslDrawCreateImage	163
XnslDrawCreateLabel	164
XnslDrawCreateLine	165
XnslDrawCreatePolygon	166
XnslDrawCreateRectangle	167
XnslDrawCreateRegularPolygon	168
XnslDrawCreateSegment	169
XnslDrawCreateSpline	170
XnslDrawCreateText	171
5. Event Functions	172
XnslDrawEventDblX	173
XnslDrawEventDblY	174
XnslDrawEventToAngle	175
XnslDrawEventX	176
XnslDrawEventY	177
XnslDrawGetKeyEventString	178
XnslDrawSync	179
6. Font Table Functions	180
XnslDrawClearFontTable	181
XnslDrawFontName	182
XnslDrawFontStruct	183
XnslDrawGetFont	184
XnslDrawGetFontIndex	185
7. Geometric Transformation Functions	186
Center	186
XnslDrawDefineCenter	187
XnslDrawGetCenterX	188
XnslDrawGetCenterY	189
XnslDrawUndefineCenter	190
Miscellaneous Geometric Functions	191
XnslDrawCircTranslateObject	192
XnslDrawMoveObject	193
XnslDrawMoveObjectTo	194
Rotation Functions	195
XnslDrawGetRotation	196
XnslDrawRotateObject	197
XnslDrawSetRotation	198
Scale functions	199

XnsldrawgetPostScaleX	200
XnsldrawgetPostScaleY	201
XnsldrawgetPreScaleX	202
XnsldrawgetPreScaleY	203
XnsldrawgetScaleX	204
XnsldrawgetScaleY	205
XnsldrawPostScaleObject	206
XnsldrawPreScaleObject	207
XnsldrawScaleObject	208
XnsldrawSetPostScale	209
XnsldrawSetPreScale	210
XnsldrawSetScale	211
Translation Functions	212
XnsldrawGetTranslationX	213
XnsldrawGetTranslationY	214
XnsldrawSetTranslation	215
XnsldrawTranslateObject	216
8. Graphic Transformation Functions	217
XnsldrawGetBackground	218
XnsldrawGetBackgroundPixel	219
XnsldrawGetBitmap	220
XnsldrawGetForeground	221
XnsldrawGetForegroundPixel	222
XnsldrawGetLineWidth	223
XnsldrawSetBackground	224
XnsldrawSetBackgroundIndex	225
XnsldrawSetBackgroundPixel	226
XnsldrawSetCloseMode	227
XnsldrawSetDashes	228
XnsldrawSetFillMode	229
XnsldrawSetForeground	230
XnsldrawSetForegroundIndex	231
XnsldrawSetForegroundPixel	232
XnsldrawSetLineWidth	233
XnsldrawSetPctFill	234
XnsldrawSetStipple	235
XnsldrawSetStippleIndex	236
XnsldrawSetStipplePixmap	237
XnsldrawSetTSOrigin	238
9. Input/Output Functions	239

XnslDrawChangeFile	240
XnslDrawLoadFile	241
XnslDrawReadFile	242
XnslDrawSaveWidget.	243
10. Miscellaneous Functions	244
XnslDrawComplete.	245
XnslDrawCos	246
XnslDrawDegToRad.	247
XnslDrawFlush	248
XnslDrawGetCoordFromDrawPoint.	249
XnslDrawGetNthChild	250
XnslDrawRadToDeg	251
XnslDrawRandom.	252
XnslDrawRedisplay	253
XnslDrawRedisplayAll	254
XnslDrawRefreshControl	255
XnslDrawSin	256
XnslDrawSleep	257
XnslDrawStringToWidget	258
XnslDrawWidgetToString	259
11. General Object Management Functions	260
XnslDrawAdjustObjects	261
XnslDrawAlignObjectToGrid.	262
XnslDrawDeleteAllChildren.	263
XnslDrawDeleteAllObjects.	264
XnslDrawDuplicateObject	265
XnslDrawDupObject.	266
XnslDrawFaceGetObjectPoints	267
XnslDrawFaceGetObjectXPoints	268
XnslDrawFaceSetObjectPoints.	269
XnslDrawFlipObject	270
XnslDrawGetObjectName	271
XnslDrawGetUserData	272
XnslDrawHideObject	273
XnslDrawNameToObjct	274
XnslDrawObjectVisible	275
XnslDrawParent	276
XnslDrawRefNameToObjct	277
XnslDrawSetObjectName.	278

XnslDrawSetUserData	279
XnslDrawShowObject	280
Stack Management	281
XnslDrawBringToFront	282
XnslDrawCirculateDown	283
XnslDrawCirculateUp	284
XnslDrawSendToBack	285
SelectionManagement	286
XnslDrawFirstSelected	287
XnslDrawNextSelected	288
Object Type Checking Functions	289
XnslIsDrawArc	290
XnslIsDrawEllipse	291
XnslIsDrawGroup	292
XnslIsDrawImage	293
XnslIsDrawLabel	294
XnslIsDrawLine	295
XnslIsDrawObject	296
XnslIsDrawPath	297
XnslIsDrawPolygon	298
XnslIsDrawRectangle	299
XnslIsDrawSegment	300
XnslIsDrawSpline	301
XnslIsDrawText	302
XnslIsDrawWidget	303
Image Functions	304
XnslDrawGetImage	305
XnslDrawSetImage	306
Group Functions	307
XnslDrawAddInGroup	308
XnslDrawRemoveFromGroup	309
XnslDrawUngroupGroup	310
Ellipse Functions	311
XnslDrawGetEllipseEndAngle	312
XnslDrawGetEllipseStartAngle	313
XnslDrawSetEllipseAngles	314
XnslDrawSetEllipseEndAngle	315
XnslDrawSetEllipseStartAngle	316
12. Pixmap Table Functions	317
XnslDrawClearPixmapTable	318

XnslDrawGetPixmap	319
XnslDrawPixmapName.	320
XnslDrawPixmapPixmap	321
XnslDrawPixmapType	322
XnslDrawReleasePixmap	323
13. Script Functions	324
XnslDrawArmScript	325
XnslDrawCallScript	326
XnslDrawCallScriptByName	327
XnslDrawDisableScripts	328
XnslDrawDisarmScript	329
XnslDrawEnableScripts	330
14. Slot Functions.	331
XnslDrawClearSlotSelection	332
XnslDrawGetSlotPixmap	333
XnslDrawGetSlotSize	334
XnslDrawSelectObjectSlots	335
XnslDrawSelectSlot	336
XnslDrawSetSlotPixmap.	337
XnslDrawSetSlotSize	338
XnslDrawToggleSelectSlot.	339
15. Text Functions	340
Text Alignment Functions	340
XnslDrawTextSetAlignment.	341
XnslDrawTextSetJustif.	342
XnslDrawTextSetParAlignFromPos.	343
XnslDrawTextSetParAlignment	344
Text Attribute Functions.	345
XnslDrawTextChangeAttributesAtPos.	346
XnslDrawTextChangeFont	347
XnslDrawTextGiveStateAtPos	348
XnslDrawTextInsertMark	349
Text File Functions.	350
XnslDrawTextInsertAsciiFile	351
XnslDrawTextInsertDtiffFile	352
XnslDrawTextReadDtiffFile	353
XnslDrawTextSaveAsciiText	354
XnslDrawTextSaveDtiffText	355
Text Insertion Functions	356

XnslDrawTextGetAsciiText	357
XnslDrawTextGetString	358
XnslDrawTextInsertAsciiAtPos	359
XnslDrawTextInsertAsciiText	360
XnslDrawTextInsertDtifText	361
XnslDrawTextInsertDtifTextAtPos	362
XnslDrawTextInsertString	363
Text Killer Functions	364
XnslDrawTextClearText	365
XnslDrawTextEraseBuffer	366
XnslDrawTextRuboutChar	367
XnslDrawTextRuboutCharAtPos	368
XnslDrawTextYank	369
Text Position Functions	370
XnslDrawTextGetInputPosition	371
XnslDrawTextGetLastPosition	372
XnslDrawTextGetPosition	373
XnslDrawTextSetInputPosition	374
XnslDrawTextSetPosition	375

CHAPTER 1: Introduction

Welcome to XDrawMaker, NSL's graphics design program for Unix. This User's Guide is designed to get you started using the program immediately. You'll find brief descriptions of XDrawMaker's many options, including the innovative *Activate* feature, which enables you to animate the widgets you draw by writing scripts to define their behavior.

Besides the usual drawing and editing features you are no doubt already familiar with, XDrawMaker has easy and intuitive ways to:

- draw a wide variety of objects
- create schematic diagrams by linking objects
- resize, align, and transform objects using the mouse and menus
- combine two or more Draw files
- incorporate images and text into a Draw file
- generate PostScript files for printing.

XDrawMaker has a wide range of graphics options that can be applied to the objects you create. Simply clicking on the appropriate icon selects the color, fill, background, line width, or open/close options for your drawings. A set of dialog boxes helps you to specify your preferences with great precision. OptionMenus let you select distance and angle units for your drawing.

Most importantly, XDrawMaker lets you *animate* the drawings you create by writing scripts in the easy-to-use FACE language. You can specify simple or complex actions for the widgets you design, then test them interactively.

When used with XFaceMaker, NSL's interface builder for Motif, you can create new *classes* of widgets to include in your application or interface. Thus, you are no longer limited to using the components of the Motif toolkit, but can build widgets for almost any conceivable purpose.

The XdrawMaker and Draw Library distribution includes a number of examples that illustrate how to do things. Please refer to the **demos** directory in the installed tree.

1.1 XDrawMaker Architecture

XDrawMaker is a drawing editor enhanced by the ability to animate objects. It is based on the **XsnlDraw** widget. Once a drawing has been created, you can specify object behavior with FACE scripts, then test it by using the ***Run in DrawMaker Window*** or ***Launch a Separate Process*** menu items in the Activate menu.

The ***Launch a Separate Process*** command is a test mode that launches a new process (called **dme**) in which the drawing is loaded in the XDrawMaker **Exec** window. In test mode, the editor is not active. The ***Run in DrawMaker Window*** command runs the script in the XDrawMaker window.

Furthermore, with XFaceMaker, you can create new classes of widgets, based on the animated drawings made with XDrawMaker. The following is a brief overview of the process:

1. Create your drawing with XDrawMaker.
2. Specify the internal behavior of the widget with FACE scripts within XDrawMaker.
3. Test it interactively using the ***Run*** or ***Launch*** commands.
4. Use XFaceMaker to specify resources for the widget.
5. Include the widget in your application or **XFM** interface.

1.2 About this Guide

This guide to XDrawMaker will take you through the steps needed to create, modify, and activate drawings, using the mouse, a set of convenient icons, menu options, and scripts.

Some knowledge of X Motif and of C programming is required. For information on the FACE language see Chapter 6. If you've already worked with a graphics program, you will find many of the procedures familiar, but you might want to check for features of XDrawMaker that are new to you.

This guide is organized as follows:

Introduction	- Chapter 1
Drawing and graphics	- Chapters 2-3-4
Animating your drawings	- Chapter 5
The FACE language	- Chapter 6
Functions, menus, options	- Appendix A,B, C, D

1.3 Terminology

We assume that you are familiar with using windows; i.e., that you can open, resize, iconify, and close windows. We also assume you are working with a three-button mouse. If not, you will need to know which combinations of keyboard and mouse actions correspond to mouse Button 1 (*Select*), mouse Button 2 (*Menu*), and mouse Button 3 (*Custom*). We'll use the following terms to describe mouse actions:

Press	Hold down a mouse button.
Release	Release a mouse button after it has been pressed.
Click	Quickly press and release the mouse <i>Select</i> button without moving the mouse.
Drag	Move the mouse while pressing a mouse button.
Rubber band	This is a rectangular outline displayed when the cursor is dragged. One of its corners is fixed at the point where the button was pressed and the corner opposite follows the pointer. Releasing the button creates a rectangle that defines either the size of an object or a selection area.
Outline box	This is the outline of an object displayed as it is moved so that it can be placed correctly.

For the sake of brevity, the term *click* will always mean clicking the *Select* button on the mouse, unless stated otherwise. To get the most out of XDrawMaker you should be using a color monitor, although it is not a requirement. Naturally, when using a black and white monitor, you won't be able to see certain graphical options (e.g., certain background patterns) on your screen.

Commands are presented in the format ***menu name/ item name***. For example, the command ***File/Save*** indicates that you should open the ***File*** menu and click on the ***Save*** item. Some menu items are followed by an ellipsis, to indicate that some further action is needed to complete the action. For example, the ***File/Save As...*** command pops up the File Selector so that you can enter a new file name. Some items have sub-menus.

1.4 Typesetting Conventions

Type style	Used for:
<code>typewriter</code>	Anything you should type as it appears.
<i>italic</i>	Mouse buttons and emphasis.
<i>bold-slant</i>	Menu commands.

1.5 NSL Technical Support

Customer support for XDrawMaker is available by phone, fax, or electronic mail (e-mail: support@nsl.fr). You can contact NSL at:

Nova Software Labs S.A.R.L.
57-59 rue Lhomond
75005 Paris
France

Phone: (33) [0]1.44.08.70.80

Fax: (33) [0]1.44.08.70.81

E-mail: requests@nsl.fr

CHAPTER 2: Getting Started

This chapter gets you started using XDrawMaker. We will:

- Introduce the main features of the XDrawMaker window.
- Explain how to open and save XDrawMaker files.
- Discuss working with objects.
- Describe the different files you can generate

Many of the topics in this chapter are explained in more detail in the following chapters.

Before starting XDrawMaker, make sure that:

- XDrawMaker has been installed on your system. If not, consult the separate NSL document *Installation Guide for NSL Products*.
- the X Window System is up and running.
- your window manager is running. Although you may use other window managers, (e.g. twm and olwm), XDrawMaker has been developed and tested with the Motif window manager mwm. If problems arise when using XDrawMaker, verify if they remain when you use mwm.
- The NSL license server has been configured to give you access to the product. If it has not, XDrawMaker will start in “demo” mode, i.e. you will not be able to save your work or to load existing files.

Practice Using XDrawMaker

To help you discover the basics of using XDrawMaker, we will illustrate our explanations with a simple example, marked: “**Practice**” and displayed in this font. We suggest you follow this example each step of the way to create a simple animated drawing.

Let’s begin by launching the program and looking at the XDrawMaker main window.

2.1 Starting the program

Assuming that the X server and a window manager are running, and that XDraw-Maker has been installed and configured, you can launch XDrawMaker with the command

```
dm &
```

The XDrawMaker main window will be displayed on your screen. An indicator at the bottom of the window shows program's status. When loading a draw file, a progress bar is displayed; by default, it lengthens in increments of 10% of load time. You can change this to increments of 1%, 50%, or none, using the **-indicator** option. The current file name is displayed at the top of the window, or, by default, unnamed.draw.

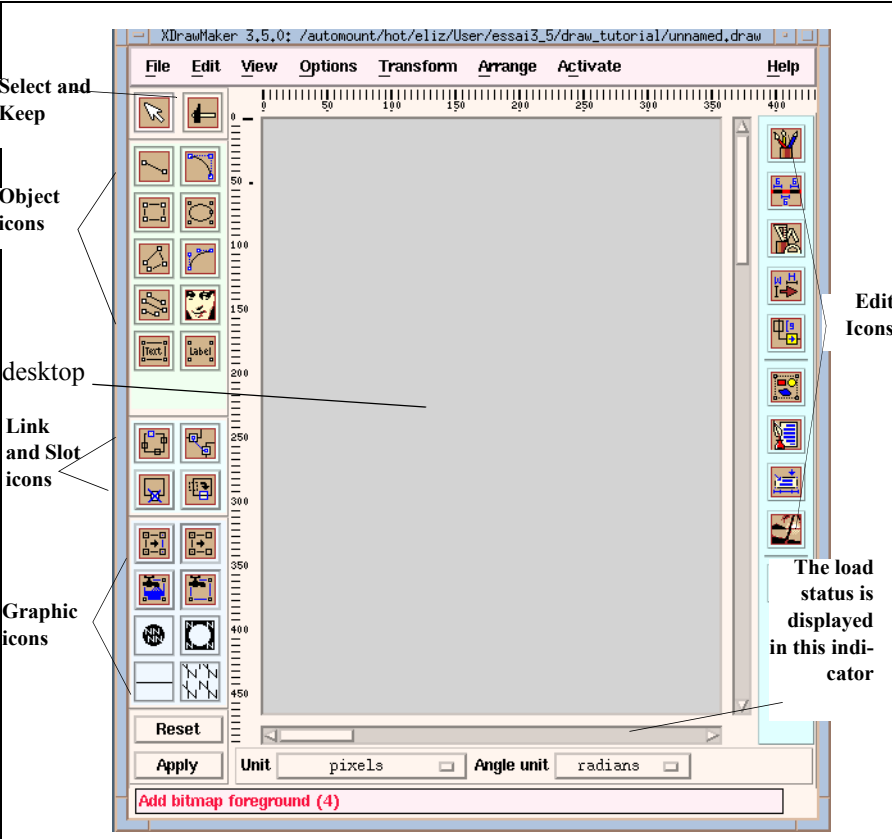


Figure 2.1: The Main Window

2.2 The Main Window

The XDrawMaker main window has three main parts.

- The **menu bar** contains pull-down menus which list the many commands and options available. Click on a menu name to open it. Click on a menu item or enter its key combination to select it. Click outside the menu to close it.
- The **icon bar** on the left contains the **Object** and **Graphics** icons you'll use to create your drawings. The icons on the right are used to open editing windows. As you move the mouse cursor over the icons, a short text identifying the icon is displayed in the indicator line at the bottom of the main window.
- The **desktop** is the drawing area. Scrollbars let you view parts of the desktop that are not currently visible. The status indicator line at the bottom of the draw area displays the latest command given. Rulers help you to measure sizes in the units chosen in the Units menu.

Below the icon bar, there are two buttons: *Apply* and *Reset*.

- *Apply* applies the graphic choices to the selected object.
- *Reset* returns the graphic icons to their default values.

There are two pulldown menus below the desktop: Unit and Angle Unit.

- The **Unit** menu has four choices for the unit of measurement used in your drawing and displayed on the rulers: pixels, points, millimeters, 1/8th inches. You may set different units for different draw files. However, you cannot insert a draw file into a draw file measured in different units. If you attempt to do so, a warning will be issued by XDrawMaker. In that case, change the unit of measurement for one of the files and re-attempt to insert the file.
- The **Angle Unit** pulldown menu has three choices: radians, degrees, and revolutions. You may set different units for different draw files. However, you cannot insert a draw file into a draw file measured in different units. If you attempt to do so, a warning will be issued by XDrawMaker.

2.3 Setting up Your Document

Before you begin drawing, it is useful to consider the *end product*. Will the drawing cover one page or many? What about printing? You can choose the size and layout for your drawing by setting values in the Page Settings box. To display it, choose *Options/Page Description...*

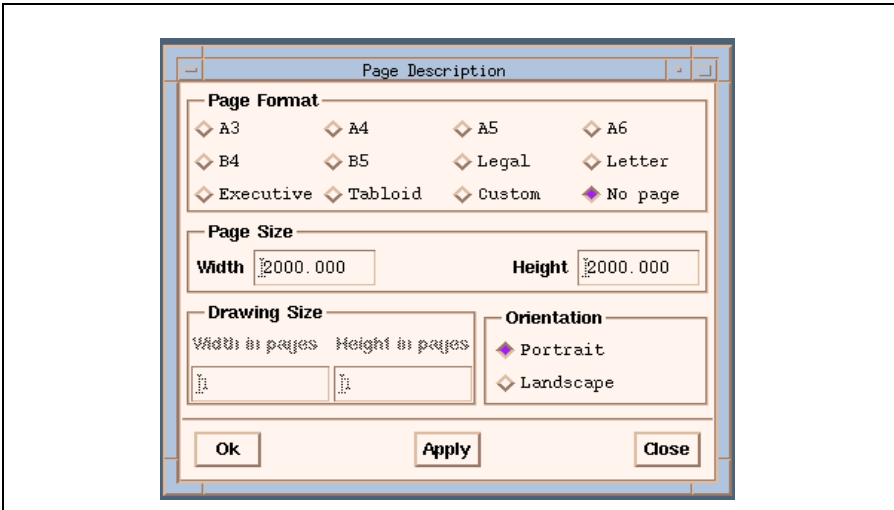


Figure 2.2: The Page Description Box

Click on the indicator next to your choice of the following options:

- **Page Size** This field lets you choose the size of the PostScript page for your drawing. Choose one of the standard sizes by clicking on the indicator next to its name. The dimensions of the chosen page are listed in the two boxes below. Or select the **Custom** or the **No page** option. The Page Size textFields are insensitive when a standard page size is selected.
 - m If you choose **No Page**, the size of the PostScript drawing is computed from the size of your drawing. It is the minimum size necessary to enclose all the objects of your drawing in a rubber band box. The **No page** option is useful for drawings that you intend to format in PostScript and insert into a document, or for drawing widgets with XDrawMaker. If you select this option, you cannot change the Drawing Size.

m If you choose the **Custom** option, enter the width and height of the page in the corresponding text boxes. The unit of measurement is the one you set in the main window.

- **Portrait or Landscape** Choose the upright or horizontal format.
- **Drawing Size** If your drawing will cover more than one page, enter the number of pages here.

Click **Apply** to apply the choices to your current document without closing the dialog box. Click on **OK** to apply the page settings and exit the box. Or click on **Close** to exit the box without applying the page settings.

It may help to view the page setting choices beforehand. To do this, click on **View/Page Size**. The option is checked when it is set. An outline of the page size is displayed on the desktop or, if the viewing window size is smaller than the drawing, the scroll bars are adjusted to the drawing size. Click again on **Page Size** to hide the outline. Note that objects which overlap the page outline are printed that way—part of the object on one page and part on the other.



Note: The space available for your drawing will be smaller than the page size you have chosen, in order to allow for the margins required by most printing devices. The margins are 0.24 inches, (6mm).

Practice

Set up a single page in A5 size in portrait orientation.

2.4 Setting up the Grid

It is often useful to display a grid in the background of your desktop to help you draw and align objects. Of course, the grid is not a part of your Draw file and will not appear when you print your work.

To hide or show the grid, click on **View/Grid**. This is a toggling item. When the **Grid** feature is “on”, the menu item is checked. Turn it “off” to hide the grid, by clicking again on **Grid**.

XDrawMaker gives you several options for changing the grid, and for determining the way objects align themselves on it. To do this, open the Grid Configuration box by choosing **Options/Grid settings...**

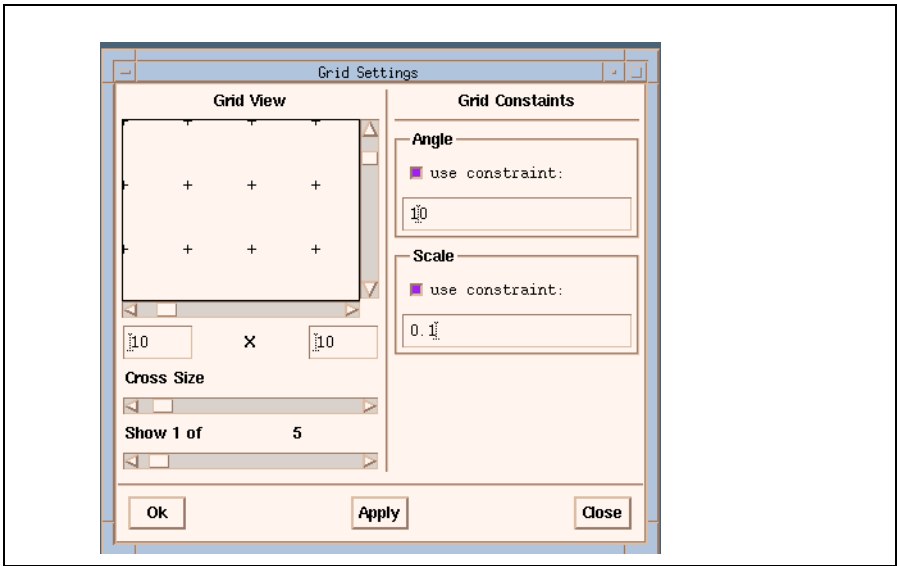


Figure 2.3: The Grid Configuration Box

Specify your choices, then click on **Apply** to apply them, or click on **OK** to apply the choices and close the box.

The *scrollbars* allow you to change the number of pixels—maximum value 91—separating the points of the grid vertically and horizontally. Use the sliders to choose the spacing of the grid points, then click **Apply** or **OK**. The values are displayed in boxes just below the grid viewing box. You can also enter the values directly into the boxes and click **Apply** or **OK**.

There are two additional scrollbars below the grid viewing box.

- The *Cross size* scrollbar allows you to change the size of the grid points.
- The *Show 1 of* scrollbar lets you choose how many of the possible crosses are to be displayed. Choosing “Show 1 of 1” means that *all* the points of your grid will be displayed on the screen.

When the ***Snap to Grid*** item in the Options menu is set, any object that you draw, move or resize “snaps to” the nearest grid point.

You can set constraints that define an object’s behavior when you rotate or scale an object. In the Grid Constraints field:

- In the Angle field, set the *use constraint* checkbox and enter a value in the input field. This value determines the rotation angle increment as measured in the ***Angle unit*** specified in the pulldown menu at the bottom of the main window.
- In the Scale field, set the *use constraint* checkbox, enter a value in the input field. This value determines the scale factor increment for an object.

Practice

Select a 10 x 10 grid, show 1 of 4 points of the grid, click OK, and select the *Snap To Grid* option from the Options menu.

2.5 Working with Objects

XDrawMaker is an *object* based program: each geometric figure, image or text you create is treated as a separate object. An XDrawMaker (`.draw`) file consists of one or more objects; i.e., rectangles, text, images. These objects can be grouped together and treated as one, or can be treated separately. All these objects are, in fact, Xmgadgets that are placed in an XnslDraw widget.

2.5.1 Using the Escape (osfCancel) Key

The `osfCancel` key, which is usually the ***Escape*** key, can be used to interrupt most actions in XDrawMaker. To abort an action that you have not yet completed, press the ***Escape*** key *before* you release the mouse button you are using to perform the action.

For example, when you are drawing an object by dragging it out on the desktop, pressing the ***Escape*** key before you release the mouse button aborts the action. The object is not drawn and the icon is deselected.

You can use the ***Escape*** key to abort the following actions:

- drawing and selecting objects and text.
- rotating and scaling objects (including pre- and post- scale).
- moving, pasting, and resizing objects.
- adding and removing slots.
- zooming.

2.5.2 Drawing Objects

Drawing objects is achieved through the use of the Object Icons. They are located at the top of the icon bar on the left side of the main window. Using the object icons is quite simple: to select an icon, simply click on it. Then click in the desktop where you want the object to be inserted. For a line, you would select the position of each end; for a rectangle or an ellipse, you would select the top left hand corner and then the bottom right hand corner. Chapter 3 explains in detail how to use the object, graphics and editing icons.

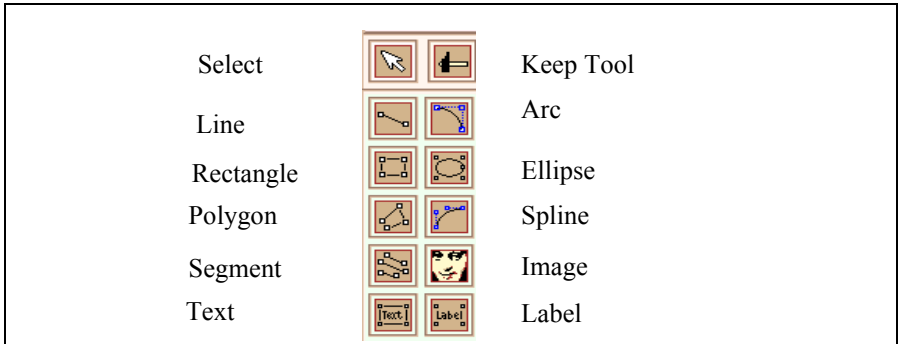


Figure 2.4: The Object Icons

Practice:

Draw six objects in the desktop, as shown in Figure 2.5: an ellipse, a label placed inside the ellipse, three squares, and a text object. To draw the ellipse and the squares, select the corresponding icon and then select the top left hand corner and then the bottom right hand corner.

To draw the text object, select the corresponding icon and then select the top left hand corner and then the bottom right hand corner. Place the cursor inside the rectangle of the text object and type the text you wish to see, in this case "Automatic Mode".

To draw the label object, select the corresponding icon and then click the top left hand corner and then the bottom right hand corner. Select *Label Box* in the Object Boxes submenu of the Options menu (or *Label Options* in the Object icons) to bring up the Label box. This box is described in Section 3.7.4. Change the text of the label to "Click Here", set the Text Alignment to center and apply your changes by clicking on the *OK* button.

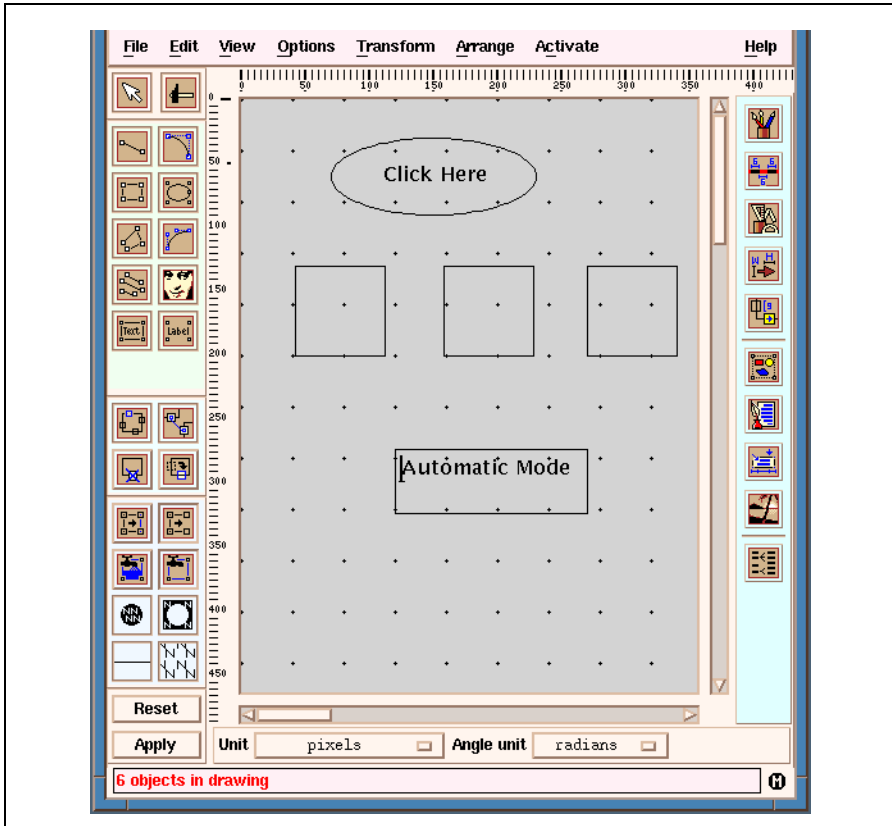


Figure 2.5: Drawing Objects

2.5.3 Naming Objects

Every object is automatically named when it is created, but you can change that name at any time in the Tree window. To pop up the Tree window, select **Options/Tree...** (or click the Tree in the Object icons). To change the name of an object:

1. Double-click on the object's name in the Tree. A text box and text cursor appear.
2. Delete the current name by double-clicking on the name, then pressing **Delete** or **Backspace**.
3. Enter a new name and press **Return**.

Practice:

Change the objects' names to:

"Ellipse" for the ellipse object.

"Label_Click" for the label object.

"Box_1" for the first box object.

"Box_2" for the second box object.

"Box_3" for the third box object.

"Text_Auto_Step" for the text object.

2.5.4 Linking Objects

Objects can be linked so that the connections remain even when the objects are moved. Slots are the attachment points used to connect one object to another. The connecting link can be a line, arc, polygon, spline, or segment. Although a slot *belongs* to an object, the slot itself does not have to be on, or even near, the object it belongs to.

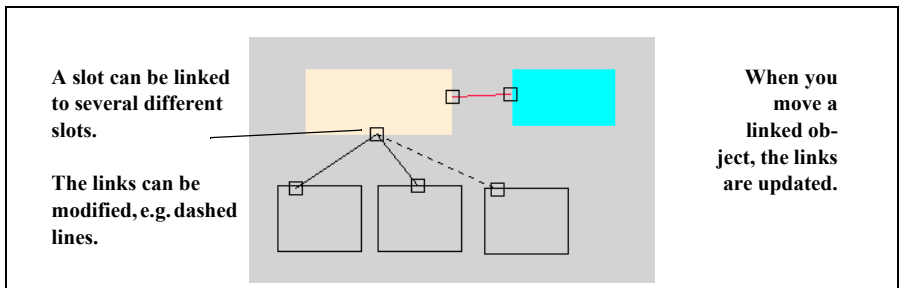


Figure 2.6: Drawing Slots and Links

Use the **Slots** icon to position a slot for a selected object. Use the **Link** icon to connect the slots.

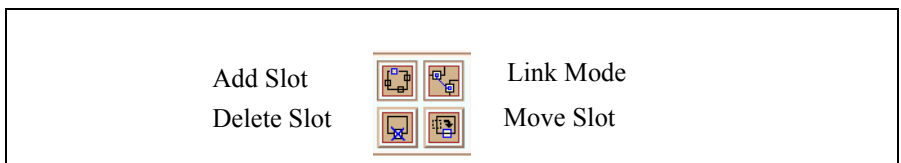


Figure 2.7: The Slot and Link Icons

You **must** select an object before you attach a slot to it. This is because a slot *belongs* to an object even if the slot is not on or near the object. If you try to attach a slot to an un-selected object, the slot will appear wherever you place it, but it will be attached to whichever object is selected at that time. You cannot create a slot if no object is selected. If you attempt to do so, XDrawMaker doesn't create a slot.

To connect two objects using slots and links:

1. Select the first object.
2. Click on the Slot icon, move the pointer over the desktop to the position where you want to place the slot, and click. The slot does not have to be on or near the selected object.
3. Select the second object and attach a slot to it.
4. Click on the Link icon, then on one of the possible connection icons, i.e. the line, arc, polygon, spline, or segment. The other icons are dimmed because they cannot be used as links.
5. Click on one of the slots, then draw the link, with its end point on the second slot.

To delete a slot, click on the Delete Slot icon, move the pointer over the slot, and click. When you delete a slot to which you had attached a link, the link's end point stays attached to the center of the object that the slot belonged to. To attach the link to another slot, you have to delete the link and create a new one.

If you delete an object that had slots and links, the link remains, although the slot belonging to the deleted object has disappeared. You can move the link's end point freely.

To move a slot, click on the Move Slot icon, move the pointer over the slot, and drag it to a new position.

You can resize slots and add images to them. Section 3.6.5 explains how to use the Slot box.

Practice

1. Add a slot to the right hand side of the left square.
2. Add a slot to the left hand side of the right square.
3. Add a slot on each side of the center square.
4. Link the left and center squares using a line.
5. Link the center and right squares using a line.
6. Change the links' names into:
7. "Link_1" for the first link object.
8. "Link_2" for the second link object.

The drawing should now look like this:

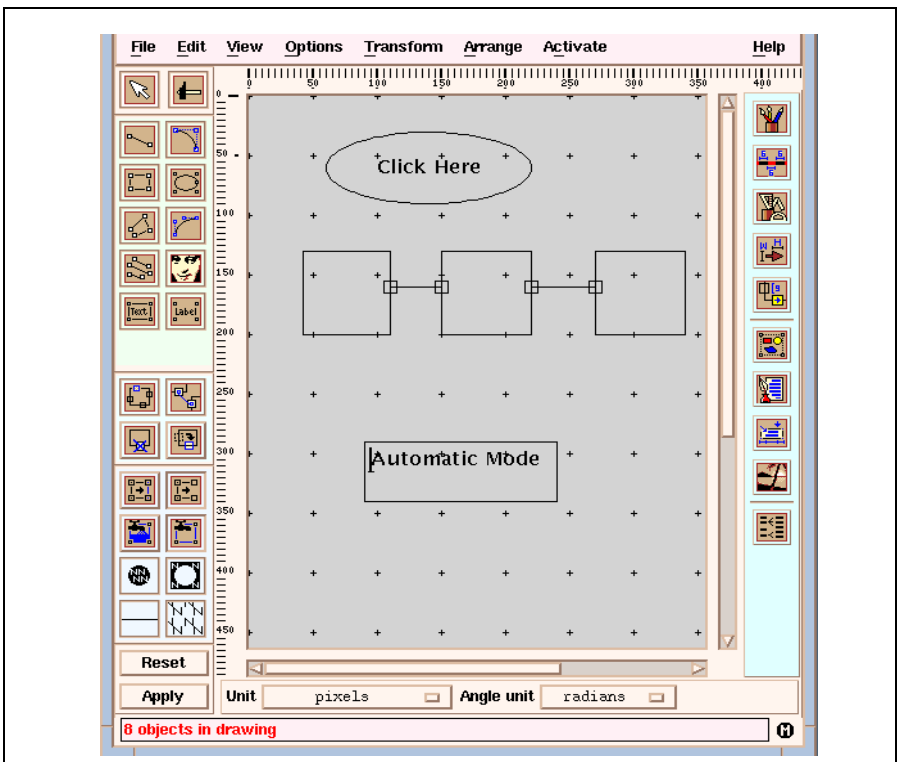


Figure 2.8: Creating Links

2.5.5 Selecting Objects

To work with objects you must first select them. Selected objects have *handles*—small black boxes—at each corner.



A selected object's *handles* can be used to resize or reshape an object. An ellipse also has two special handles, Start and End, which are shown in Figure 3.4.

There are several methods for selecting objects or groups of objects.

- Click anywhere inside a filled object, or position the pointer on the border of an empty object and click.
- Control-click (i.e. press the *Control* key and click) on the object to *add* it to the selection.
- Drag a rubber band box around the object or group of objects. To do this, place the pointer on the desktop above and to the left of the object, and press the *Select* button on the mouse. The pointer changes to a cross. Drag the pointer down until the rubber band box completely surrounds the object or group of objects, then release.
- To select *all* the objects on the desktop, choose **Select All** from the Edit menu.
- When objects overlap, you can:
 - m Select objects in the Widget Tree. This is useful if you want to select an object without changing its place in the stack.
 - m Use the **Circulate Up** or **Circulate Down** commands in the Arrange menu to bring the object to the top.
 - m Use Shift-Control-click to select the object beneath the selected object. Do this repeatedly to descend in the stack of objects.

To deselect an object:

- Click anywhere outside the selected item or group.
- Draw a new object.
- *Ctrl-click* on the (selected) object.

2.5.6 Grouping Objects

You can *group* the objects you draw so that they can be treated as a single object. This can be a *temporary* or *permanent* grouping. You can also create a group which is a collection of groups.

- To group objects temporarily, draw a rubber band box enclosing the objects *completely*. Each object will display its own selection handles. To disband a temporary group, simply click anywhere outside the objects.
- To group objects “permanently”, select the objects, then select the ***Selection to Group*** command in the ***Arrange*** menu.

Objects that are grouped “permanently” can of course, be easily disbanded. To create a “permanent” group, first select a group of objects, then choose ***Arrange/Selection to Group***. Note that the objects lose their individual handles, which are replaced by a set of *group handles*. To disband a permanent group, select it, then choose ***Arrange/Ungroup***.

A group can be moved or modified in many ways. Select it as you would select any object, then treat it as you would treat any single object. It is not possible, however, to resize a group by using its handles. Note that it is possible to apply graphic attributes, e.g. color or a fill pattern, to groups. The ways in which graphic characteristics are inherited by members of a group is discussed in Section 3.6.1.

2.5.7 Moving Objects

There are different ways to move an object. You can drag it to a new position using the mouse, or bring it to the top of a “stack” of objects. You can also select the object, then press **Shift** and one of the arrow keys.

To move an object or group of objects by dragging:

1. Place the pointer on the object outside the handles and press the *Select* mouse button: the pointer changes to a Greek cross (the *fleur* icon).
2. Drag the object to a new position, and release the mouse button.

To move an object or group of objects using the arrow keys:

1. Select the object or group.
2. While pressing the **Shift** key, press one of the arrow keys to move the object or group up, down, left or right.

The object moves one grid point at a time if the ***Snap to Grid*** option is set, or one measurement unit at a time, if ***Snap to Grid*** is not set.

2.6 Writing Scripts

XDrawMaker is based on the XnslDraw widget. In XDrawMaker, each object in a drawing is able to respond to events, such as pressing a key or mouse button. When the program is in “run” mode (i.e. when you select **Run** or **Launch** from the Activate menu) and an event occurs, the XnslDraw widget launches the script attached to the object, if the script’s conditions are satisfied.

To specify an event and the associated object’s subsequent action, you must write a script—a program written in the FACE language. The Event mask is entered automatically. You can use the Script Editor which is displayed by choosing **Activate/Script**. Or use the editor of your choice by selecting the **Custom Editor** button once you have opened the script editing window. Use the FMEDITOR environment variable to specify an editor. To use NSL’s Wx editor, specify wx242. FACE is described in Chapter 6.



Figure 2.9: The Script Editor

A script has the following format:

```
on action and location
Script Body
end
```

A script is launched if the conditions specified in the first line of the script occur. The *action* refers to one of the events listed in the *Actions* menu. The *action* can also be a translation in Xt form; for example:

`"<KeyPress>a"`

Note that the quotation marks must be included. Also, you have to add to the Event Mask any events you specify in translations.

The *location* is used to locate where the event occurs. If the event has no coordinate information, the XnsIDraw widget performs an XQueryPointer to locate the pointer. (See Xlib Reference manual.)

Scripts are written in the Script Editing box, shown in Figure 5.1. It contains two pulldown menus:

- **Functions** lists the XDrawMaker functions by category.
- **Actions** lists some of the events which can launch a script.

To write a script for an object, proceed as follows:

1. Select the object to be activated in your drawing.
2. Open the Script Editing box by clicking on *Activate/Script*.
3. Open the *Actions* menu by clicking on that item in the Script Editor.
4. Click on the name of the event that will launch the script. The script format is displayed.
5. Enter the location of the event. Possible locations are:
 - m **draw_nowhere** The script is never launched.
 - m **draw_inside** The script is launched if the event occurs inside the object.
 - m **draw_on_border** The script is launched if the event occurs on the object's border.
 - m **draw_outside** The script is launched if the event occurs outside the object.
 - m **draw_everywhere** The script is launched regardless of the location of the event.
6. Open the *Functions* pulldown menu to insert XDrawMaker functions into your script. When you click on a function name, it is inserted into the script at the insertion point, with dummy arguments which you need to replace with the appropriate values.
7. Click on the *Apply* button. This associates the script with the selected object. If there are errors in your script, a Message box will be displayed, describing the problem.

Practice

We will add scripts to the objects of the drawing, to define their response to various events. Let's start with the three boxes: We want them to move up the first time you click the *Select* button on them and to return to their original position the second time you click the *Select* button on them. Enter the following script for each box, changing the reference to `Box_1`, to `Box_2`, or `Box_3` in the corresponding scripts:

```
on Button1Down
  Float delta = 25.0;
  local toggle;
  /* Get the first box's id */
  C1 = XnslDrawNameToObject(parent, "Box_1");
  /* Move the box */
  if (toggle) {
    XnslDrawMoveObject(parent, C1, 0.0, delta);
    toggle = False;
  } else {
    XnslDrawMoveObject(parent, C1, 0.0, -delta);
    toggle = True;
  }
end
```

Try the Draw widget by launching the XDrawMaker Exec window with the *Launch a Separate Process* option in the Activate menu. Once the Exec window is ready, click on the boxes and watch how they respond to your actions. Notice how the links between the boxes are updated as the boxes move. Click on File/Exit to close the XDrawMaker Exec window.

Now, we want the three boxes to move once as soon as the draw widget appears. We will add a script that calls each of the three box scripts; this script has to be invoked when the widget is loaded. To do this, deselect all objects by clicking in the desktop in a free place and then enter the following script:

```
on Load
  function None Launch_Scripts (Widget w) {
    /* Launch the scripts of the three boxes */
    C1 = XnslDrawNameToObject(w, "Box_1");
    C2 = XnslDrawNameToObject(w, "Box_2");
    C3 = XnslDrawNameToObject(w, "Box_3");
    XnslDrawCallScript(w, C1, "Button1Down", NULL, $);
    XnslDrawCallScript(w, C2, "Button1Down", NULL, $);
    XnslDrawCallScript(w, C3, "Button1Down", NULL, $);
  }
  Launch_Scripts(self);
end
```

We now want to specify an action for the ellipse as follows: clicking in the area defined by the ellipse launches the scripts of the three boxes and moves the ellipse and the label. Select the ellipse, then enter the following script:

```
on Button1Down
  local toggle;
  Float delta = 150.0;
  /* Get the ellipse's and the label's ids */
  L1 = XnslDrawNameToObject(parent, "Label_Click");
  E1 = XnslDrawNameToObject(parent, "Ellipse");
  /* Change the label and move both objects */
  if (toggle){
    toggle = False;
    L1:drawLabelString = "Click Here";
    XnslDrawMoveObject(parent, E1, -delta, 0.0);
    XnslDrawMoveObject(parent, L1, -delta, 0.0);
  } else {
    toggle = True;
    L1:drawLabelString = "Or Here";
    XnslDrawMoveObject(parent, E1, delta, 0.0);
    XnslDrawMoveObject(parent, L1, delta, 0.0);
  }
  /* Call the function designed to move the three boxes */
  Launch_Scripts(parent);
end
```

Try the Draw widget again. Now only the "Automatic Mode" text object is lacking functionality. We want the boxes

to move continuously once the user has clicked on this text object and the text to be changed to "Step Mode". We also want to stop the motion and change the text back to Automatic Mode when the user clicks a second time in the text object. The continuous movement is achieved by a script which calls itself after a specified time interval. Select the text object and enter the following script:

```
on Button1Down
    local toggle;
    if (toggle) {
        toggle = False;
        /* Change the text to "Automatic Mode" */
        XnslDrawTextClearText(self);
        XnslDrawTextInsertAsciiText(self, "Automatic
Mode");
        XnslDrawTextChangeFont(parent, self, "**helvetica-
bold-
        r-* -14-*", 0, 15);
        XnslDrawTextSetAlignment(self,
XnslDRAW_TEXT_ALIGN_CENTER);
        /* Stop the AutoRepeat script */
        XnslDrawDisarmScript(parent, self, "AutoRepeat", 0);
    } else{
        toggle = True;
        /* Change the text to "Step Mode" */
        XnslDrawTextClearText(self);
        XnslDrawTextInsertAsciiText(self, "Step Mode");
        XnslDrawTextChangeFont(parent, self, "**helvetica-
bold-
        r-* -14-*", 0, 15);
        XnslDrawTextSetAlignment(self,
XnslDRAW_TEXT_ALIGN_CENTER);
        /* Run the AutoRepeat script */
        XnslDrawCallScript(parent, self, "AutoRepeat", NULL,
$);
    }
end
on AutoRepeat
/* Launch the scripts of the three boxes */
Launch_Scripts(parent);
/* Rerun this script */
```

```
XnslDrawArmScript(parent, self, 100, "AutoRepeat", 0, 0);  
end
```

Try the Draw widget again and combine the actions: click on the boxes or on the ellipse while in Automatic Mode.

2.7 Saving your work

There are two ways to save your work: choose **File/Save** or **File/Save As...** XDrawMaker can create a backup copy of each file you save. It has the same name as the copy you save, with a tilde (~) appended. To enable automatic backup, use the command line option or the backUp resource, as described in the appendix “Configuring XDrawMaker” on page 131.

Saving a new file. The **Save As...** command is used to save a *new* draw file or to save a copy of an existing file under a different name. When the File Selector appears, enter a name for your file in the *Selection* field, and click **OK**.

Saving a copy of an existing file. To save a *copy* of an existing file under another name, use the **Save As...** command. When the File Selector appears, enter the name of the copy in the “Selection” field, and click **OK**.

Saving an existing file. The **Save** command is used to save an *existing* file without changing its name. If you modify an existing file and then choose **Save**, the modified file is saved and no Alert box will remind you that you’ve made changes. The default file name is `unnamed.draw`.

Saving a combined file Two files that have been combined using the **Read** command (see section 2.10) can be saved as follows:

If you use the **Save** command, the combined file will be saved under the name of the first file. Say for example, that you are working on `file-1`. If you choose the **Read** command to add `file-2` to `file-1`, the **Save** command will save this combined file under the name `file-1`.

If you use the **Save As...** command for a combined file, the File Selector will be popped up, and the combined file will be saved under the new name you enter, e.g. `file-3`. `file-1` and `file-2` will remain as separate files.

Before saving your files, make sure you deselect all objects. Otherwise, when you re-load the file, objects that were selected will still have the selection handles, and XDrawMaker will not be aware that they are selected unless you toggle the selection.

2.8 Generating Files

You can generate three different types of file from your Draw file, using the ***File/Export*** command.

- ***XFaceMaker File.*** When you select this item, the File Selector pops up. Enter the name of the file you wish to generate as a .fm file, and click on the ***OK*** button. XDrawMaker will generate the .fm file which can then be used in an XFaceMaker application, or run through XFaceMaker to generate the C code equivalent of your draw file.
- ***PostScript File.*** When you select this item, the File Selector pops up. Enter the name of the file you wish to generate as a .ps file, and click on the ***OK*** button. XDrawMaker will generate the .ps file, which can then be printed on a PostScript printer. You can also use the Print Box to save a .ps file.
- ***Fax Group 4 File.*** When you select this item, the File Selector pops up. Enter the name of the file you wish to generate as a .tif. or .fax file, and click on the ***OK*** button. XDrawMaker will generate the file, which can then be used as a CCITT Group 4 fax.

2.9 Printing

You can print the current .draw file, or save it to a .ps file for later printing. To do so, open the Print Box by selecting **File/Print**.

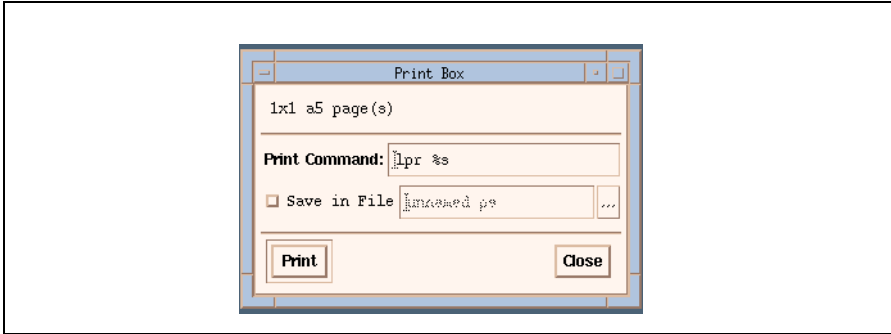


Figure 2.10: The Print Box

The size and number of pages is displayed at the top of the Print Box.

To print the current file:

1. Check that the Print Command is correct. The Print Command field lists the default command. You can enter a new print command in this field. You can also set a new default print command as explained below
2. Click on the **Print** button.

To save the current file to a .ps file for later printing:

1. Click on the **Save in File** checkbox.
2. Enter a name for the file, giving it the .ps suffix, and click **Print**. The file will not be printed, only saved as a PostScript file.

To save the file in a different directory, click on the ellipsis button to pop up the File Selector. Enter the directory and file name in the Selection field, then click **OK**.

You can specify a default print command when you launch XDrawMaker by using the `-printcmd` command line option. You can also set the `printCmd` resource in your `.Xdefaults` file to the desired value.

2.10 Opening an Existing File

Once you have the XDrawMaker main window on your screen, you can open an existing file or simply start drawing.

- To open an existing file select **File/Open**. The File Selector will pop up displaying the files available. Select one from the list and click. If you are currently working on a file, an alert box will warn you. Click on **Cancel** if you want to save the current file before opening a new file. Then re-select **Open**.

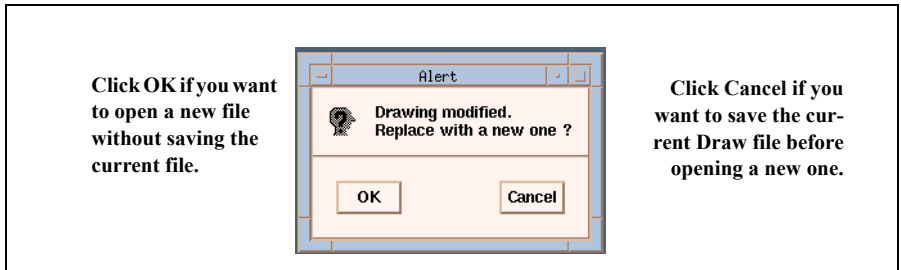


Figure 2.11: Alert to Save File

- To open a new file select **File/New**. The **New** command clears the desktop for your new file. It clears all images from the desktop, but not color attributes you have chosen. An alert box will warn you to save your current file before opening a new one.
- To *combine* a draw file with the one currently on your screen, select **File/Read**. The File Selector pops up. The file you select will be added into the current XDrawMaker window. This new combined file must be saved under a new name by using **Save As...** if you wish to keep a copy of the original file. See section 2.7.

CHAPTER 3: Drawing And Graphics

This chapter explains how to draw objects and add graphic characteristics to them. In XDrawMaker, drawings are created on the “desktop” using the Object icons, then given fill patterns and color using the Graphics icons or menu items. You can also set graphic attributes, such as line width or color, using the editing boxes. Open these boxes from the Options menu or by clicking on the corresponding Edit icon.

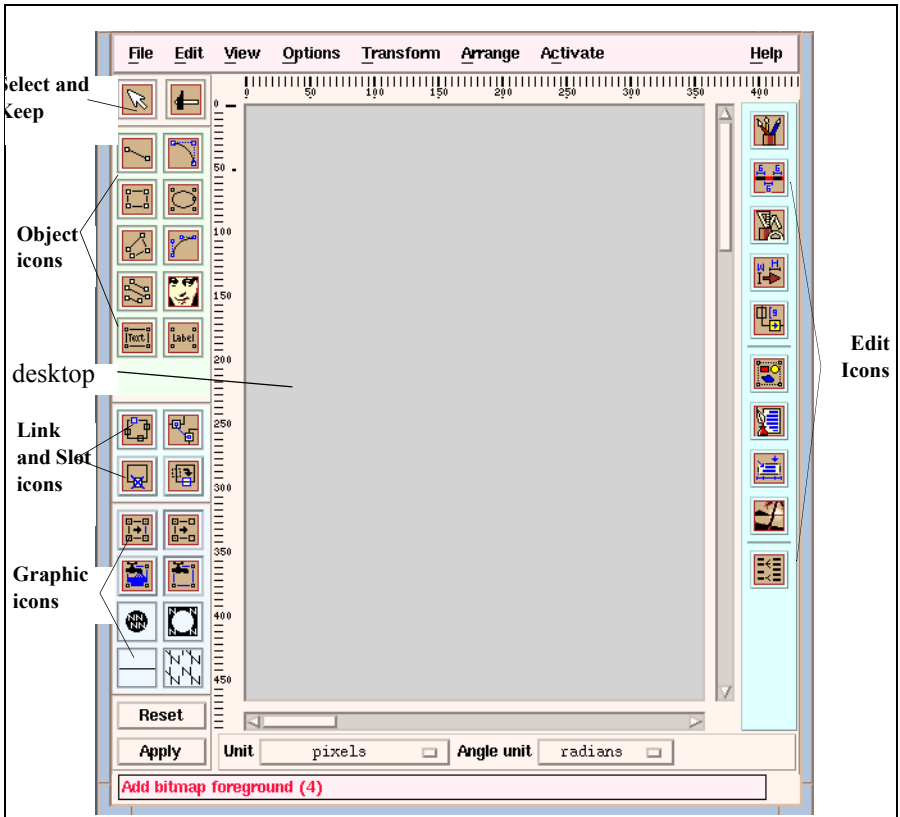


Figure 3.1: The XDrawMaker Main Window

3.1 Drawing Objects

The icon bar on the left side of the main window has two sets of icons, Object and Graphics icons, which you'll use to create your drawings. The Editing icons on the right hand side of the window represent editing boxes you can open by clicking on the corresponding icon. When the mouse cursor is moved over an icon, a brief message on its purpose is displayed in the status indicator line at the bottom of the window. Use this information to help you choose the icon you want if the icon image is not sufficient to clearly identify an icon's functionality.

To select an icon, simply click on it.

- **Object** icons are used to draw geometric objects, write text, or insert images into your Draw file.
- **Graphics** icons are used to modify the objects you draw by adding color, fill patterns, etc.
- **Editing** icons pop up the corresponding editing box.

There are two buttons below the icon bar. The **Apply** button applies your graphics choices to an object while the **Reset** button returns the Graphics icons to their default values.

The objects you draw are displayed with the graphic characteristics,(i.e., color, line width, fill pattern), that you have chosen using the Graphics icons or the Graphic Attributes box.

Your drawing will first appear, (until you deselect it), with *handles*, or small black squares located at each "corner" of the object, as shown in Section 2.5.5. They indicate that the object is selected and they can be used to resize or reshape the object. See Section 4.2 for more on handles.

Once you've drawn an object, there are several ways to modify it.

- Make changes using the Graphics icons. (See section 3.5).
- Specify values in the editing boxes. (See section 3.6).
- Rotate, resize, or scale it using the mouse or the options in the Transform menu. (See Chapter 4).

The status indicator line at the bottom of the window displays the latest command given.

3.1.1 The Object Icons

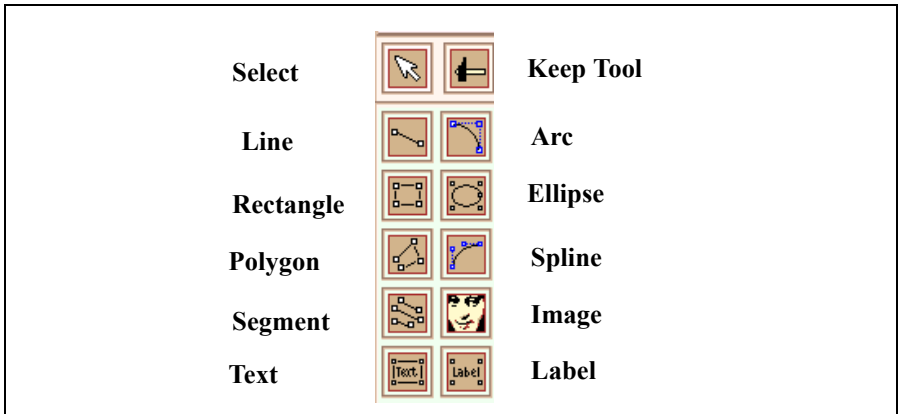


Figure 3.2: The Draw Icons

3.1.2 The Select and Keep Tools

The two icons at the top of the icon bar have special uses.



The **Select** icon is used for selecting and deselecting icons and objects.



The **Keep Tool** icon enables you to re-use the tool you're working with, without having to re-select it. When toggled, the icon displays a hand holding the tool.

For example, to draw a series of lines:

1. Click on the **Keep Tool** icon.
2. Click on the **Line** icon and draw as many lines as you wish.

You can then select another Object icon to draw with, or deselect the **Keep Tool** icon.

3.2 Geometric Objects

The Draw icons used for geometric objects are the **Line**, **Arc**, **Rectangle**, **Ellipse**, **Polygon**, **Spline**, and **Segment**. To use one of the icons described below, simply click on it, move the pointer over the desktop, and draw. Draw icons are highlighted when selected. Drawing an object, or clicking on the **Select** icon deselects the icon.

The object you draw will display the graphic values that you've chosen; for example, a polygon will be displayed as a closed figure if the **Close** icon is selected. To deselect an object, click outside of the object on the desktop. For more details on using the Graphics icons, see section 3.5.



To draw a **Line**:

1. Select the Line icon.
2. Move the pointer over the desktop to the starting point of the line, and click the Select mouse button.
3. Move the pointer to the end point of the line, and click again.

The width of the line can be changed using the Graphics icons or the Graphic attributes box. See section 3.5 to find out about modifying lines.



The **Arc** is used to draw curved objects. The curve is defined by placing three definition points on the desktop. To draw an arc:

1. Click on the Arc icon and move the pointer over the desktop.
2. Click once to mark the starting point of the curve.
3. Move the pointer to the next position and click. Move the pointer to the end point of the curve and click. This completes the figure.
4. Click on the desktop *outside* of the object to deselect the curve and view the completed object.

Three points define the curve of this selected arc.



When the object is deselected only the arc is displayed.

Figure 3.3: Drawing an Arc



An ellipse is created by drawing a rectangular rubber band box that defines its size.

1. Click on the **Ellipse** icon, then position the pointer on the desktop.
2. Press the *Select* button and drag until the rubber band box is the size you want.
3. Release the button and the ellipse will be displayed.

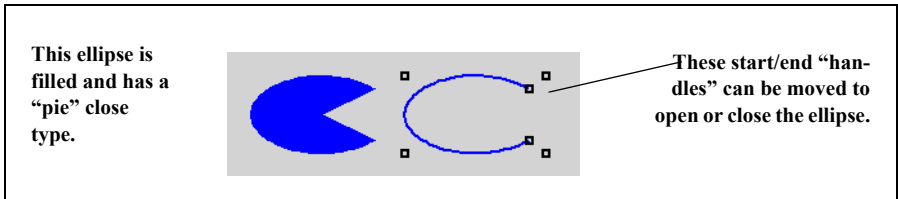


Figure 3.4: Drawing an Ellipse

An ellipse has two special *handles* that let you change the start or end angles of the ellipse with the mouse—provided that the **Close** icon is not selected. To use them:

1. Position the pointer over one of the start/end handles. It becomes a cross-hair cursor.
2. Press the *Select* mouse button, drag the handle to the new position, and release.



A rectangle is drawn like an ellipse. Select the **Rectangle** icon, position the pointer on the desktop, and drag the rubber band box to the size you choose. Release the *Select* button and the rectangle is displayed with the chosen graphic characteristics.



The **Polygon** lets you draw many-sided objects. To use it:

1. Click on the Polygon icon and move the pointer over the desktop.
2. Click once to mark the starting point of your object.
3. Move the pointer to the next position and click. Repeat until you have completed the figure.
4. Click the *Custom* mouse button to finish drawing, or click on the *Select* icon.



The **Spline** draws Bezier curves along a series of points. Polygons and splines are drawn in the same way although, of course, the objects created look quite different. When selected, a spline shows not only its handles but also its tangents, or “velocity controls”.



The **Segment** icon enables you to draw a series of disconnected lines which comprise a single object. You define the end-points of each segment as you would define a line---by clicking on the desktop at the position you want for each end-point. A segment must have an even number of points to define it. Click the *Custom* mouse button to finish drawing. The orientation of any of the individual lines in the segment can be changed by dragging one of the selection handles.

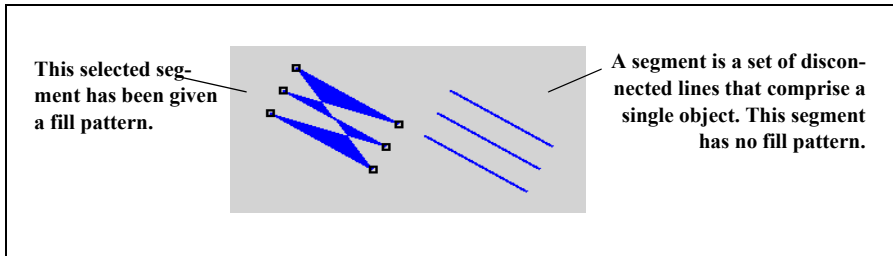


Figure 3.5: Drawing a Segment

You can move a segment just as you would move any object on the desktop, by pressing mouse Button 1 on the object and dragging the object when the move cursor appears.

You can also change the *orientation* of each individual line that comprises the segment by selecting the segment, then dragging the end-point of the line to a new position.

3.3 Grouping Objects

Objects can be grouped as described in Chapter 2. To create a group, drag a rubber band box around the objects, then select *Arrange/Group/Selection to...* To ungroup a selected set of objects, select *Arrange/Ungroup*.



Once you have created a group, you can edit certain of its attributes using the Group Edit Box. Open it by selecting *Options/Object Boxes/Group Edit Box*. Or click on the Group Edit icon.

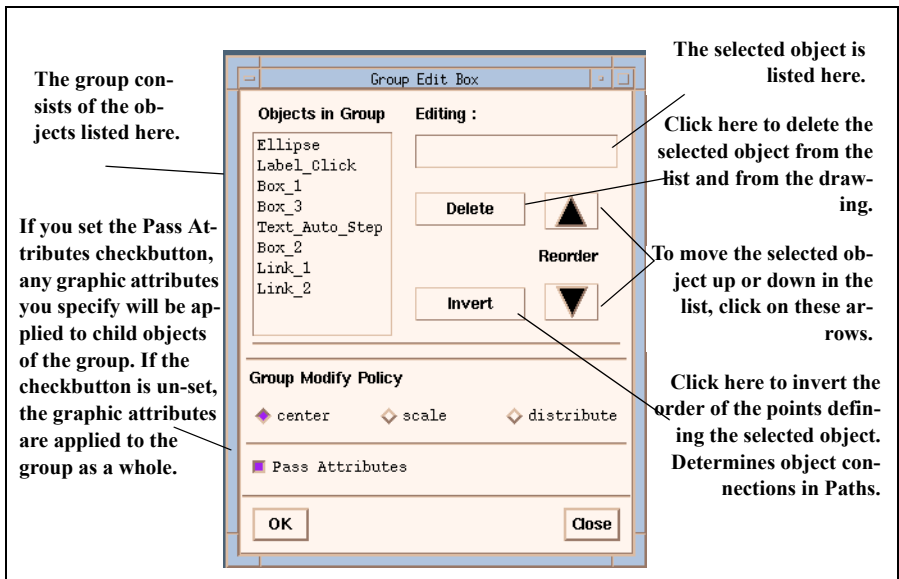


Figure 3.6: The Group Edit Box

The Group Modify Policy field lets you choose how the objects of a group will behave when the group is resized:

- **center** (default) Objects remain centered when the group is resized.
- **scale** Objects are scaled and centered when the group is resized.
- **distribute** When the group is resized, objects are not resized or scaled but are repositioned relative to the new dimensions of the group.

3.4 Creating Paths

XDrawMaker provides you with a convenient way to create complex shapes; for example, objects made from lines and curves. You do this by creating a *path* that connects the objects. Once the path connecting the objects is created, the objects become a single object, and can be used just as you would use any other object; for example, you can then move or rotate the new object, add fill attributes, etc. You can also modify the shape of the new object, because it contains all the points used to create the individual objects.

You can create a path from the following objects: lines, polygons, segments, arcs, and splines. XDrawMaker creates the path by connecting the individual objects with lines. The path follows the *order* and *direction* in which you created the individual objects.

- The *order* in which you create the objects is important because the path is drawn according to the order in which those objects were created; i.e. the path starts at the first object created, then moves to the second, then the third, etc. You can use the Group Edit box to re-order the objects in your Path: select the Path, open the Group Edit box and select the object whose creation order you want to change, move it up or down in the objects list using the **Reorder** arrows.
- The *direction* in which you created each individual object is important because the path starts at the first point of an object, follows its shape until it reaches the last point, then connects to the *first* point of the *next* object. You can use the Group Edit box to change the direction used to draw a given object: select the Path, open the Group Edit box and select the object whose direction you want to change, click **Invert**.

To create a path:

1. Select the objects that you wish to connect.
2. Select **Arrange/Selection to Path**.

Once the path is created, you can modify the new object just as you would any other object. The new object is represented in the Tree as a single object composed of several objects.

You can undo a path:

1. Select the object.
2. Select **Arrange/Ungroup**.

3.4.1 The Tree

The Tree is a window that displays the objects you have drawn as a tree structure. If the objects have no hierarchical relationship, this window is simply a list of the current objects. If the objects are grouped, the window displays them as sub-items of the group.



To open the window click on **Options/Tree...**, or click on the Tree icon in the main window.

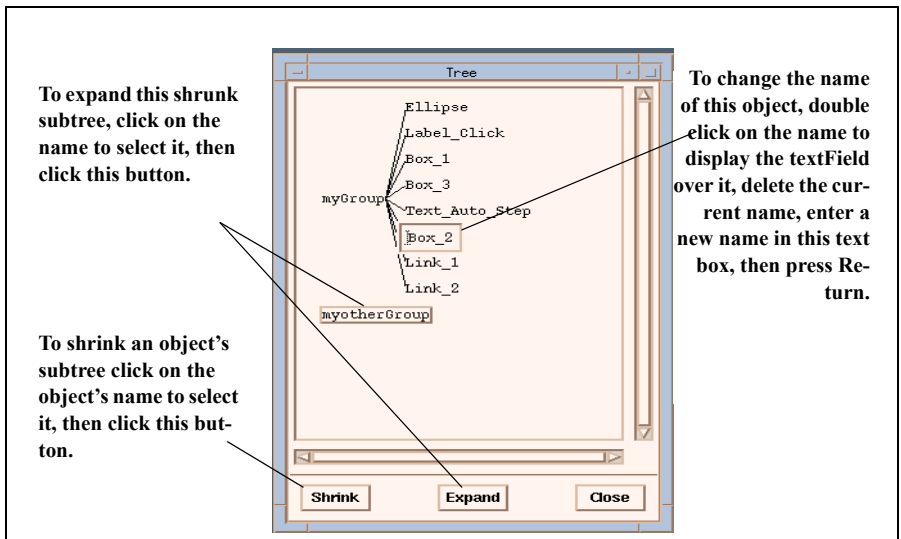


Figure 3.7: The Tree Window

- The **Expand** button displays all the sub-trees of a selected object.
- The **Shrink** button hides the sub-trees of the selected object.

Changing Object Names - You can use the Tree to change the name of an object. To do so:

1. Double-click on the object's name in the Tree. A text box and text cursor appear over the name.
2. Delete the current name by double-clicking on the name, then pressing **Delete** or

Backspace.

3. Enter a new name, and press ***Return.***



Note: the object name is not equivalent to the value returned by the *XtName* function.¹

¹. Since the Xt toolkit does not allow you to change the name of a widget, all *XnsIDrawObjects* have an *XnsIDrawName* resource which can be changed at any time. At initialization time, the Xt name, (which can be retrieved by the *XtName* function) and the *XnsIDrawName* are identical. See the description of functions such as *XnsIDrawNameToObject* for more detail.

3.5 Adding Graphics

XDrawMaker enables you to create very complex designs and to give them a variety of graphical characteristics. This section describes the ways you can modify your drawings using the Graphics icons, and the various editing boxes. The Graphics icons are used to:

- close or open objects
- fill or empty objects with a chosen color or pattern
- select foreground and background colors
- set line width, and create arrows and dashes
- choose a fill pattern for objects.

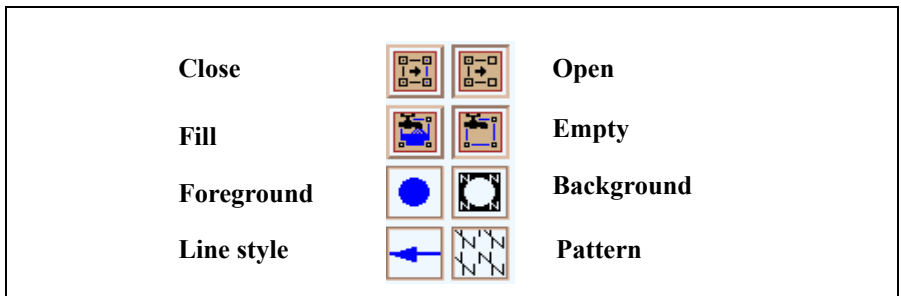


Figure 3.8: The Graphic Icons

Close and Open The Close icon closes an arc, ellipse, polygon, or spline. When you select an object of that type, then click on the Close icon, XDrawMaker will “finish” the object by drawing a line or curve between its start and end points—provided those points don’t already coincide. (See section 3.6.1 on closing modes: pie, chord, or full). Clicking on the Open icon will open the selected object between its start and end points. Likewise, if you draw one of the objects listed above while the Close icon is selected, the object will be closed by XDrawMaker.

Fill and Empty The Fill icon is used to fill objects with patterns or colors. Select an object, then click on the Fill icon. The object will display the fill patterns that you have chosen using the other Graphics icons or the Object Attributes box. Clicking on the Empty icon empties the selected object of any fill pattern or color. Likewise, if you draw an object while the Fill icon is selected, the object

will be filled with the current Fill pattern.



Foreground color This icon is used to set the current foreground color. It specifies the foreground color of any selected objects and of any objects being drawn. To set the foreground color, open the pulldown box by pressing the *Select* button on the icon. Drag the pointer over your choice of foreground color, then release. Note that when foreground color is applied to a group, any object that already has a foreground color will keep it, and the new foreground color will only apply to the other objects in the group.

The **Foreground color** choices are the following:

- Choosing **none** means that no color will be assigned to the foreground pixels.
- Choosing the **white** box assigns white to all foreground pixels.
- Choosing the **black** box assigns black to the foreground pixels and so on with **red**, **green** and **blue**.
- Choosing **New** pops up the **RGB** box for defining new foreground colors.

The **RGB** box, shown in Figure 3.9, consists of a series of slider bars for defining color mix, as well as the hue, intensity, and saturation of the color. The resulting color is shown in a viewing box at the bottom of the RGB box. You can also enter the *name* of the color or its *color value*—a hexadecimal number representing the RGB values—directly. Click on the **OK** button to apply the color choice to the selected object. The RGB dialog box lets you define colors using the *red*, *green*, *blue* representation or the *hue*, *saturation*, *value* representation for colors.



Background color This icon is used to set the current color of the background pixels in an object. It specifies the background color of any selected objects and of any objects being drawn. To set the background color, open the pulldown dialog box by pressing *Select* on the icon. Drag the pointer over your choice of background color. Note that when Background color is applied to a group, any object that already has a background color will keep it, and the new background color will only apply to the other objects in the group. The choices are the same as for the Foreground color icon, but are applied to the object's background pixels.

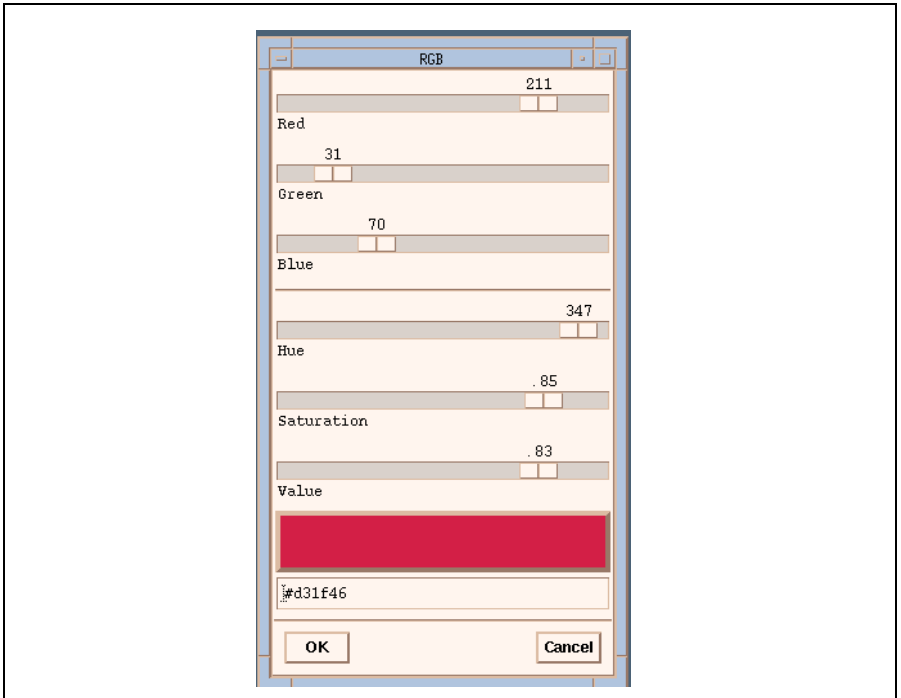


Figure 3.9: The RGB Box



Line Style This icon lets you choose the type and width of line you draw, and to create arrows. To use it, click on the icon. A pulldown box is displayed with three columns that let you set the current line type, line width, or type of arrows. Make a selection in the left column to choose a line type, in the middle column to specify a line width, in the right column to specify arrows. The box closes at each selection so you may need to open it several times to fully specify the lines you want. Any lines you draw will have the combined characteristics that you set in each of the columns. To apply current line characteristics to an existing object, select the object and click the *Apply* button.



Patterns Icon Use this icon to choose a fill pattern. Open the pulldown box by clicking on the Patterns icon. Click on the pattern you choose. If the Fill icon is selected, the pattern will be applied to any objects you draw and to

any selected objects. Choosing the New option pops up a dialog box for specifying a user-defined fill pattern. When a fill pattern is applied to a group, any object that already has a fill pattern will keep it, and the new fill pattern will only apply to the other objects in the group. To apply the current fill pattern to an existing object, select the object and click the ***Apply*** button.

3.6 The Attribute Boxes

The Attribute boxes enable you to define precise graphical characteristics for the objects you draw. These attributes can also be set using the Graphic icons. To open a box, select *Options/Attributes boxes....* Or click on the corresponding Edit icon.

- *Graphic Attributes*
- *Dash Specification*
- *Geometric Attributes*
- *Arrow Specification*
- *Slot Attributes*

3.6.1 The Graphic Attributes Box



This box is used to set graphic attributes for a selected object. To open it select *Options/Attributes boxes/Graphic Attributes...* or click on the icon.

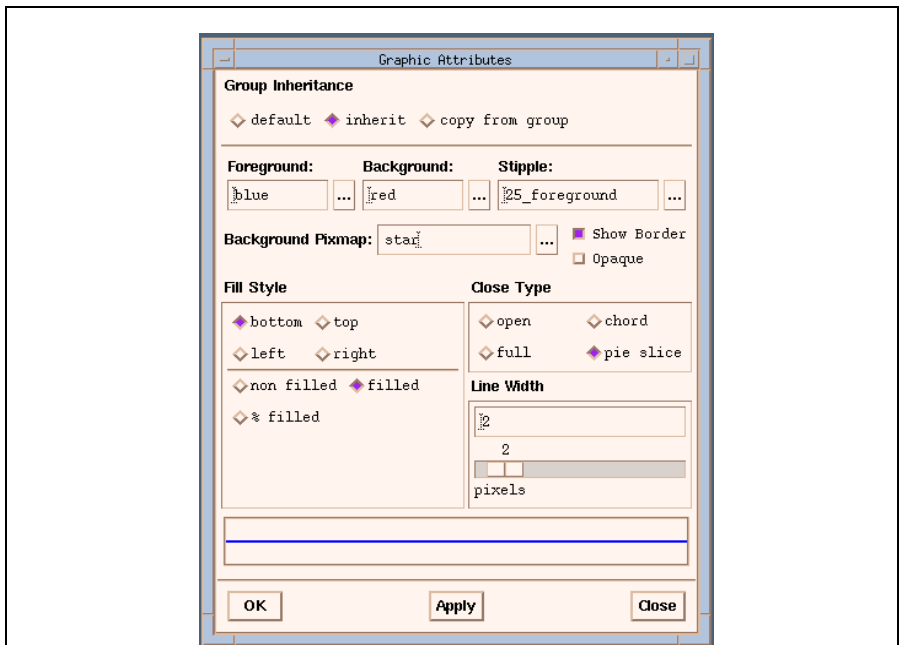


Figure 3.10: The Graphic Attributes Box

After entering changes in the appropriate fields, you can apply the changes to the selected object by clicking on the **Apply** button at the bottom of the box. Or click on the **OK** button to apply the changes and close the box. Click on the **Close** button to close the box without applying any changes.

The fields in the Graphic Attributes box are the following.

Group Inheritance Select the way in which the object will inherit its graphic attributes if it belongs to a group.

- m *default* The object does not inherit attributes from the group.
- m *inherit* The object inherits only the attributes it does not already have.
- m *copy from group* The object inherits attributes from the group, even if it already has values specified for those attributes.

Foreground color This item is used to set the foreground color for an object. Enter a color *name* or color *value* (e.g. #e453fc) in the input field or click on the ellipsis to pop up a menu giving you color choices. Click **Apply**. When a foreground color is applied to a group, the Group Inheritance option selected determines how the color is applied to the objects in the group.

The **Foreground** choices are the following:

- m Choosing **none** means that no color will be assigned to the foreground pixels.
- m Choosing the **white** box assigns white to all foreground pixels.
- m Choosing the **black** box assigns black to the foreground pixels and so on with **red**, **green** and **blue**.
- m Choosing **New** pops up the **RGB** box for defining foreground colors.

The **RGB** box, shown in Figure 3.9, consists of a series of slider bars for defining color mix using the *red*, *green*, *blue* representation, or the *hue*, *saturation*, and *value* representation of the color. The resulting color is shown in a viewing box below the sliders. You can also enter the color name or the color value—a # followed by a hexadecimal number representing the RGB values, e.g. #e453fc — directly into the text input field at the bottom of the box. Click on the **OK** button to apply the color choice to the selected object.

Background This item is used to set the color of the background pixels in an object. Enter a color in the input field or click on the ellipsis to pop up a selection menu. Click *Apply*. When a background color is applied to a group, the Group Inheritance option selected determines how the color is applied to the objects in the group. The color choices are the same as for the Foreground color icon, but are applied to the object's background pixels.

Stipple Use this item to choose a fill pattern or stipple for a selected object. Enter a stipple name or choose a pattern from the pop up menu. Click *Apply*. If the Fill icon is selected, the pattern will be applied to the object. Choosing the New option pops up a dialog box for selecting a user-defined fill pattern. When a stipple is applied to a group, the Group Inheritance option selected determines how the stipple is applied to the objects in the group.

Background Pixmap Enter the name of a pixmap file here, or click on the ellipsis to pop up the File Selector. Click *Apply*. The pixmap will be applied to the selected object.

Show border When set and applied, the borders of the selected object are displayed.

Opaque When set and applied, the object is filled with the specified background color. The object is thus opaque, and will hide objects underneath.

Fill Style This field lists the following choices for a selected object. Choose one and click *Apply*.

- m **non filled**, like the Empty icon, displays the selected object with no fill color or pattern.
- m **filled**, fills the selected object with the colors and patterns you choose.
- m **% filled** enables you to choose a percentage and zone of an object to be filled. If you choose this option, choose the *percentage* of fill by using the slider beneath the *% filled* field.
- m **bottom, top, left, right** These items specify the direction from which to fill the selected object. For example, if you choose to fill 50% of the object, and you select "bottom", the bottom half of the object will be filled.

Note that the choice of patterns or colors that fill an object, will depend on the choices you've set with the Graphics icons or in the *Colors* field of the Object Attributes box.

Close Type This field gives you four choices for closing an object (ellipse, polygon, spline, and arc). Choose a close type and click *Apply*.

- m **open** leaves the object open between its *Start/End* handles. See section 3.2.

m **full** completes the object.

m **chord** draws a line between the *Start* and *End* handles of the object.

m **pie** creates a pie figure by drawing lines from the center point of the object to the *Start* and *End* handles. Note that the center point of the object can be redefined and the pie figure will always close to the center point, wherever the center is placed.

Line Width Allows you to choose the line width, in pixels. You can also use the slider to specify the line width. Click ***Apply***.

3.6.2 The Geometry Box



This box lists a selected object's position on the desktop, as well as its scale and angle of display. To open it select *Options/Attributes box/Geometry Box...* or click on the *Geometric Attributes* icon.

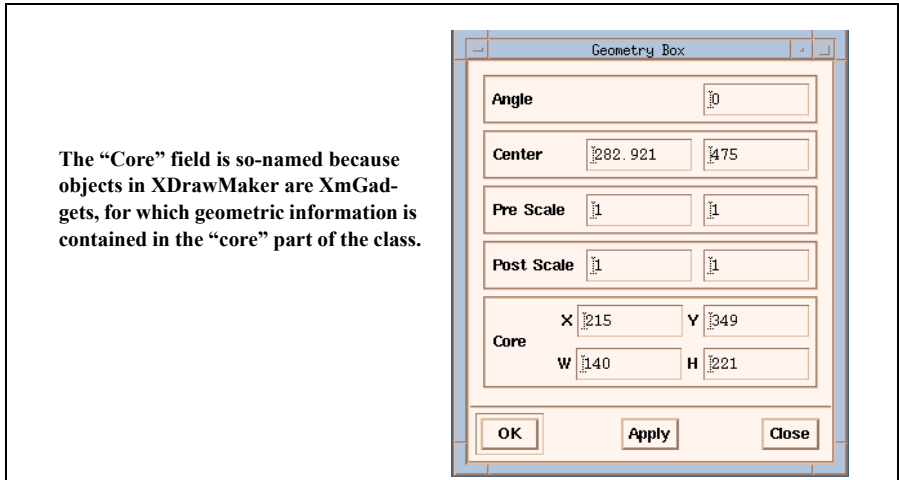


Figure 3.11: The Geometry Box

You can change the shape and position of a selected object by typing the desired values in the corresponding field and clicking **OK** or **Apply**. These values will also change if you rotate the object or choose other options in the **Transform** menu.

- **Angle** displays the angle of the selected object, with “0” as the object’s angle before transformation. The unit of measurement is that specified in the main window. The direction of measurement is clockwise.
- **Center** Displays the “x” and “y” coordinates of the selected object’s center point on the desktop, i.e. the reference point for geometric transformations. The unit of measurement is that specified in the main window. The center point need not be in the middle of an object.
- **Pre Scale** displays the “x” and “y” scales of the selected object, before rotation. The default value is 1.
- **Post Scale** displays the “x” and “y” scales of the selected object, after rotation. The default value is 1.
- **Core** displays the object’s position and size (x,y and w,h), in pixels.

3.6.3 The Arrow Box



This box is used to specify the style of arrow used to terminate a line, spline, or polygon, and to specify the type and size of its arrowhead. To open it select *Options/Attributes boxes/Arrow Box...* or click on the *Arrow Specification* icon.

Click on the style of arrow you wish, set the width and height of the arrowhead, and click *Apply*. The *Reset* button resets the box to the default style. Arrows can also be created using the *Line style* icon. See section 3.5.

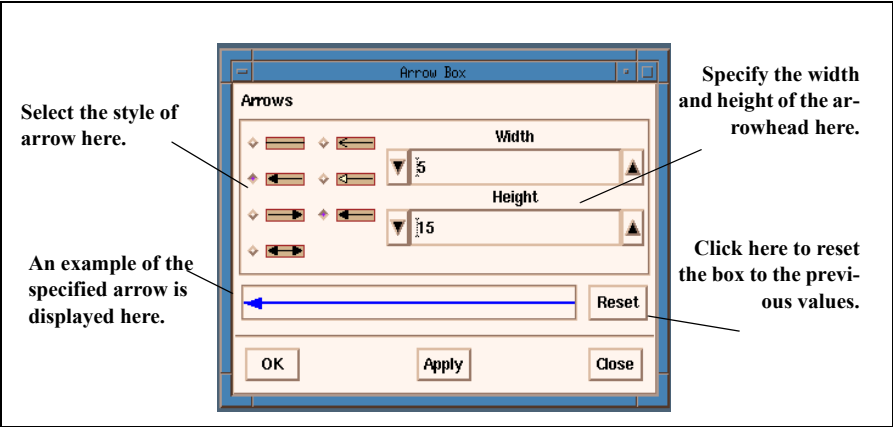


Figure 3.12: The Arrow Box

3.6.4 The Dash Box



The Dash box enables you to specify the style of dashed line. To open it select **Options/Attributes boxes/Dash Definition...** or click on the **Dash Specification** icon.

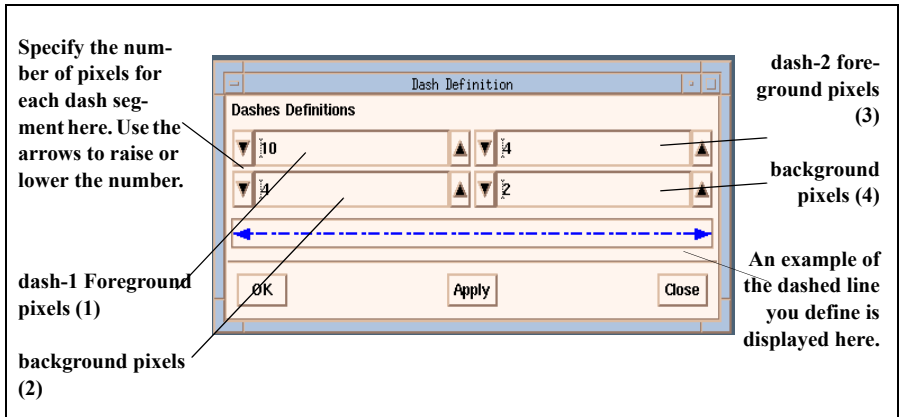


Figure 3.13: The Dash Box

This box enables you to define the style of a dashed line. In XDrawMaker, a dashed line consists of four segments which alternate between the foreground color and the background color. The foreground and background colors can be chosen using the Graphics icons or the Graphic Attributes box.

Choose the length of each segment, in pixels, by entering a value in the corresponding field. The four fields in the Dashes box define:

1. The number of foreground color pixels in the first segment.
2. The number of background color pixels in the second segment.
3. The number of foreground color pixels in the third segment.
4. The number of background color pixels in the fourth segment.

Simple dashed lines can be implemented using the **Line style** icon. See section 3.5



The Slot Box

To open the Slot Box, click on the icon or on the menu item **Options/**

Attributes Boxes/Slot Box... or on the *Slot Attributes* icon. In the Slot box you can:

- change the size of a slot.
- insert a pixmap into a slot.

Modify an individual slot by selecting it and changing it. Once you modify an individual slot, it will keep its definition until you **Reset** it to its default condition. To modify all the slots attached to a given object, select the object first, then make the modifications. If no slot is selected, the modification is applied to all the slots when you click **Apply** or **OK**.

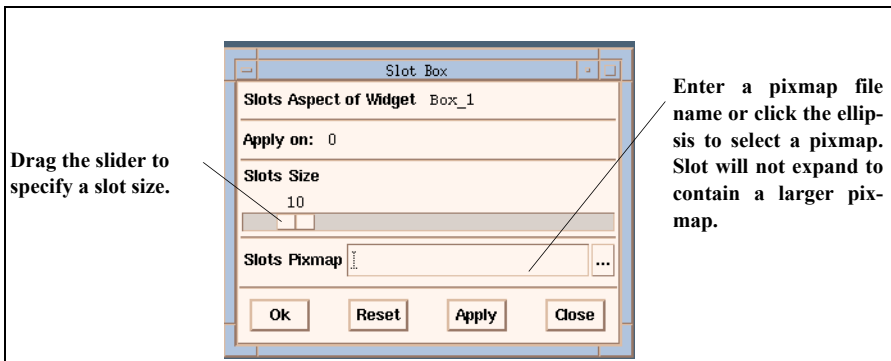


Figure 3.14: The Slot Box

- To change the size of slots, drag the slider. The unit of measurement is set in the main window in the **Unit** pulldown menu.
- To insert a pixmap, enter the filename in the Slots Pixmap field, or click on the ellipsis to pop up the File Selector and choose one there. The pixmap must be the same size as the slot, or only a part of the pixmap will be displayed. The pixmap will not resize to fit in the slot nor will the slot resize to accommodate the pixmap.

Once you have chosen the size or pixmap for the slots, click on **OK** to apply the choices and close the box, or click on **Apply** to apply the choices and keep the box open. Clicking on **Reset** returns all the slots of the selected object to their default condition. If only certain slots are selected, **Reset** applies only to the selected slots. **Close** closes the box without applying changes. Once the choices specified in this box are applied, any new slots you add to objects will have the size/pixmap specified.

3.6.6 The Expert Box

The Expert box is designed to be used by expert users of XDrawMaker. It lets you specify a resource and its value. It allows you to set resources that are not available in the XDrawMaker menus or boxes.



However, this can cause serious problems. If the resource values you set conflict with those defined by XDrawMaker, it may crash.

Open the box by clicking on *Options/Object Boxes/Expert Box....*

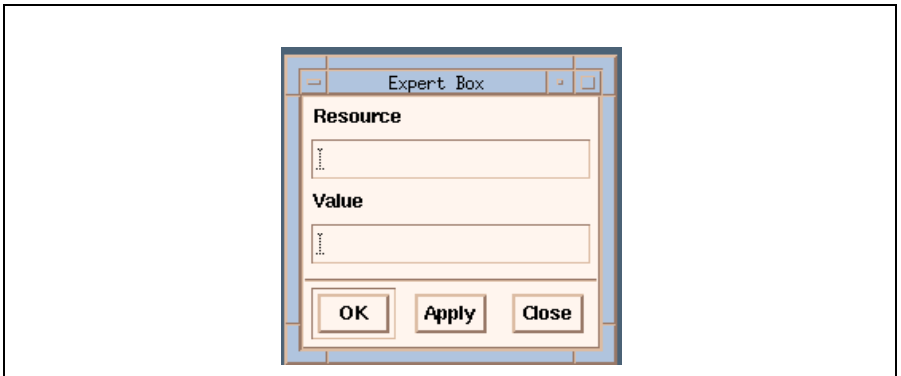


Figure 3.15: The Expert Box

Enter the name of the resource in the Resource field (without the XnslN, e.g. drawRadius, not XnslNdrawRadius) and a value in the Value field. Click **OK** to set the resource value and closes the box. Or click *Apply* to apply the value and keep the box open.

3.7 Entering Text

You can use the Text icon to enter text directly or the Label icon to display text. Using the Text object rather than the Label gives you more display options. However, the text you enter using the Text object cannot be transformed; i.e. rotated, resized, etc.

3.7.1 Using the Text Icon



To enter text using the Text icon:

1. Select the **Text** icon
2. Position the pointer on the desktop and drag out a rubber band box. When you release the mouse button, an insertion cursor is displayed.
3. Enter text at the insertion point.

The text box will not expand to hold the amount of text you enter into it. Words are automatically hyphenated to fit into the text box.

To modify text, you must first select it. To select text:

1. Select the text box by clicking on its border.
2. Place the pointer at the beginning of the text to be selected and click.
3. Drag the mouse to select the text. Selected text is displayed in inverse video.

3.7.2 Setting Text Options



The Text Options box provides you with a number of choices for displaying text. All measurements in this box are in pixels. The options are applied to selected text, or if no text is selected, the options will be applied to text you enter subsequently.

Open the Text Options box by choosing *Options/Object Boxes/Text Options...* or by clicking on the **Text Options** icon in the main window.

Click on **Char Settings Default** to choose default settings for the text object. Or choose your own preferences as shown below. Click the **Apply** button to apply the changes.

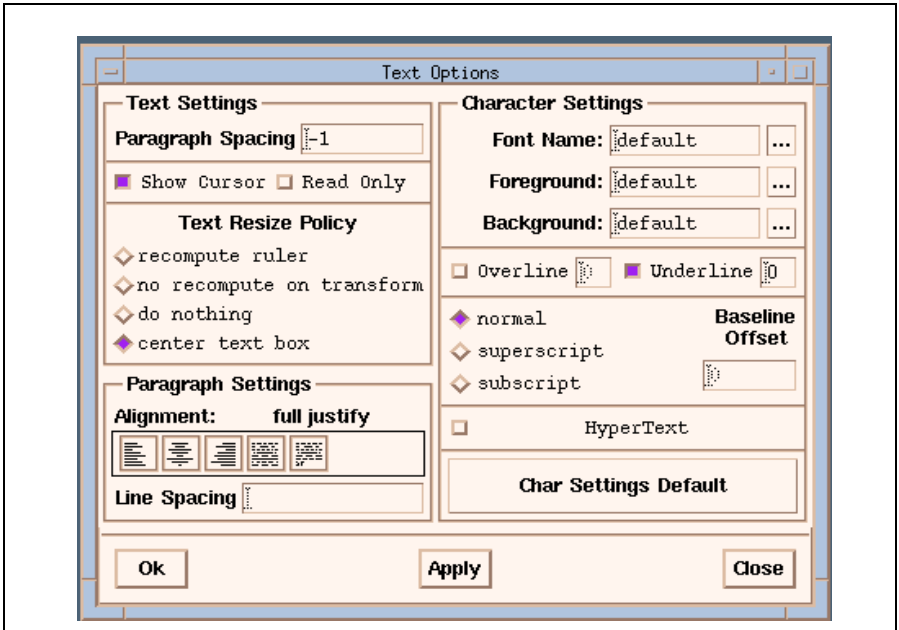


Figure 3.16: The Text Options Box

Paragraph Spacing Enter a value to specify the space between paragraphs, in pixels. If set to -1, the spacing is computed from the font used in the previous paragraph.

Show Cursor When set, the cursor is displayed at the insertion point.

Read Only When set, you cannot enter or delete text in the object.

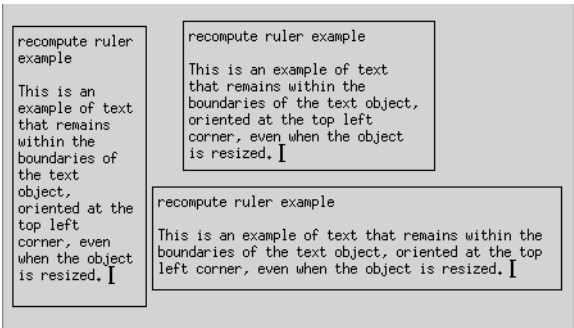
Text Resize Policy This field lets you specify how text behaves when you resize the text box. You have the following options:

- m **"recompute ruler"** The text is rearranged to fit in the new box at the upper left corner.
- m **"no recompute on transform"** This option is the same as **"recompute ruler"** except that the text remains unchanged if the object is transformed, i.e. rotated or scaled.
- m **"do nothing"** Text remains unchanged when the box is resized.
- m **"center text box"** The text is centered within the boundaries of the new box.

Examples

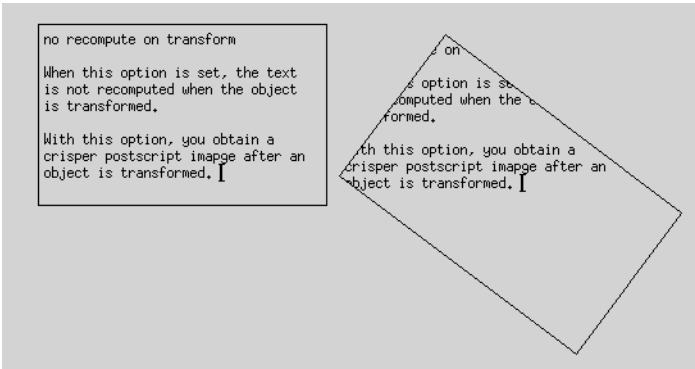
Some examples of the text resize policy options are displayed below:

“recompute ruler”



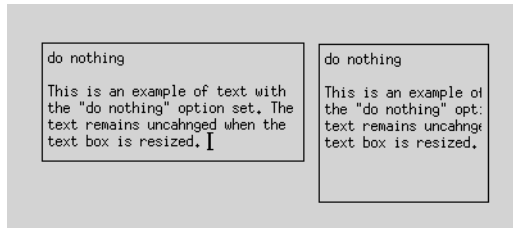
When this option is set, the ruler that controls the text settings (line width, paragraph spacing, etc.) is recomputed when the object is resized, so that the text will fit in the new box at the upper left corner.

“no recompute on transform”



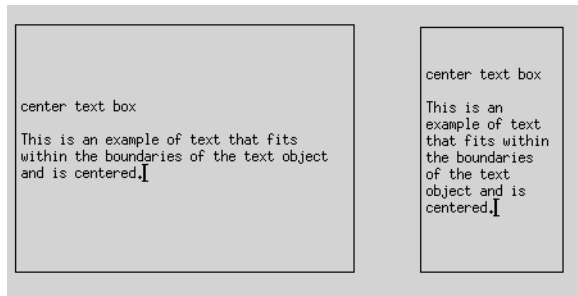
When this option is set, the text remains unchanged *when the object is transformed*; for example, when the object is rotated as shown in the example above. If the object is not transformed, this option treats text exactly like the option “recompute ruler”.

“do nothing”



When this option is set, the text remains unchanged when the box is resized.

“center text box”



When this option is set, the text is rearranged so that it fits within the boundaries of the new box and is centered.

Paragraph Settings These five icons let you justify selected text. Click on the icon to select it. Its function is listed above the row of icons. You have the following choices:

- m “**flush left**” Aligns text along the left hand border.
- m “**center**” Centers text within the borders of the text box.
- m “**flush right**” Aligns text along the right hand border.
- m “**justify**” Justifies text by adding space between words and between characters if the space between words is greater than 1 1/2 times the width of the space between characters---except on the last line which is flush left.
- m “**full justify**” Is equivalent to “justify” but includes the last line.

Line Spacing Specifies the spacing between lines, in pixels.

Character Settings If you have selected text, these settings will apply to that selected text. If no text is selected, these settings apply to the cursor so that all subsequent text that you enter in the object will have the font, foreground, and background you choose here. You have the following choices:

- m “**Font**” Enter a font for the text or click on the ellipsis to pop up the Font Box to choose a Font.
- m “**Foreground**” Enter a color name or click on the ellipsis to select a foreground for the text segment.
- m “**Background**” Enter a color name or click on the ellipsis to choose a background for the text segment.

Overline this checkbox creates a line above the text segment. You may enter a value in the box on the right which specifies the distance in pixels between the text and the line.

Underline this checkbox creates a line below the text segment. You may enter a value in the box on the right which specifies the distance in pixels between the text and the line.

Baseline Offset This specifies where the text is situated in relation to the baseline (normal cursor position). You have the following options:

- m “**normal**” The text is on the same level as the baseline.
- m “**superscript**” The text is entered above the baseline. The distance above the baseline must be specified in the box on the right. If it is 0, the text is level with *normal* text.
- m “**subscript**” The text is entered below the baseline. The distance below the baseline must be entered in the box on the right. If it is 0, the text is level with *normal* text.

Hypertext This check button changes the selected text segment to hyper text. In Run mode, the cursor will change to a hand cursor when moved over a hyper text segment.

Char Settings Default Clicking on this pushbutton resets the text options to their default values.

Clicking on the **Apply** button applies your choices to the currently selected text. You can then select different text, and change its settings, or close the box by clicking on the **OK** button.

If no text is selected, clicking on the **OK** button will close the Text Options box and apply your choices the *next* time you add text.

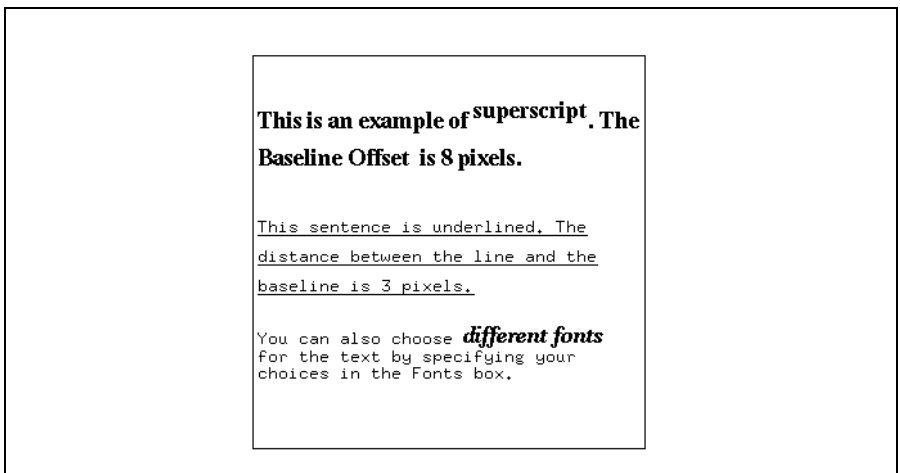


Figure 3.17: Text Attributes

3.7.3 Changing Fonts

You can select a font for text, and change the font’s slant, weight, and size in the Font Box.

 XDrawMaker uses the X server’s fonts and not its own. Therefore, fonts may differ according to the server you use, and some fonts may not be available on your system.

To change the font for selected text:

- 1. Select the text object, enter a font name in the Font Name field; if you do not know the font name you want, open the Fonts Box by clicking on the ellipsis in the Text Options Box and choose your font. Click **OK** to validate your choice and close the Font box.
- 2. Click **OK** or **Apply** in the Text Options Box. The selected text is now displayed with the chosen font.

When you apply a font, it becomes the *current* font, and all the text you enter in that object from then on is displayed in that font. Text can have multiple fonts. Simply change the *current* font and the new text you enter will be displayed with the new font---or select the characters you wish to change and specify the new font choices.

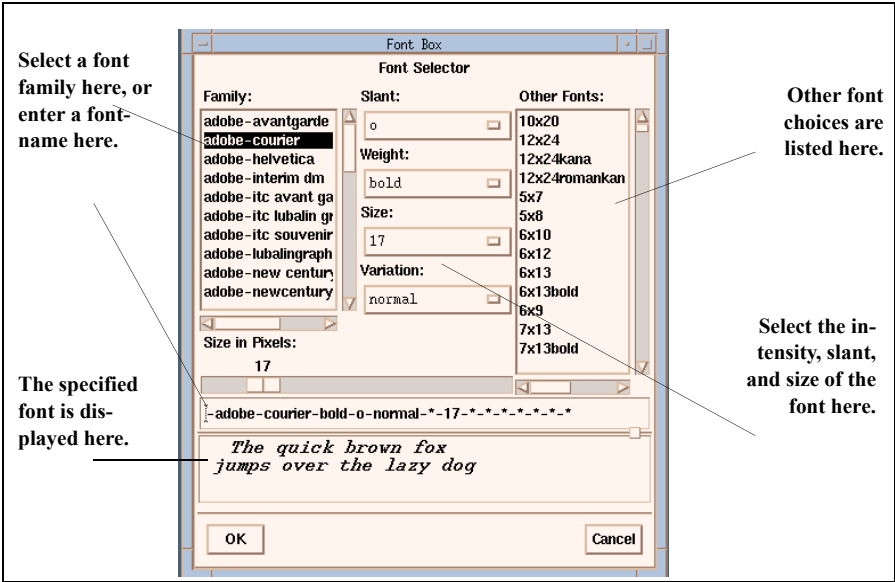


Figure 3.18: The Font Box

3.7.4 Using the Label



To enter text using the Label:

1. Select the Label icon, position the pointer on the desktop and drag out a rubber band box, and release the mouse button.



2. Open the Label box by clicking on **Options/Object Boxes/Label** or on the **Label Options** icon.
3. Enter your choices and click **OK** to apply them and close the box, or click on **Apply** to apply the choices and keep the box open.

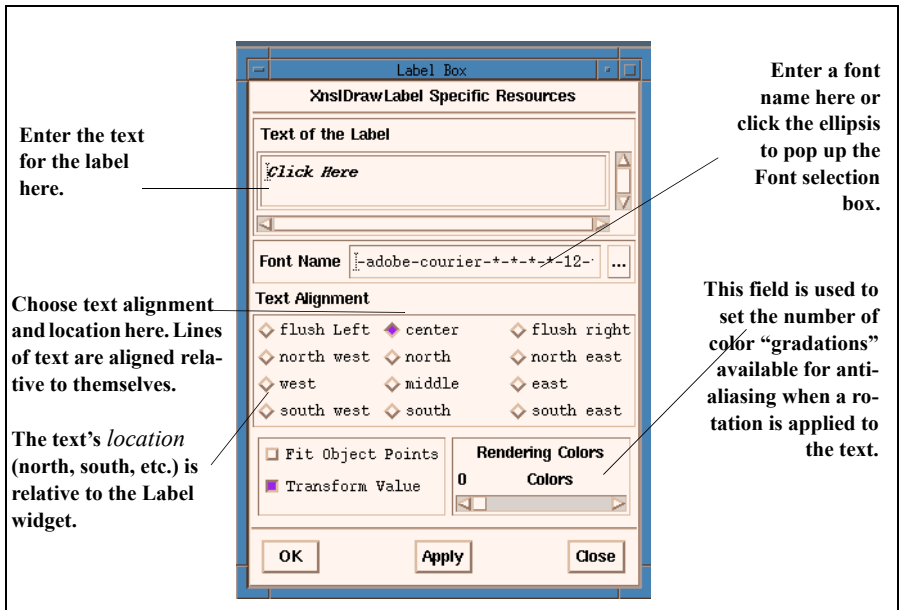


Figure 3.19: The Label Box

In addition to the label's text and font, the following options can be set:

- **"Fit Object Points"** When set, the object's size equals the text boundary box.
- **"Transform Value"** When set, the text will follow any geometric transformations that you make on the object; e.g. rotations.
- **"Text Alignment"** There are a number of options for aligning text within the label, as illustrated below.

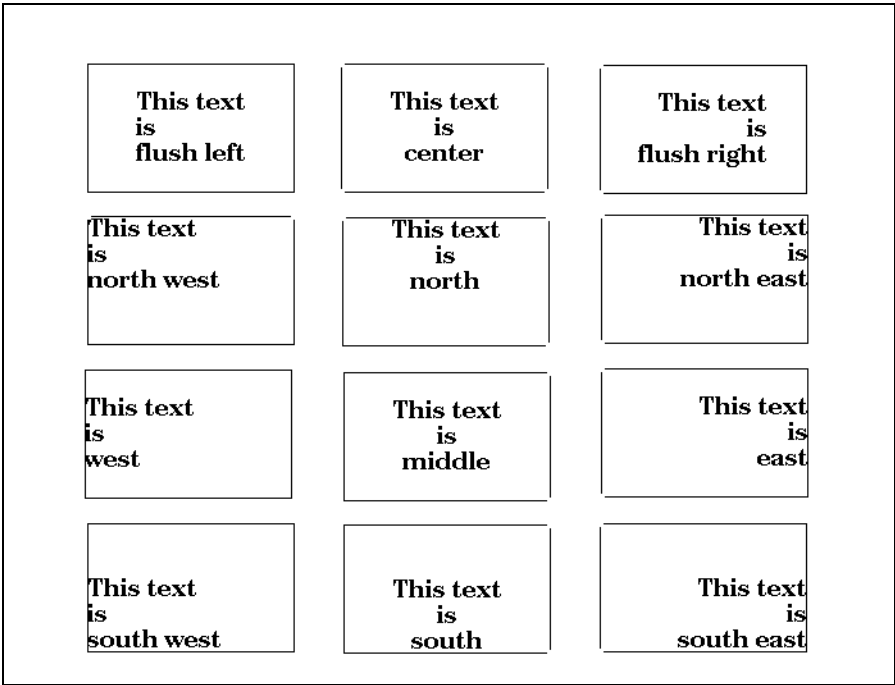


Figure 3.20: Examples of Text Alignment

To show the border of the label, set the “Show Border” checkbox in the Graphic Attributes box.

3.8 Inserting Images

You can insert images into your draw file using the Image Selector. XDrawMaker reads the following file formats: Xbitmap (`.bm`), GIF (`.gif`), XPMv.3 (`.xpm3`), and X Window Dump (`.xwd`). In general, working with images may take slightly more time and memory. You can pre-load a directory of images at startup time by using the command line option `-imgdir`.

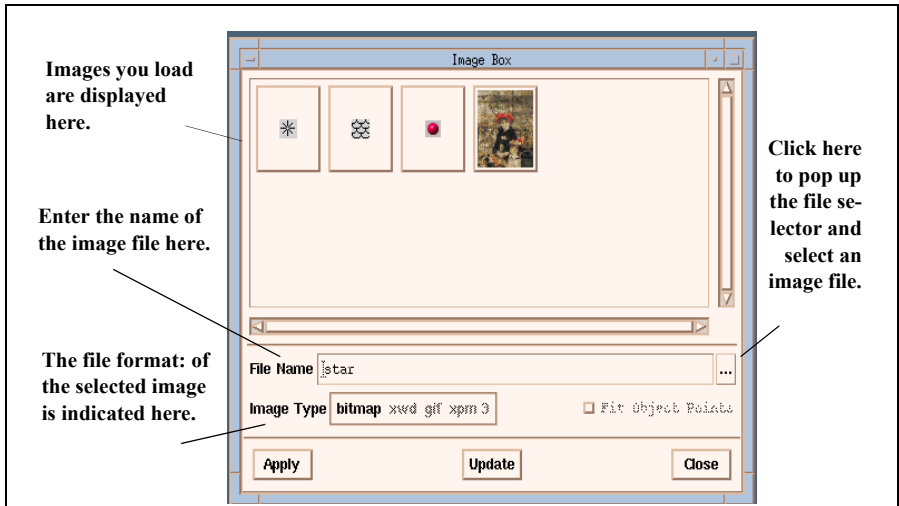


Figure 3.21: The Image Selector



Use the **Image** icon to insert images into your Draw file.

1. Select the Image icon, move the pointer onto the desktop, and drag out a rubber band box.
2. Open the Image Selection box by selecting **Option/Object Boxes/Images...**. Or click on the **Images** icon.



3. Enter the file name of the image you wish to insert into the Draw file or click on the image in the display box.
4. Click **Apply**. The image you chose will be displayed within the rubber band box, or the box will be resized to the size of the image, depending on the setting of the Fit Object Points toggle. The image is displayed at its original size, it is not scaled but clipped if the rubberband box is smaller than the image. You can move the image, rotate it, or scale it.

The ***Update*** pushbutton is used to update the display area so that the latest images you have opened are displayed.

When the ***Fit Object Points*** checkbox is set, the object's size is set to that of the image.

3.8.1 Color Allocation

How are colors allocated? XDrawmaker reads the image and tries to allocate all of its colors by order of appearance, starting with the colors that appear most frequently. If XDrawMaker cannot allocate all the colors, it will use the closest colors that have been allocated already. Therefore, colors will not necessarily match those defined for the image exactly.

When you select the ***New*** command, all images are deleted and the colors allocated by XDrawmaker are freed.

If XDrawMaker is launched with the RGB cube option (-urc page 132), then all the colors for images are taken from the RGB cube. If a color in an image does not exist in the RGB cube, the closest RGB cube color is used.

CHAPTER 4: Editing Techniques

Once an object is created, it can easily be cut or copied, moved around the desktop, rotated, resized, or rescaled, using the mouse and the XDrawMaker menus.

In general, you can use the Escape key to abort any editing action you are performing. Simply press the Escape key before completing the action--and before you release the mouse button you are using to perform the action.

4.1 Moving Objects

There are different ways to move an object. You can drag it to a new position using the mouse, or bring it to the top of a “stack” of objects.

To move an object or group of objects:

1. Position the pointer on the object (avoid the handles), and press the mouse Button 1: note that the pointer changes to a solid cross.
2. Drag the object to a new position, and release the mouse button.

The **Arrange** menu has options for moving objects that overlap.

- ***Bring to Front*** makes the selected object the “top” of the stack.
- ***Send to Back*** positions the selected object behind all the others.

You can also bring a selected object “up” one level, or send it “down” one level in a stack of objects using the ***Circulate Up*** and ***Circulate Down*** commands in the **Arrange** menu. As their names imply, these commands can be repeated as many times as desired and the selected object will eventually return to its original position. These commands allow you to circulate an object while maintaining its relative position in the stack of objects.

When you change the order of objects in the stack you change the order in which objects are created.

4.2 Resizing Objects

Objects can be resized easily using the mouse or the editing boxes.

To resize objects using the mouse:

1. Select the object and position the pointer over one of its handles. The pointer changes to a cross-hair cursor.
2. Press the mouse *Select* button, drag the pointer to resize the object, and then release.

You can resize text objects in the same way. The text within the text object changes according to the options you set in the Text Options box.

4.2.1 Adding and Removing Points

You can resize a selected polygon or spline by adding or removing definition points. To *add* points:

1. Choose **Add Point** from the Transform menu. The cursor changes to a small dark cross.
2. Position the cursor where the added point is to be placed, and click the mouse *Select* button. The new point appears and the lines of the original object are extended from the nearest points on the object to include the new point.
3. Click on the Select icon to deselect the **Add Point** option.

The **Remove Point** option works in a similar way, except that you should position the cursor next to the point you wish to remove from the object.

4.3 Rotating Objects

Objects or groups rotate around their *center* point—which can be redefined. Rotating an object can be done easily with the mouse, or with more precision, by entering a value in the *angle* field of the Geometry box. See section 3.6.2.

To rotate a selected object or group using the mouse:

1. Choose ***Transform/Rotate***. The pointer changes to the rotation symbol.
2. Position the icon anywhere on the desktop and press the *Select* button. While pressing the button, move the mouse. The object will rotate around its center point. To deselect the rotation option, click on the Select icon.

To reset the selected object to its original rotation angle choose ***Transform/Reset Rotation***.

By default, the center point of an object or group is the geometric center of the object or group. You can redefine the center point of an object or a *permanent* group using the ***Transform/Define Center*** command, or by entering a value in the “Center” field of the Geometry box. A temporary group—formed by drawing a rubber band box—cannot have its center point redefined. The center point can be positioned *anywhere* on the desktop; even outside the object or group.

To redefine the center point of an object or group using the mouse:

1. Select the object or group.
2. Choose ***Transform/Define Center***. The pointer changes to a small solid cross.
3. Position the cross at the new center point and click. A small *center mark* will appear to mark the new center position. The object or group can now be rotated around this new center point.

To change the center point of a selected object back to its original position, choose ***Transform/Reset Center***.

4.4 Scaling Objects

You can scale a selected object or group using the commands in the Transform menu or by entering the desired value in the Geometry box and clicking **Apply**. Objects and groups are scaled from their center point.

To scale a selected object or group using the mouse and menu commands:

1. Choose **Pre Scale** or **Post Scale** from the Transform menu. The pointer will change into the *scale* symbol.
2. Position the pointer anywhere on the desktop—inside or outside of the object, press the *Select* button, and drag the pointer to scale the object.
3. Release the button. Note that the scale symbol is still active. You can rescale the object as many times as you wish. Click on the **Select** icon to deselect the scale option.

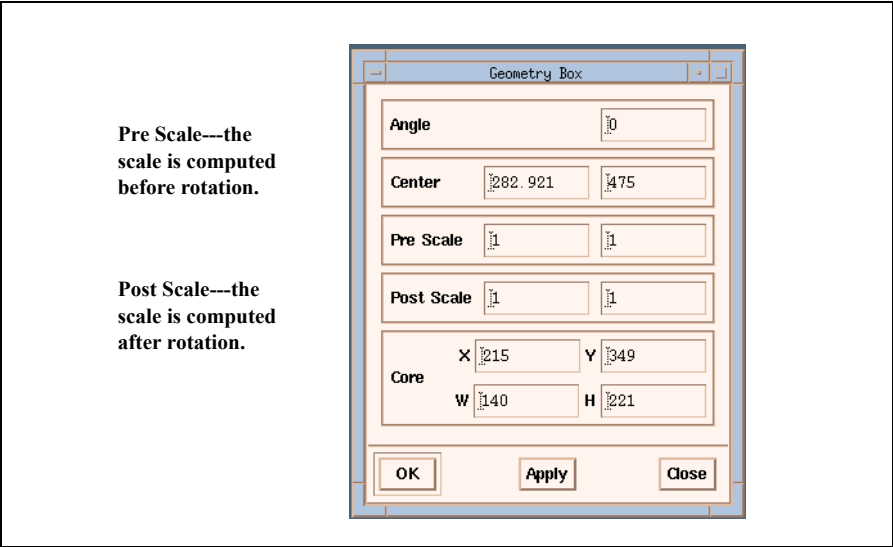


Figure 4.1: The Geometry Box

Pre Scale This defines the scale of the object relative to its original values computed before rotation.

Post Scale This defines the scale of the object relative to its original values computed after rotation.

4.5 Cut, Copy, Paste, Duplicate, Delete

You are probably already familiar with these common editing features. To use them, first select the object you want to edit; then choose the appropriate item from the **Edit** menu, or use the keyboard combinations that are listed in the menu. There is no **Undo** feature in XDrawMaker, so the editing choices you make are definitive.

Cut When you *cut* an object, a copy is put into a temporary file called the “clipboard”. The copy can then be pasted onto the desktop *if no other object has been cut or copied in the meantime*. When you *cut* an object, it disappears from the desktop until you paste it.

Copy When you *copy* an object, it remains on the desktop and a copy of it is available to be pasted. The copy of the object is put into a temporary file called the “clipboard” and it can then be pasted onto the desktop *if no other object has been cut or copied in the meantime*.

Paste The **Paste** command pastes a copy of the *latest* cut or copied object from the clipboard. To paste an object that has been cut or copied, choose **Paste** from the **Edit** menu. The pointer changes to a solid cross. Move it to the position you want to place the cut or copied object, and click the **Select** mouse button. The object will be displayed, with its center located at the position of the pointer.

Duplicate The **Duplicate** command draws a copy of the selected object at a point slightly lower and to the right of the selected object. You can continue duplicating the same object using **Ctrl-d**.

Delete and Delete All A deleted object is *not* placed in the clipboard and cannot, therefore, be retrieved and pasted. You can also delete *everything* from the desktop by selecting **Delete All** from the **Edit** menu. Remember however, that there is no **Undo** function in this version of XDrawMaker, and no alert box to ask you to confirm this potentially regrettable move!

4.6 Grids and View Options

XDrawMaker has different options for viewing your work: They include:

- hiding or showing objects.
- a zoom feature.
- using grids.
- aligning objects.

4.6.1 Viewing Options

The ***Hide/Show (All)*** commands in the View menu are used to hide or show a selected object or group. Select the object, then choose the appropriate command from the **View** menu.

Once an object has been hidden, and then deselected, it will be completely invisible on the screen. In order to *show* it again, you must first select it; but this may be difficult if you don't have a good idea of its exact location. In this case, choose the ***Select All*** command from the Edit menu. The hidden object's selection handles will appear on the screen, enabling you to locate it.

4.6.2 The Zoom Feature

You can enlarge a selected part of a drawing by using the zoom commands in the View menu.

To enlarge a part of a drawing:

1. Select ***View/Zoom In***.
2. Drag a rubber band box around the area you want to enlarge.

The zoom factor, which is displayed at the bottom of the window is relative to the size of the rubber band box you draw. The box is enlarged to fit the size of the XDrawMaker window. You can zoom in on an area of your drawing up to a size of 32,000 pixels.

- Selecting ***View/Undo Zoom*** undoes the last Zoom. The command can be repeated until no more zooms are left to undo.
- Selecting ***Reset Zoom*** resets the desktop to the original state.

Label and image objects can be zoomed faster if you set the fast zoom (-fz) command line option. When this option is set, XDrawMaker will zoom only the perimeter of the objects, not their contents.

4.6.3 Grids

It is often useful to display a grid in the background of your desktop to help you draw and align objects. Of course, the grid is not a part of your Draw file and will not appear when you print your work.

To hide or show the grid, click on **View/Grid**. This is a toggling item. When the **Grid** feature is “on”, an indicator is set next to the item. Turn it “off” to hide the grid, by clicking again on **Grid**.

XDrawMaker gives you several options for changing the grid, and for determining the way objects align themselves on it. To do this, open the Grid Configuration box by choosing **Options/Grid settings...**

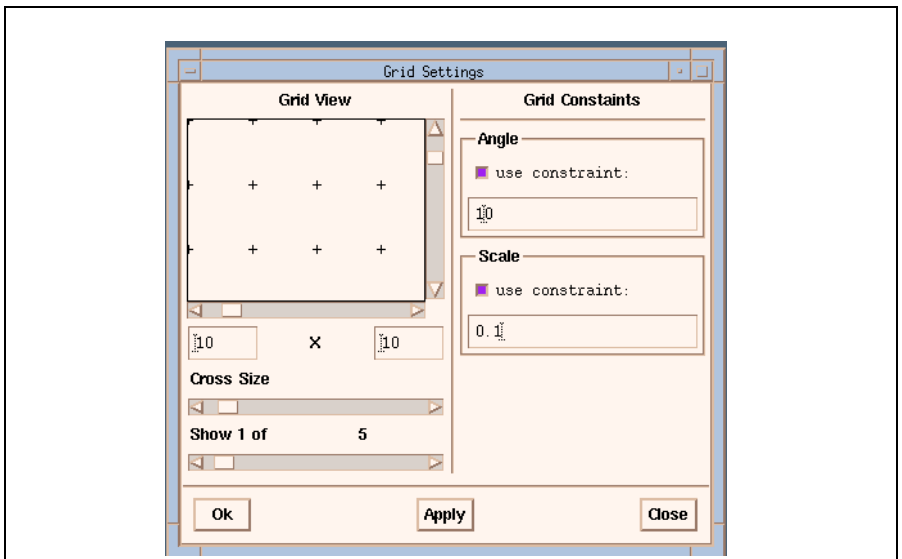


Figure 4.2: The Grid Configuration Box

Specify your choices, then click on **Apply** to apply them, or click on **OK** to apply the choices and close the box. The viewing box at the upper left side of the Grid Configuration box displays the current grid.

The *slider bars* allow you to change the number of units—maximum value 91—separating the points of the grid vertically and horizontally. Use the slider bars to choose the spacing of the grid points, then click **Apply** or **OK**. The values are displayed in boxes just below the grid viewing box. You can also enter the values directly into the boxes and click **Apply** or **OK**.

There are two additional slider bars below the grid viewing box.

- The *Cross size* slider bar allows you to change the size of the grid points.
- The *Show 1 of* slider bar lets you choose the *percentage* of crosses that are displayed. Choosing “Show 1 of 1” means that *all* the points of your grid will be displayed on the screen. Remember that if all the grid points are not displayed, it will be difficult to view the alignment of objects or if they snap to a grid point.

When the ***Snap to Grid*** item in the Options menu is set, the selected object “snaps” or aligns itself to the nearest grid point.

You can set constraints that define an object’s behavior when you rotate or scale an object. In the Grid Constraints field:

- In the Angle field, set the *use constraint* checkbox and enter a value in the input field. This value determines the rotation angle increment for an object. The unit of measurement is set in the **Angle unit** pulldown menu at the bottom of the main window.
- In the Scale field, set the *use constraint* checkbox and enter a value in the input field. This value determines the scale factor increment for an object. This applies both to pre-scale and post-scale coefficients.

4.6.4 Aligning Objects on a Grid

You can align objects on a grid by using the Alignment box. When you align objects using the grid, set the *Options/Snap to Grid* item. To open the Alignment box, select *Arrange/Align*.

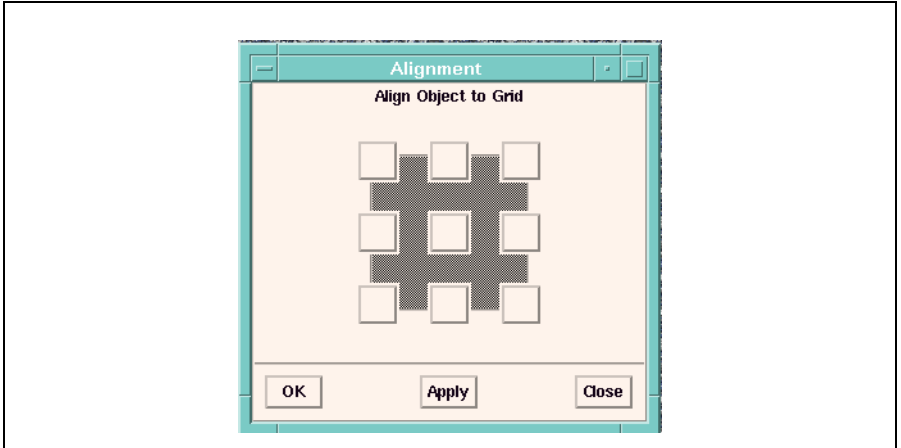


Figure 4.3: The Alignment Box

The nine squares in the **Alignment** box align the corresponding points of a selected object to the grid. For example, when you select the upper left-hand corner square in the box, then click *Apply*, the upper left-hand corner of the selected object will be aligned to the *nearest* point on the grid. Remember that the *Snap* option must be set in the **Options** menu.

There are two points to note concerning the alignment box.

- This option can be useful when moving objects, because once an alignment has been selected, the object will be automatically aligned to the grid no matter where you move it—provided the **Snap** option is set.
- Individual objects keep the specific alignment chosen for them. Thus, object #1 can be aligned to the nearest grid point at its upper left-hand corner, while object #2 is aligned to the nearest grid point at its lower right-hand corner.

4.7 Display options

A number of display options are available in the **Options** menu.

- The Double Buffering feature eliminates the “flashing” that can occur when the screen changes; e.g. when you activate drawings. When the ***Double Buffering*** option is selected, XDrawMaker copies the current drawing into a buffer, and records all modifications in the buffer as well as in the original drawing. When your screen changes, the *complete* drawing is displayed on your screen, rather than an object-by-object display. The Double Buffering feature makes the display of your drawings more attractive, but uses more memory.
- When the ***Resize Line Width*** item is set, the width of a scaled line is resized when a drawing is scaled. Setting this feature may cause the scale action to take a slightly longer time to complete.
- When the ***Icon Bar*** item is set, the Edit box icons on the right side of the main window are displayed.

CHAPTER 5: Animating a Drawing

XDrawMaker's Activate menu has options that enable you to animate your drawings. You do this by writing scripts in the FACE language to specify object behavior. You can then test the animation using one of the commands listed below.

The Activate menu has the following three items:

- m The ***Script*** item pops up the Script Editing box for writing scripts.



- m The ***Run in DrawMaker Window*** command runs the script in the XDrawMaker window. However, an error in the script may cause XDrawMaker to crash.

- m ***Launch a Separate Process*** spawns the **XDrawMaker Exec** process for running the script. In this case, an error encountered in the script will not affect the XDrawMaker process.

5.1 Writing scripts

As we explained in Section 1.1, the XDrawMaker program is based on the XDraw widget. In XDrawMaker, each object in a drawing is able to respond to events, such as pressing a key or mouse button. When the program is in “run” mode (i.e. when you select **Run In DrawMaker Window** or **Launch a Separate Process** from the Activate menu) and an event occurs, the Draw widget launches the action for the object associated to that event.

To specify an event and the associated object’s subsequent action, you must write a script— which is a program written in the FACE language. You can use the Script Editing box which is displayed by choosing *Script* from the Activate menu. Or use the editor of your choice by selecting the Custom Editor button. Use the **FMEDITOR** environment variable to specify the editor; for example, use **wx242** to specify NSL’s Wx editor. See Chapter 6 for information on the FACE language.

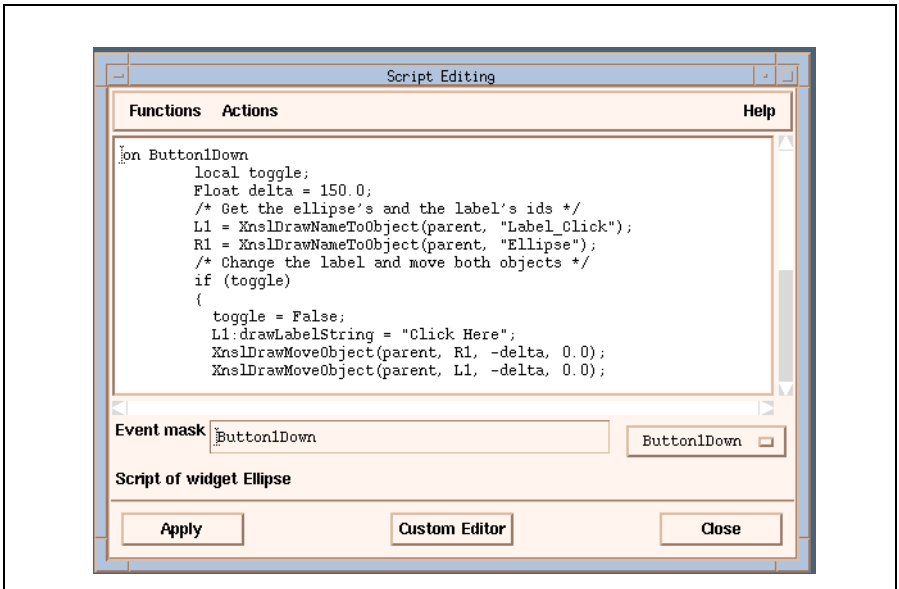


Figure 5.1: The Script Editor

The Event mask is entered automatically.

A script has the following format:

```
on action and location  
  Script Body  
end
```

A script is launched if the conditions given in the first line of the script occur *and* the XDraw widget is in **Run** mode.

The *action* refers to one of the events listed in the *Actions* menu. These are:

- m **Button1Down**: Script is triggered when button 1 is pressed
- m **Button2Down**: Script is triggered when button 2 is pressed
- m **Button3Down**: Script is triggered when button 3 is pressed
- m **Button1Down2**: Script is triggered when button 1 is double clicked
- m **Button2Down2**: Script is triggered when button 2 is double clicked
- m **Button3Down2**: Script is triggered when button 3 is double clicked
- m **Button1Motion**: Script is triggered when button 1 is pressed and mouse is moved.
- m **Button2Motion**: Script is triggered when button 2 is pressed and mouse is moved moved.
- m **Button3Motion**: Script is triggered when button 3 is pressed and mouse is moved.
- m **Button1Up**: Script is triggered when button 1 is released
- m **Button2Up**: Script is triggered when button 2 is released
- m **Button3Up**: Script is triggered when button 3 is released
- m **Destroy**: Script is triggered when the widget (or gadget) to which it is attached is destroyed.
- m **KeyPress**: Script is triggered when any key is pressed.
- m **KeyRelease**: Script is triggered when any key is released.
- m **Load**: Script is triggered when a file is loaded (and the widget is in Run mode) and/or when the widget is toggled from Edit to Run mode.
- m **MouseMoved**: Script is triggered when the mouse is moved with no buttons pressed.
- m **Resize**: Script is triggered when the XDraw widget is resized.

The *action* can also be a *translation* in Xt form; for example:

```
"<KeyPress>a"
```

Note that the quotation marks must be included. Any sequence of translations can thus be specified. The event mask in the Event Mask field of the Script Editing box must be changed accordingly. For example, for a translation such as `<KeyPress>a`, **KeyPress** has to be added to the event mask.

The *location* is used to locate where the event occurs. If no location is specified in a script, it is activated if the event occurs inside or within the border of the object. If the event has no coordinate information, the XDraw widget performs an XQuery-Pointer¹ to locate the pointer.

Scripts are written in the Script Editing box, shown in Figure 5.1. It contains two pull-down menus:

- **Functions** lists the XDrawMaker functions by category.
- **Actions** lists the events which can launch a script.

Writing a script for an object follows the sequence shown below:

1. Select the object to be activated in your drawing.
2. Open the Script Editing box by clicking on *Activate/Script*.
3. Open the Actions menu by clicking on that item in the box.
4. Click on the name of the event you want to launch the script for the selected object. The script format will be displayed.
5. Enter the location of the event. Possible locations are:
 - m **draw_nowhere** The script is not launched.
 - m **draw_inside** The script is launched if the event occurs inside the object.
 - m **draw_on_border** The script is launched if the event occurs within the border of the object.
 - m **draw_outside** The script is launched if the event occurs outside the object.
 - m **draw_everywhere** The script is launched regardless of the location of the event.
6. Open the *Functions* pull-down menu to insert XDrawMaker functions into your script. When you click on a function name, it will be inserted into the script at the insertion point, with dummy arguments which you need to replace with the appropriate values. In general, the dummy argument **w** refers to the XDraw widget

1. See Xlib Reference manual.

itself, i.e. the parent of the object you are working with if you have selected an object: you can replace **w** with **parent**. See below, if no object is selected.

The dummy argument **o** refers to an object; if you want to pass the ID of the object whose script you are writing, pass **self**. If you want to pass the ID of another object, you can get the object ID with functions such as `XnslDrawNameToObject` or `XnslDrawRefNameToObject`, or `XnslDrawGetNthChild`. The other arguments are self explanatory.

7. Click on the **Apply** button. This associates the script with the selected object. If there is an error in your script, a Messages box will appear describing the problem.
8. Deselect the object if any, save the Draw file as usual. The script you wrote is now associated with the selected object. Whenever that object is selected, and the Script Editing box is popped up, the associated script will be loaded into the box.

Here is an example of a simple script.

```
on Button1Down and draw_on_border
    printf("hello\n");
end
```

This script will be launched when the user clicks the *Select* mouse button on the border of the object. This event calls the pre-defined function `printf`.

You can also write scripts for the widget itself. This is done by opening the script window with no objects selected. In this case, the *location* is ignored, you should replace **w** with **self** in the functions.

5.2 Running a script

The object behavior you defined in the script will be applied to the object when the event you entered in the script occurs. To see this happen, select either the ***Run In DrawMaker Window*** or the ***Launch a Separate Process*** command in the Activate menu.

- If you select ***Run In DrawMaker Window***, the script will be run in the XDrawMaker window. However, if an error occurs in the script it may cause XDrawMaker to crash. The editor is inactive while the ***Run In DrawMaker Window*** command is active. To get back to the editor, click ***Edit*** in the Activate menu.
- If you select ***Launch a Separate Process***, the XDrawMaker Exec window, shown below, will be displayed.

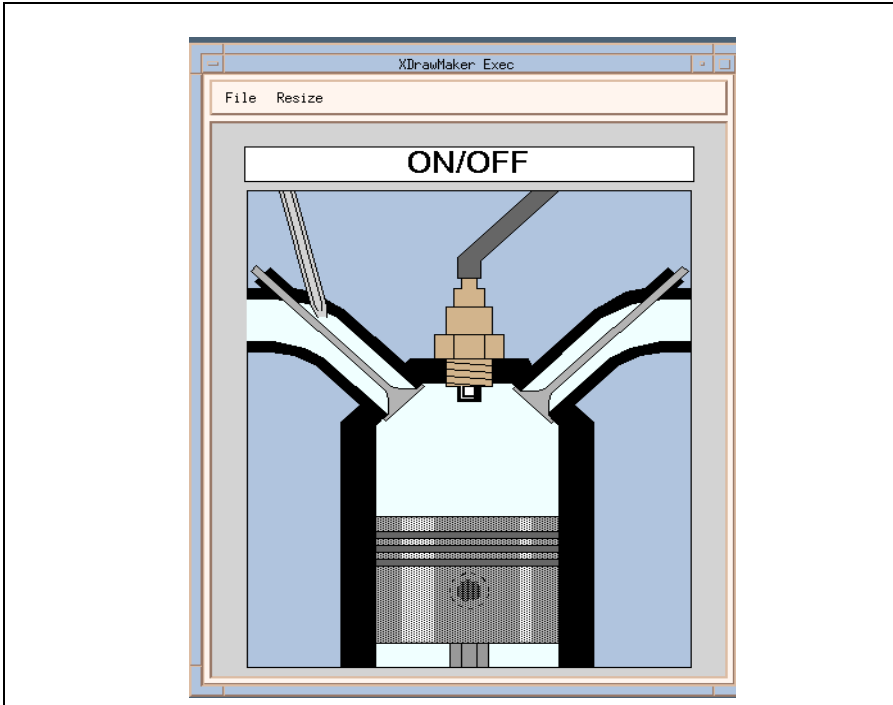


Figure 5.2: The Exec Window

Trigger the event which you entered in your script. In the Exec or the XDrawMaker window, you can observe the objects in your drawing responding to the event that you specified in your script. If appropriate, you can re-trigger the event.

The Resize Policy pulldown menu lets you choose how the drawing will behave if the window is resized. You have the following options:

- **None** specifies that the drawing will not change size.
- **Full** specifies that the drawing will be resized with the window. This may distort the drawing.
- **Uniform** specifies that the drawing will be resized, but will keep its aspect ratio.

CHAPTER 6: The FACE Language

FACE is a language whose syntax is very close to that of C. It was designed specifically to program the dynamic part of a user interface, and not as a general-purpose language. In contrast to ordinary C, FACE scripts are not compiled while in the development phase of interface construction but are *interpreted*, so that the typical compile-execute cycle is not required. The FACE interpreter reads and executes one statement at a time until the end of the script is reached.

The FACE interpreter is used in several NSL products. This chapter describes only the general features of the FACE language.

The examples used in this chapter have been taken from the XFaceMaker documentation. For other NSL products, do not use the keywords **root** or **toplevel**.

6.1 Syntax in FACE

6.1.1 Keywords in FACE

The following identifiers are reserved for use as keywords, and may not be used otherwise:

```
if else while for
script
self parent toplevel root
global local auto extern
type cprototype
function return
define include
resource
```

`resource` is reserved for future use. `auto` and `extern` are synonymous with `local` and `global`. Avoid using `auto` and `extern` in any new scripts that you write. They have been kept for backward compatibility only. The other keywords are described in this chapter.

6.1.2 Special Characters

The following characters have special meanings in FACE:

`; . , : $ @ ' ' " { } ( ) < > = ! & | + - * /`

6.1.3 Comments

The characters `/*` introduce a comment, which terminates with the characters `*/`. Comments do not nest.

6.1.4 Arithmetic Expressions

Arithmetic expressions have the same syntax as in C. The following operators are supported:

Assignment operator `=`

Unary operator `-`

Arithmetic operators	+	-	*	/		
Logical operators	&&		!			
Relational operators	==	!=	<=	>=	<	>

All arithmetic operations work on integer or floating-point operands. If any of the operands is a floating-point value, then the operator performs a floating-point operation, but the other operands are *not* converted automatically to a floating-point type (you must use the function `citof`). Expressions can be parenthesized and operator precedence is as in C.

6.1.5 Include Files

A file may be included in a FACE script using the `include` keyword, followed by a file name. i.e.,

```
include "project.face";
```

The included file may contain any FACE statements. The including script is parsed as if the contents of the file had been included in the script.

6.2 Basic Objects

FACE has the following kinds of objects:

- constants
- variables
- special variables
- widgets
- widget resources
- active value variables (when used with XFaceMaker)
- X properties

6.2.1 Object Types

The type of objects in FACE is specified by a string whose content is arbitrary; i.e., an application may define new variable types if it so desires, as long as they satisfy the type checking and/or type conversion done by FACE when a variable is used. The usual type strings are defined in `<Xm/Xm.h>`. They correspond to frequently used widget resource types, such as “Int”, “String”, “Pointer” etc. In addition FACE has some special types, described in the next section.

All FACE objects are stored by the FACE interpreter in equally sized memory units which have the size of the X Toolkit-defined `XtArgVal` typedef, which is usually defined as a `C long`. When the actual size of the memory needed to represent an object is greater than the size of an `XtArgVal` (e.g. character strings), a pointer to the object is stored instead.

Most common FACE objects have the same representation as in C: integers are represented as `C ints`, floating-point values are represented as `floats`, character strings are represented as pointers to null-terminated character arrays. Only FACE arrays have a special representation.

FACE checks the types of objects as follows:

- when a function is called, the types of the actual parameters are checked against the expected types;
- when a widget's resource is set or retrieved, the type of the value is checked against the type of the resource (except for String values which are automatically converted: see Section 6.2.8);
- when a value is assigned to a statically typed variable, the type of the value is checked against the type of the variable.

FACE accepts a number of resource types as equivalent to the type `Int`: they are `Short`, `Boolean`, `Position` and `Dimension`. In addition, when used with `XFaceMaker` and `Motif`, all Motif types ending with `Position` or `Dimension` (e.g. `VerticalDimension`) are also equivalent to type `Int`. Similarly, the type `Pixmap` and all its Motif variants like `XmBackgroundPixmap` are equivalent. Finally, FACE accepts the types `Window` and `MenuWidget` as equivalent to `Widget`.

Unlike C, when variables are used or declared, their type need not be specified. If the type of a variable is not specified when the variable is declared, the type is determined each time the variable is assigned and it takes the type of the function or widget resource to which it is assigned. For example:

```
x1 = self:x;
```

`self:x` is a widget resource. Its type is `Dimension`. With this assignment, `x1` is defined as being of type `Dimension`.

```
a = f(0);
```

The type of the variable `a` is the type of the function's return value. FACE checks the type of variables in two instances; when assigning a variable, and when passing it as a function parameter.

6.2.2 Special FACE Types

You can prevent argument type checking by using the special types “None”, and “Any”:

None If, when *attaching* a function, i.e., in `FmAttachFunction`, you declare the type of an argument as “None” you are in effect telling FACE that the argument type for this argument is not to be checked whenever the function is called.

Any If, when *calling* a function whose arguments have all been specified, you use a type “Any” for one of the arguments, the type of that argument is not checked for that call.

Obviously, these two types should be used only rarely and with care, since circumventing type checking prevents the detection of possible errors.

6.2.3 Constants

Integer constants begin with an optional - (minus) sign followed by a sequence of decimal digits 0-9. FACE integer constants can only be input in decimal form. They have the FACE type `Int`.

Floating-point constants begin with an optional - (minus) sign followed by a sequence of decimal digits 0-9, a . (dot) character and a second sequence of decimal digits. FACE floating-point constants do not have an exponent part. They have the FACE type `Float`.

FACE character string constants are placed within optional single or double quotes. A string which does not include blanks does not *have* to be placed between quotes but it is good practice to always quote strings: if a variable with the same name exists, an unquoted string is assumed to refer to the variable though you may have intended it to be a string. Strings which contain blank characters *must* be quoted. The single and double quote characters may also be escaped within a string with \ (backslash). A quoted string may contain the “other” quote character unescaped.

The memory holding a string constant is allocated by the FACE interpreter the first time the script in which the string appears is parsed, and the same memory is re-used each time the script is interpreted. Thus, modifications made to FACE string constants remain in place. FACE character string constants have the FACE type `String`.

These are examples of FACE integer, floating-point and character string constants:

```
1234
-5678
12.34
-56.78
Hello
"Hello world"
'Hello world'
"Awakening he said: \"Hello world\"."
'Awakening he said: \'Hello world\'.'
"Awakening he said: 'Hello world'."
'Awakening he said: "Hello world".'
```

6.2.4 Variables

Variable names are composed of letters and digits. They must start with a letter and are case sensitive. Thus `X` and `x` do not refer to the same variable. All FACE variables have the same size, that of a word or pointer (i.e. the size of an Xt Toolkit `XtArgVal`), whatever their type. For example, a `char` variable, which would normally occupy a byte, is converted if necessary to an integer the size of a word.

There are local and global variables. Local variables are known only in the script in which they appear. Thus a local variable may be known in the script of one callback resource but not in another, even of the same widget. You need not declare local variables. They are automatically declared the first time they are used.

```
Message = "Warning: Not found!";
Size = 10;
```

Here, the variable `Message` has been declared implicitly the first time it has been set. It is of type `String`. The variable `Size` is also implicitly declared when first used and is of type `Int`.

Variables may be declared anywhere in the script, as long as they are not used before they are declared. It is possible to declare a local variable explicitly if it must be used before it is first set.

Local variables are declared using the `local` keyword:

```
local i;
```

This declares a local variable named `i`, initializes its value to zero and sets its type to “None”. Local variables declared outside FACE functions are *not* automatic variables, i.e., they are not allocated dynamically each time the callback is invoked. They are like static variables in a C function: they are initialized to 0 when the script is parsed and retain their value from one call to another. Local variables declared inside FACE functions *are* automatic, and are allocated and set to 0 each time the function is called.

Global variables can be referenced by all scripts of the same interface. They are declared using the `global` keyword:

```
global button_pressed;
```

Global variables can also be declared extern as in:

```
extern global button_pressed;
```

Extern global variables differ from global variables only in the way they are declared in the C code generated. The ‘extern’ modifier should be used in cases where a global variable is used in several .fm files in order to avoid multiple declarations of the variable in the C code generated: declare the variable as ‘extern’ in all but one script.

Global variables must be declared in all the scripts in which they are used. The parser creates the variable and associated storage the first time it encounters the declaration.

It is possible to declare the type of a FACE `local` or `global` variable, either when the variable is declared, with the following syntax:

```
[local|global] <type> <name>;
```

or when a local variable is first set, using:

```
<type> <name> = <value>;
```

When a variable is declared with a type, then the variable is *statically typed*, which means that you cannot assign an expression of a different type to it. If you do so, an error message is issued. Static typing is useful for error checking, and also to help the FACE compiler find the type of a variable; e.g., in floating-point expressions.

Note: the keyword `auto` is equivalent to `local`, `extern` to `global`. Use `local` and `global` rather than `auto` and `extern` which are kept for backward compatibility only.

6.2.5 Special Variables

Whenever a FACE script is parsed, the two special variables `$` and `@` are passed to it. `@` is the contents of the variable pointed to by `$`. This is analogous to `&x` and `x` (if `x` is an int), or `p` and `*p` (if `p` is a pointer to an int). The values of `$` and `@` depend on the context in which the script is being evaluated.

If `$` is the address of a structure, then the elements of the structure can be accessed using either regular structure addressing constructs, (Section 6.8), or through the `@0` to `@9` variables:

```
@ or @0 is the first element
@1 is the second element
.
.
.
@9 is the 10th element.
```

This assumes that the structure pointed to by `$` contains only elements whose size are the same as an `XtArgVal` in the Xt Toolkit. For this reason, this usage is discouraged, and FACE scripts should use the normal structure mechanism (see Section 6.8) to refer to elements in a structure.

The variables `$` and `@` have type “Any” which allows them to match any other type wherever they are used. This means, however, that argument and return type checking is not done for `$` and `@` used in function calls unless they are cast to the proper type; e.g.:

```
i = atoi(String($));
```

6.2.6 Defined Constants

A constant may be defined with the `define` keyword. Defined constants are like variables, but their value cannot be changed. Their value is either an integer, a string or a floating-point constant, and is specified after the constant name:

```
define <name> <value>;
```

Examples:

```
define MAX_WIDGETS 10;
define PI 3.142;
define TITLE "Project X";
self:labelString = TITLE;
```

A defined constant may be defined in several places, but it may not be defined with different values. A defined constant can be used in a script even if it has been declared in another script, provided that the script where the constant has been defined is parsed first. Using the `define` keyword in the Creation script of the root widget of a hierarchy guarantees that it will be known in all the scripts of the hierarchy.

The FACE interpreter has some predefined constants:

- The constants `true`, `True`, `false`, and `False` are defined globally. Their type is “Boolean”.

```
true  = True  = 1
false = False = 0
```

- When using FACE in XFaceMaker, XFaceMaker also adds most constants defined by the OSF/Motif toolkit.

6.2.7 Widgets

The FACE naming scheme uses the `.` (dot) operator, which lets you travel from a widget to one of its children. Widget names have the following form:

```
[<reference>.[<name1>[.<name2>]...]]
```

where `reference` is either one of the keywords below, or a variable name, and `name1`, `name2`, ... are widget names that form the path of the designated widget in the widget hierarchy.

FACE recognizes the following keywords to designate the reference of a widget name:

`self` refers to the widget from which the script is called. In the case of `FaceEvalString`, it refers to the widget that was passed as an argument.

`parent` refers to the parent of `self`. In the case of `FaceEvalString`, it refers to the parent of the widget that was passed as argument to `FaceEvalString`. Multiple `parent` keywords can be concatenated to designate the parent's parent, etc.

`toplevel` refers to the child shell of the `ApplicationShell` (root) which is an ancestor of `self`, or `root` itself if `root` has no shell child that is an ancestor of `self`.

`root` refers to the `ApplicationShell` which is an ancestor of `self`. This is necessary, for instance, to travel from a `TopLevel` shell to another `TransientShell` at the same level, e.g. `Show(root.TransientShell)`.

If a variable name is used as the first component of a widget name instead of one of the keywords below, then the variable must contain a valid widget pointer which will be used as the reference widget. The remainder of the widget name, i.e. the path of sub-widget names, is used to find the sub-widget by calling an equivalent of the X Toolkit function `XtNameToWidget` with the reference widget and the sub-widget path. If there is no sub-widget path, the reference widget is returned.

When naming a widget, the name must be made explicit by starting it with one of the above reserved names, unless the widget name contains a `.` (dot) character, or the widget name is followed by a widget reference with a `:` (colon) character (see Section 6.2.8). The following are legal widget names in FACE:

```
self
parent
```

```
root
  toplevel
    self.Wix2
      parent.ScrollBar
        parent.parent.WarningBox
          root.GrabShell.GrabBoard.Input
            toplevel.GrabBoard.Input
              GrabBoard.Input
                Input:value
                  wid.Button /* wid is a variable */
```

You can refer to a widget in a FACE script using the wildcard character `*` in a manner similar to the X Toolkit function `XtNameToWidget`. The following limitations apply:

- The `*` can only be used in the sub-widget path part of a widget name, and it cannot be used at the beginning or at the end of a widget name.
- If used in association with a variable name, the variable must be explicitly declared of type `widget`. Otherwise, the `*` will be interpreted as *multiply*.

These are legal examples of the use of the wildcard character in widget names:

```
self*Button
root.Shell*ScrollBar
wid*Text /* wid is a statically typed Widget
           variable */
```

6.2.8 Widget Resources

Widget resources can be accessed by a widget name, followed by the `:` (colon) operator, followed by the widget-defined resource name as specified in the widget reference manual. You must use the resource name without its prefix, as in a `.Xdefaults` resource file; e.g.

`"self : x", not "self : XmNx"`.

The following are examples of legal resource names:

```
self:x
self:arrowDirection
parent.parent.button:borderWidth
```

A variable to widget resource assignment is of the form:

```
widget:resource = abc;
```

Each time an assignment statement such as

```
i = self:x;
```

is encountered, the function `XtGetValues` is called to retrieve the current value of the resource referred to.

Similarly, each time an assignment statement such as

```
self:x = i;
```

is encountered, the function `XtSetValues` is called to update the current value of the resource referred to.

Note: If you refer to the documentation on widget resources, you will see that the size of variables representing these resources varies according to the resource type. This is not so in FACE where all variables are the same size (that of `XtArgVal`). This means that resources whose size are smaller than a word are converted to a word-sized integer, and resources of type `String` are replaced by a pointer to the beginning of the string.

If the right-hand side of a resource assignment is of type “String” and the resource is of another type, an implicit type conversion is done: the FACE interpreter calls the converter registered with the Xt Toolkit to convert the string to the appropriate resource type.

If the right-hand side of a resource assignment is of another type, then this type must be compatible with the type of the resource, or an error will occur. For example:

```
self:width = "1";
```

converts the string "1" to the value 1 before doing the assignment.

```
self:width = "hello";
```

produces an error because the **String** to **Int** conversion function is unable to convert **hello** to a type **Int**.

```
self:width = 2;
```

does not produce an error and does no type conversion: the type of the variable and the type of the resource are both **Int**.

```
self:labelString = 1;
```

produces an error because the resource type for **labelString** is **XmString**, not **Int**. In such a case, no implicit conversion function would be called since 1 is not of type **String**.

```
arrowButton1:arrowDirection = "arrow_up";
```

converts the string to type **Arrow Direction**.

Note: If you use the predefined OSF/Motif constants as right-hand side values in resource assignments, you might have to cast them to the appropriate type because they are all declared with the type "Int".

```
type ArrowDirection;
```

```
self:arrowDirection=ArrowDirection(XmARROW_RIGHT);
```

FACE accepts a number of resource types as equivalent to an **Int**: they are "Short", "Boolean", and all Motif types ending with "Position" or "Dimension", such as "VerticalPosition". Similarly, the types "Pixmap" and all its Motif variants such as "BackgroundPixmap", are equivalent.

In a few cases, you may have to specify the type of a resource explicitly using the following syntax:

```
<widget-reference>:<resource-name>:<resource-type>
```

For example, **self:value:Int** means that the resource **value** is of type **Int**. This is useful only to inform the FACE compiler of the type of the resource when generating stand-alone C code if the compiler cannot determine it by itself.

6.2.9 X Properties

FACE lets you refer to X properties to store data. X properties are variables associated to an X window. For that reason, they can be used only for widgets that have already been realized. Since such a variable is associated to the particular instance of

the widget for which you define it, its value is independent of any other. Thus, the value of a property with the same name defined for a different instance may be different.

It is legal in FACE to use properties for gadgets but, since gadgets do not have an X window, the property of a gadget is associated to the gadget's parent. Thus, for gadgets, you must be careful not to use the same property for two gadgets with the same parent, or for a gadget and its parent, if you want the variables to be independent.

To define a property, an assignment statement is required, such as:

```
widget:<property_name> = self:x;
```

where:

widget is the name of the *realized* widget for which you are defining a property.

property_name is the name of the property, i.e., the name you will use to refer to that variable. Note that the name is placed in <angle brackets>.

With such a statement, you have defined a new variable name, and its type: the type of the property is the type of the resource or of the variable which is on the other side of the assignment statement.

For example, you can write:

```
self:<area> = self:height * self:width;  
parent:<area> = parent:height * parent:width ;
```

to define two variables (properties) called area, both of type **Int**, both being the area of the associated widget. If self is a gadget and not a widget, however, area is a single variable, whose value is the area of parent after the second statement is executed.

6.3 Type Casters

Type casting functions are available in FACE. They do not *convert* an object, i.e., they do not change its value but they *cast* the object type to another type to satisfy FACE type checking requirements. These functions return their argument, which may be of any type, in the appropriate type. FACE type checking does not consider all resource types as distinct. For example, types “Position”, “Dimension”, “Boolean” are all equivalent in FACE.

6.3.1 Declaring New Type Casters

New type casting functions can be declared using the keyword `type` followed by the variable type to which you want to be able to cast. After such a declaration, the new type casting function exists. For example:

```
type NewType;  
n = NewType(x);
```

6.3.2 Declaring Enumerated Resource Types

The `type` keyword is also used to declare a new enumerated type along with its values with the following syntax:

```
type <type-name> = <value0>, <value1>, ... ;
```

This declares a new type caster, and defines constants for every value.

For example:

```
type Fruit = apple, orange, banana;
```

declares the new type caster `Fruit`, and defines 3 constants:

```
define XfmAPPLE 0  
define XfmORANGE 1  
define XfmBANANA 2
```


In addition, a new type converter is registered in the X Toolkit so that the strings "apple", "orange" and "banana" can be converted to the corresponding constant.

The prefix of the new array can be specified like this:

```
type Fruit = [Xyz] apple, orange, banana;
```

This would define the contents:

```
define XyzAPPLE 0;
```

etc.

6.3.3 Explicit Type Conversion

An explicit type conversion function is available to convert a string to the type specified as one of the arguments:

```
ConvertString(widget w, String SI, String target_type);
```

where:

w is the widget used for additional arguments, if required, as in the XtConvert function.

SI is the string you want to convert.

target_type is the string specifying the type you want to convert to.

The return type is type "Any". For example:

```
ConvertString(w, "10", "Int");
```

returns the value 10 of type "Any".

6.4 Statements

A FACE script is a sequence of one or more statements. Each statement is either:

- an assignment
- a control structure
- a function call
- a call to a named script.

6.4.1 Assignments

Assignments are, as in C, of the form:

`<left-hand-side> = <right-hand-side> ;`

The left-hand side of an assignment can be:

- a variable (local or global),
- a widget resource,
- a widget property,
- the special variable @,
- an active value of the form @name, widget:@name, or widget:\$name (in XFaceMaker).

The right-hand side of an assignment can be any expression, that is:

- a constant (integer, character string or floating-point),
- a variable,
- a widget reference,
- a widget's resource,
- a widget's property,
- a function call,
- a special variable (\$ or @),
- an active value's address \$name (or widget:\$name), or contents @name (or widget:@name) (in XFaceMaker),
- an arithmetic expression, which itself can be composed of arbitrary expression operands.

6.4.2 Control Structures

FACE provides the `if`–`else`, `while`, and `for` statements. These have the same syntax and semantics as in C. The following example illustrates the use of `if`–`else`:

```
valueChangedCallback =
    Loudness = self:value;
    if (Loudness == 0)
        ResetVolume();
    else if (Loudness < 10) {
        Slide = Slide + Slide/10;
        SetVolume(Slide);
    } else
        SetVolume(Slide);
```

FACE provides a `while` loop equivalent to that used in C. For example, the following code performs a linear search in an application database for information keyed on a string selected from a `TextList`:

```
Running = 1;
ResetTable();
while (Running) {
    Key = GetTable();
    Running = equal(Key, $);
}
DisplayCurrentTable();
```

For example:

```
Int a[String];
...
for(String s = first(a, 0); s != 0; s = next(a, s, 0)){
    printf("element %s = %d\n", s, a[s]);
}
```

6.4.3 Function calls

Function calls have the same syntax as in C:

```
<function_name> ( <arg1> , <arg2> ... ) ;
```

A function call can be used as a statement by itself, in an expression, or as the right-hand side of an assignment. When a function call is used by itself, its return value is lost.

FACE checks the types of the arguments when calling a function, as explained in Section 6.2.1. An argument type mismatch causes an error, and execution aborts.

The current implementation of the FACE interpreter uses the “traditional” function calling conventions. A function call is performed by the FACE interpreter through a function pointer declared in C as follows:

```
typedef int(*FaceFunctionPtr)();
```

This means that the function pointer is *not* prototyped as it could be with an ANSI C compiler, so all arguments are passed to the function using the default argument promotion rules used by non-ANSI C compiler:

- `chars`, `shorts`, and `ints` are passed as `longs`,
- `floats` are passed as `doubles`.

For this reason, the FACE interpreter only distinguishes between float and integer (or pointer) arguments when calling functions (see Section 6.6). In addition, pointers are passed as `longs`.

6.4.4 Named Scripts

The keyword `script` must always be followed by an identifying *name*, and has two uses:

- to name scripts,
- to share scripts.

The *name* must begin with an alphabetic character.

The keyword `script` labels and executes a named script if one is defined immediately after a `:` (colon), or executes the named script if no colon announcing a script definition follows (i.e., it is followed by an end of statement `;` (semicolon)). The first time FACE encounters a script, it caches the script, using its name as an index. Therefore two different scripts should never have the same name or the second one will not be executed.

`Eval` scripts, should be named systematically even if they are only used in one place, to make use of this caching mechanism. A *named Eval* script is parsed only the first time it is encountered whereas *un-named Eval* scripts are parsed each time they are executed, thus increasing memory allocation.

The second usage is to share a script. For example, if two buttons in an interface are to have the same help facility, the first defines the script named `help`:

```
helpCallback =  
    script help:  
        root.HelpBox.dialogMessage = "help!";  
        ret = PopupAndWait(root.HelpBox);
```

while the second simply calls it causing it to be retrieved from the cache:

```
helpCallback =  
    script help;
```

The effect will be the same when either callback is called; the values of `self` and `parent` are those of the widget which called the callback.

Note however, that a script definition or call can only appear on its own within a FACE script. The following is *not* legal:

```
helpCallback =  
    self:insensitive = true;           /* wrong */  
    script help;
```

6.5 FACE Functions

It is possible to define C-like functions in FACE, with the following syntax:

```
function Int MyFunction(String s, Boolean b)
{
    if ( b == True )
        return(strlen(s));
    else
        return(0);
}
```

FACE function definitions can be nested, that is, a function can be defined inside another function, but the scope of functions is always the whole interface (functions are declared globally). Recursive calls are also allowed.

FACE functions can be defined anywhere in a FACE script. They can be declared (to be defined later) using the same syntax to declare application functions, as described in Section 6.5.2. When a function is defined or declared several times in FACE scripts, the following rules are applied:

1. An application function (i.e. a function name attached to a real function by the application, or a predefined function) is never overridden;
2. If a function is declared several times with different prototypes, a warning is printed, but the function declaration will be changed, unless it is an application function (first rule).

The `return` keyword has the same semantics as in C. It must be called with parentheses, and it may be called with zero or one argument (of any type). It can be used in FACE functions, but also anywhere else in a FACE script, in which case it will immediately leave the FACE script (the return value is then ignored). No type checking is done between the type of a FACE function and the argument of `return`.

6.5.1 Scope of Variables in FACE Functions

In FACE functions, local variables are automatic: a new instance of the variable is created every time the function is called. The scope of variables is defined as follows:

- the scope of a local variable declared outside a function is the whole script, *not including* functions defined in the script;
- the scope of a global variable declared outside a function is the whole script, *including* functions defined in the script (i.e., it is not necessary to declare the global variable again inside the function);
- the scope of a local variable declared inside a function is the function itself, not including other functions defined inside the function;
- the scope of a global variable declared inside a function is the function itself, and the rest of its surrounding script.

When a FACE function is called, the reference widget and `call_data` (accessed with the special variable `$`), are those of the script from which the function was called. For example, in:

```
widget XmPushButton {
    activateCallback = function Widget MySelf()
    {
        return(self);
    }
} PB1
widget XmPushButton {
    activateCallback = function Widget MySelf();
    w = MySelf();
} PB2
```

the call to `MySelf` in widget PB2 returns PB2, not PB1.

6.5.2 Declaration of Application Functions

Application functions are not known to FACE until the application is linked with the interface; as a consequence, any reference to an application function in a script generates an error message. To avoid this, you can declare your application functions, using the keyword `function`.

The syntax is:

```
function ret_typ fun_name(typ_arg1, typ_arg2, ...) [= val];
```

where:

`ret_typ` is the type of the value returned by the function

`fun_name` is the name of the function

`typ_arg1` is the type of the first argument

`typ_arg2` is the type of the second argument

`...` further argument types up to eleven.

`= val` is an optional argument giving the value to be returned during simulations.

Example:

```
function Int my_fun(Widget, Int, String) =1;
```

declares the function `my_fun` which returns an `Int` whose value during simulations is 1.

In principle, such a function need be declared only once, the first time it is referred to. When in doubt as to the order in which scripts are analyzed, you can insert the declaration before each reference to the function.

FACE functions can also be declared as `global`:

```
global function None Hello(String message)
{
    printf("Hello, %s\n", message);
}
```

Global FACE functions differ from non-global functions only in the way their are output in the generated C code.

6.6 Floating-Point Variables

Single-precision floating-point values are implemented in FACE. A floating-point constant is of the form `i.f`, where `i` and `f` are decimal numbers. Floating-point values work as in C, with the following restrictions in FACE:

- Only the first six arguments of a function call can be floating-point values;
- There is no automatic conversion between floating-point and integer (or other) values; values can be explicitly converted using the FACE functions `cftoi` and `citof`, which convert a float to an integer and vice-versa; the definition of the conversion (truncating or rounding) is the same as the definition of the conversion operator in C, and is machine-dependent;
- An arithmetic operator is assumed to be a floating-point operator if one of its arguments is a float, but the other argument is not converted automatically to float: the converter `citof` must be used if needed; if a non-float argument is passed to a float operator, a parse error occurs;
- Floating-point arguments are passed correctly to functions only in the following cases:
 - m The function is declared as taking an argument of type `XtRFloat` with `FmAttachFunction`,
 - m The argument is a floating-point literal,
 - m The argument is a statically typed variable (or a *defined* constant) of type `Float`,
 - m The argument is a call to a function returning a float.

The last case makes it possible to force `XFaceMaker` to pass an argument as a float, even if the function was not declared as taking a float, by using the `Float` type caster.

For example, `printf` accepts arguments of unspecified type: to pass a float, you must cast it to `Float`, as follows:

```
pi = 3.1415926;
printf("pi=%f\n", Float(pi));
```

Another solution is to use a statically typed variable, or a constant:

```
define pi 3.1415926;
Float pi2 = pi/2.0;
printf("pi=%f, pi2=%f\n", pi, pi2);
```

If a FACE-attached application function returns a float, it must be declared as returning a value of type `XtRFloat` in `FmAttachFunction`, otherwise the FACE interpreter will get an incorrect return value. Similarly, in compiled interfaces, (C-code versions), if an application function returns a float this must be specified in the FACE script using the `function` keyword:

```
function    Float    myfunction(type1, type2, ...)
```

6.7 Using Arrays in FACE

FACE arrays are a form of association table which can be indexed by strings, integers or arbitrary values like pointers. FACE provides two kinds of arrays: hash arrays and linear arrays.

A FACE hash array is a hash table which can be indexed by a string key or by an arbitrary one-word key. In a string-indexed hash array, an element is stored and accessed using the value of a null-terminated character string: two strings which are lexically equal will give access to the same element, even if they are stored at different addresses. In a value-indexed hash array, only the immediate value of the index is used, and only two values which are arithmetically equal give access to the same element.

A FACE linear array is similar to a C array. The elements are stored in a contiguous area of memory, and are indexed by integers only. The memory for a linear array is allocated and expanded automatically as needed. FACE linear arrays can give access to C arrays defined by the application or by Motif widgets, for example the array of items in an *XmList*.

6.7.1 Declaring Arrays

FACE arrays are special variables which must be declared with the following syntax:

```
[<element-type>]  
<variable-name> "[" [<key-type> ["<default-value>"]]"
```

element-type is the FACE type of the elements of the array; if an element type is specified, the array elements will behave like statically-typed variables with type checking, otherwise, any type can be stored in the array, and all array elements will have the type Any;

variable-name is the name of the FACE variable which will hold the array;

key-type is the FACE type of the keys used to index the array; if the key type is **String**, then the array will be indexed by strings, otherwise it will be indexed by values. If the key type is not specified, it defaults to **NONE**, and the array is indexed by values; for linear arrays, the key type must be **Int**;

default-value allows to specify a value which will be returned when an un-initialized element is accessed; this value must be a constant; it is 0 if not specified.

For example, here is how to declare a hash array `a` of integers indexed by strings and with a default value of `-1`:

```
Int a[String, -1];
```

An array can be declared anywhere, as for other FACE variables, and can be declared `local` or `global`. The scope rules are the same as for ordinary variables.

FACE function parameters can be declared as arrays, for example:

```
function String Foo(String a[Int], Int i)
{
    return(a[i]);
}
```

Array variables can be used as any other variables. In particular, they can be passed as parameters to functions:

```
String a[Int];
...
Foo(a, 10);
```

The same array can be accessed using different array variables declared with different types. For example, say you want to access a hash array sometimes by strings, and sometimes by values. To do this, you can declare two array variables and assign the value of one to the other:

```
Int indexed-by-ints[Int];
...
Int indexed-by-strings[String];
...
indexed-by-strings = indexed-by-ints;
...
```

It is also possible to declare an array variable without actually declaring the array. To do this, declare an ordinary variable of type `FaceArray`:

```
FaceArray a;
```

Such a variable can hold the value of any array, but it cannot be used to access the elements of the array.

6.7.2 Allocating Arrays

Declaring a FACE array does not allocate any storage for it: the variable is initialized to 0. Before an array can be used, it must be allocated by calling the `new_table`, `new_array` or `new_c_array` functions.

The `new_table` function creates a new hash array. It takes no arguments.

```
Int a[String];  
a = new_table();
```

The `new_array` function creates a new linear array. The storage for the elements will be allocated and expanded automatically when required:

```
Int a[Int]  
a = new_array();
```

The `new_c_array` function also creates a new linear array, but allows you to initialize the array with a C array. It takes two arguments: the first argument is of type `Pointer` and specifies the C array; the second argument is of type `Int` and specifies the size of the C array.

```
XmString items[Int]  
a = new_c_array  
    (Pointer(parent.list.items), parent.list.itemCount);
```

The C array passed to `new_c_array` must be an array of elements of the size of an `XtArgVal`. An initialized array is *not* expanded automatically. When the array is not used any more, it should be freed by calling the `destroy` function:

```
destroy(a);
```

When an initialized linear array is freed, the C array passed to `new_c_array` is not freed. FACE arrays are allocated from the heap, so the storage of an array remains valid, regardless of the scope of the variable which holds it, until a call to `destroy` is made. In particular, an array allocated in a function is not freed automatically when the function exits.

6.7.3 Accessing Array Elements

Array elements are accessed with the following syntax:

```
<array-variable> "[" <key> "]"
```

Array elements can be used in expressions and as the left-hand side of assignments:

```
Int a[Int] = new_array();  
a[10] = 20;  
printf("element no. 10 of array a is: %d\n", a[10]);
```

Array keys can be arbitrary expressions. If the array has been declared with a key type, the type of the key is checked against the declared key type. In an assignment, if the array has been declared with an element type, the type of the right-hand side is checked.

Writing to a non-existing entry of a hash array creates a new entry. Writing beyond the current bound of a linear array expands the array automatically, except if the array was initialized using `new_c_array`. If the index used to write to a linear array is greater than the last index (i.e., greater than or equal to the size of the array), the array is also expanded automatically, but any “gap” is filled with zeros, (*not* with the array’s default value).

Reading from an array element which was not previously written to, returns the default value specified when the array was declared, or 0 if no default value was specified. Reading from an unallocated array (i.e. an array variable with a null value) also returns the default value. Writing to an unallocated array does nothing.

6.7.4 Iterating Through Arrays

There are two functions which allow the programmer to iterate through the elements of an array.

- The `first` function returns an initial key for an array. It takes two parameters: an array variable, and a key value which will be returned if the array contains no elements.
- The `next` function returns the *next* key in an array. It takes three parameters: an array variable, the previous key returned by `first` or `next`, and a key value which is returned if the end of the array is reached.

```
Int a[String];
...
for(String s = first(a, 0); s != 0; s = next(a, s, 0)){
    printf("element %s = %d\n", s, a[s]);
}
```

Notes:

- It is guaranteed that the key for each element of the array will be returned exactly once by the `first` and `next` functions, but for a hash array the order in which the keys are returned is **not** specified. For a linear array, though, the indexes are returned sequentially.
- The value passed to `first` and `next` as the end value must not be the key for a valid element of the array.

These iteration functions are most useful with hash arrays. To iterate through linear arrays, one can simply increment an integer index and use the `size` function.

6.7.5 Getting the Size of an Array

The `size` function returns the number of elements of an array. For hash arrays, it is the number of values stored in the array. For linear arrays, it is the largest index used to store a value minus one, or the number of elements specified for an initialized array.

6.7.6 Getting the Address of an Array

The function `data` can be used to pass the address of a linear FACE array, (that is, an array created with `new_array` or `new_c_array`), to an application function. `data` returns a pointer to the actual data contained in the array. It takes one argument, which is the variable holding the array. For example:

```
String t[Int] = new_array();
t[0] = "Hello";
t[1] = "World";
...
MyFunction(data(t));
```

`MyFunction` receives a regular C array.

6.7.7 Clearing an Array

The `clear` function deletes all entries in an array, but not the array itself:

```
clear(a);
```

The array is then as if it had just been allocated. When an initialized linear array is cleared, the C array passed to `new_c_array` is not freed. It is not possible to delete a particular entry of an array; instead, the default value of the array can be stored as the element value. In that case, the `first` or `next` functions will return a valid key which will yield the default value.

6.8 Structures

A FACE structure is similar to a C structure, except that FACE provides access only to structure *pointers*. A structure can be declared either from the C code of an application, or inside a FACE script.

6.8.1 Declaring a Structure from a C application

To declare a structure of your application which you want to be able to use in FACE scripts, use the `FmRegisterStructures` function.

```
int  FmRegisterStructures(
    FmStructDesc *structs,
    int num_structs)
```

This function declares the structures described by *structs*, which points to the following type:

```
typedef struct _FmStructDesc {
    String name;
    String cname;
    FmFieldDesc *fields;
    int num_fields;
    String id_field;
    XtArgVal id_value;
} FmStructDesc;
```

name is the name of the structure, i.e. the FACE type which will be used to declare pointers to the structure. This can be different from the actual C identifier.

cname is the C name which will be used to generate references to the structure in the C code generated by XFaceMaker. It is the declaration that would be used in a C file to declare a pointer to the structure.

fields is an array of `FmFieldDesc` structures:

```
typedef struct _FmFieldDesc {
    String name;
    String type;
    int size;
    int offset;
} FmFieldDesc;
```

where:

name is the name of the field (this must be the same as the C identifier for the structure field);

type is the FACE type of the field;

size is the size in bytes of the field;

offset is the offset in bytes of the field from the beginning of the structure.

num_fields is the number of elements in *fields*.

id_field is currently unused, and must be set to 0.

id_value is currently unused, and must be set to 0.

The offset of a structure field can be computed using the `XtOffset` macro contained in `X11/Intrinsic.h`. The *size* can be computed using the `FmSize` macro, which accepts the same arguments as `XtOffset`, and which is defined in `Fm.h`.

6.8.2 Declaring a Structure in a FACE script

You can also declare a structure in a FACE script with the following syntax:

```
struct <struct-name> [<c-struct-name>] {  
  <field-type> <field-name>;  
  ...  
};
```

For structures declared in FACE scripts, all fields have a size of one FACE word. The C pointer type `c-struct-name` is optional. If it is specified, it must be defined in a header file included in the C file generated by XFaceMaker using the `-cinclude` option.

6.8.3 Declaring Structure Variables

Variables which are to hold a structure pointer must be declared with the type of the structure, and can be used as ordinary statically typed variables. For example, in the `activateCallback` of a push button, one might write:

```
XmPushButtonCallback cbs;  
cbs = $;
```

6.8.4 Referencing Structure Fields

The fields of a structure are accessed using the `->` operator. The resulting value has the type of the field, as declared in the structure descriptor.

```
XmPushButtonCallback cbs;  
cbs = $;  
printf("clicks: %d\n", cbs->click_count);  
cbs->click_count = 0;
```

In general, to use callback structures, use `XmxxxCallback` and *not* `XmxxxC-CallbackStructPtr`.

6.8.5 Allocating Structures

A new structure can be allocated in a FACE script using the function `new_struct`, which takes the name of the structure as parameter:

```
struct MyStruct {
  Int i;
  String s;
};
MyStruct ms = new_struct(MyStruct);
ms->i = 1;
ms->s = "Hello";
```

Structures are allocated from the heap, so there is no automatic de-allocation when a function exits. A structure allocated using `new_struct` can be freed when no longer needed using `free` or `XtFree`.

6.8.6 Arrays as Structure Elements

This example shows how to declare a structure which contains a FACE array, and initialize the array:

```
struct Mystruct {
  Int i;
  FaceArray a;
  Int j;
};
global Mystruct s;
s = new_struct(Mystruct);
s->a = new_array();
String a[Int] = s->a;
a[0] = "zero";
a[1] = "one";
a[2] = "two";
a[3] = "three";
```

APPENDIX A: Built-in FACE Functions

This appendix lists the general FACE functions and the FACE type-casting functions that can be called within a Draw script.

1 General FACE Functions

AddGrab(Widget *w*, Boolean *exclusive*, Boolean *spring*)

This function is used to redirect user input to a widget *cascade*. A *cascade* is a sequence of widgets popped up successively in answer to user actions. *exclusive* is a Boolean that specifies whether user events should be dispatched exclusively to this widget or also to previous widgets in the cascade. *spring* is a Boolean that specifies whether the user pressed a mouse button. Same as `XtAddGrab`.

Callback(Widget *w*, String *cbk_name*, Pointer *c_d*)

Call all procedures in the widget's *w* callback list, where *cbk_name* specifies the callback list, and *c_d* the client_data to be passed to each callback procedure in the list. Same as `XtCallCallbacks`.

CallCallbacks(Widget *w*, String *cbk_name*, Pointer *c_d*)

Call all procedures in the widget's *w* callback list, where *cbk_name* specifies the callback list, and *c_d* the client_data to be passed to each callback procedure in the list. Same as `XtCallCallbacks`.

ConfigureWidget(Widget *w*, Position *x*, Position *y*, Dimension *width*, Dimension *h*, Dimension *border_width*)

Move and resize widget *w*. *x* and *y* are the coordinates of the upper left corner. Same as `XtConfigureWidget`.

ConvertString(Widget *w*, String *from*, String *to_type*)

Return Type: Any

Explicitly convert the string *from* to the *to_type* resource type. This function calls `XtConvert`.

For example:

```
 pixmap=ConvertString(self, "50_foreground", \
                        "Pixmap");
```

converts the string "50_foreground" into the corresponding Pixmap.

DestroyWidget(Widget *w*)

Destroy widget `w`.

Free(Pointer p)

Free memory allocated using Malloc. Same as `XtFree`.

IsManaged(Widget object)

Return Type: Boolean

Determine if widget `object` is managed by its parent. Same as `XtIsManaged`.

IsSensitive(Widget w)

Return Type: Boolean

Return a Boolean to indicate if widget `w` is sensitive or not. Same as `XtIsSensitive`.

LowerWidget(Widget w)

Call `XLowerWindow` on `w`'s window.

Malloc(Int size) Pointer

Return Type: Pointer

Allocate `size` bytes of memory. Returns a pointer to the allocated memory. Same as `XtMalloc`.

ManageChild(Widget child)

Add widget `child` to its parent's list of managed children. Same as `XtManageChild`.

MapWidget(Widget w)

Map widget `w` to its display. Same as `XtMapWidget`.

MoveWidget(Widget w, Position x, Position y)

Move widget `w` to new position (`x`, `y`).

NameToWidget(Widget w, String name) Widget

Return Type: Widget

Get widget ID for widget `name` starting search in the hierarchy at widget `w`. Same as `XtNameToWidget`.

NewString(String source)

Return Type: String

Allocate memory and copy `source` in it. Return the copied string. The allocated memory should be freed with `free`. Same as `XtNewString`.

Popdown(Widget popup_shell)

Remove the shell widget `popup_shell` from the screen; it must have previously been *popped up* by `Popup`. Same as `XtPopdown`.

Popup(Widget popup_shell, Int grab_mode)

Make the widget `popup_shell` appear on the screen. Same as `XtPopup`.

RaiseWidget(Widget w)

Call `XRaiseWindow` on `w`'s X Window.

RemoveGrab(Widget w)

Remove the redirection of user input to a widget cascade. Any widgets between the most recent and `w` are removed from the cascade. Same as `XtRemoveGrab`.

ResizeWidget(Widget w, Dimension width, Dimension height, Dimension border_width)

Change the widget's width, height and border width. Same as `XtResizeWidget`.

SetKeyboardFocus(Widget subtree, Widget descendant)

Redirect keyboard input to a non-popup descendant of a `subtree` widget. Same as `XtSetKeyboardFocus`.

SetSensitive(Widget w, Boolean sensitive)

Call `XtSetSensitive` to specify whether a widget should be receptive to user input or not.

SetValues(Widget widget, Int num_resources, String resource1, None value1, String resource2, None value2,...)

Return Type: Int

Set multiple resource values in one call to **XtSetValues**. **Num_resources** specifies the number of resources to be set. A maximum of 5 resources can be set simultaneously. **Resource1** is the name of the first resource, **value1** is the value to which it will be set. The values must be given in their internal form, not in their String form. The internal form of a resource can be obtained by using **ConvertString**.

UnmanageChild(Widget child)

Remove widget **child** from its parent's managed list. Same as **XtUnmanageChild**.

UnmapWidget(Widget w)

Explicitly unmap widget **w**.

Warning(String m)

Call the installed low-level warning handler, reporting message **m**. Same as **XtWarning**.

atoi(String s)

Return Type: Int

Return the conversion of ASCII string **s** into an integer.

callback(Widget w, String s, Int i)

Return Type:

See **CallCallbacks**.

cftoi(Float f)

Return Type: Int

Convert a floating-point number to an integer.

citof(Int i)

Return Type: Float

Convert an integer to a floating point number.

clear(FaceArray a)

Delete all entries in array **a** but do not destroy the array itself.

data(FaceArray a)

Return Type: Pointer

Return a pointer to C array *a*.

destroy(FaceArray a)

Free array **a**.

dtoi(Pointer dp) Int

Return Type: Int

Return the conversion of the **double** pointed to by **dp** into an integer.

eval_string(Widget w, String text, Pointer pt) Int

Return Type: Int

Interpret the **text** given as a FACE script, where *self* is replaced by **w**, **\$** by **pt** and **@** by ***pt**.

first(Int a, Int key)

Return Type: According to **key** type of array declaration. See XFM documentation on arrays.

Return an initial key for an array. **a** : array variable, **key** value which will be returned if the array contains no elements.

free(String s)

This is the standard **free** C function.

ftoi(Pointer fp) Int

Return Type: Int

Return the result of converting a **float** pointed to by **fp** into an integer.

itoa(Int i) String

Return Type: String

Convert integer **i** into a statically allocated string.

itod(Int i, Pointer dp)

Convert integer **i** into a **double** and store the result in the **double** variable pointed to by **dp**.

itof(Int i, Pointer fp)

Convert integer **i** into a **float** and store the result in the **float** variable pointed by **fp**.

malloc(Int i)

Return Type: String

This is the standard C function.

new_array()

Return Type: FaceArray

The **new_array** function creates a new linear array.

new_c_array(Pointer c_array, Int size)

Return Type: FaceArray

Create a new linear array from an array declared in C whose address is **c_array** and whose size is **size**.

new_struct(String struct_name)

Return Type: Any

Allocate a new structure of type **struct_name**.

new_table()

Return Type: FaceArray

Create a new hash array.

next(Int a, key s, key n)

Return Type: According to **key** type of array declaration. See XFM documentation on arrays.

Return the *next* key in array **a**. **s** is the previous key returned by **first** or **next**. **n** is the value returned if the end of the array is reached.

popdown()

See Popdown.

printf(None f, ...)

Call `printf` to output a message on `stdout`; the first parameter is a format string determining what other parameters are required, as in C. No type checking is done.

random(Int num)

Return Type: Int

Return a random integer whose value lies between 0 and num.

return(Int i)

Return Type: Int

Same as return in C.

size(a)

Return Type: Int

Return the number of elements of array `a`.

sprintf(None targ, None f, ...)

Call `sprintf` to store a message in String-buffer `targ`. No type checking is done.

sscanf(None src, None format, ...)

Call `sscanf` to scan for `format` in the String `src`. No type checking is done.

strcmp(String s1, String s2) Int

Return Type: Int

Call the C `strcmp` function.

strcpy(String targ, String src)

Call the C function `strcpy` to copy String `src` to String `targ`.

strlen(String src)

Return Type: Int

Call the C `strlen` function.

strncmp(String s1, String s2, Int i)

Call the C `strncmp` function.

2 FACE Type-casting Functions

Type-casting functions do no type conversion. They are intended to be used only to satisfy the type checking requirements of FACE. You can create your own type casters using the `type` keyword. The following is a list of pre-defined FACE type-casting functions that can be called within a Draw script:

Any(None)

Return Type: Any

Description: Type caster.

Array(None)

Return Type: Array

Description: Type caster.

CursorPosition(Int)

Return Type: CursorPosition

Description: Type caster.

Float(None)

Return Type: Float

Description: Type caster.

Image(None)

Return Type: Image

Description: Type caster.

Int(None)

Return Type: Int

Description: Type caster.

MenuWidget(None)

Return Type: MenuWidget

Description: Type caster.

Pixmap(None)

Return Type: Pixmap

Description: Type caster.

Pointer(None)

Return Type: Pointer

Description: Type caster.

String(None)

Return Type: String

Description: Type caster.

Widget(None)

Return Type: Widget

Description: Type caster.

Window(None)

Return Type: Window

Description: Type caster.

XmBackgroundPixmap(None)

Return Type: XmBackgroundPixmap

Description: Type caster.

APPENDIX B: Configuring XDrawMaker

1 Command Line Options

The command line options listed below can be used to change some of the resource settings at the time the program is launched. Note that resource settings using the command line options supersede resource settings specified in the `.Xdefaults` file.

- backup** XDrawMaker will copy files before saving. The corresponding resource is `backUp` (Boolean).
- colors** Specify the file containing XDrawMaker colors specification. The resource is `colors:<color file>`.
- compile** Compile a `.draw` file to a `.fm` file.
- custom** Use XDrawMaker as a custom editor for XFaceMaker (do not use directly).
- dumpcore** Dump a core file on fatal error.
- fz** During zoom procedures, XDrawMaker will zoom only the perimeter of the objects, not the contents of the objects. This results in a much faster zoom time, particularly for label and image objects. The resource is `fastZoom`.
- help** Print available options and exit.
- imgdir** Specifies the image directory to read at startup. The corresponding resource is `imageDirectory:<directory name>`.
- indicator** Specify the incrementation of the load indicator in the main window. The resource is `indicator`.
- owncmap** XDrawMaker uses its own colormap. The corresponding resource is `ownCmap` (Boolean).
- printcmd** Specify the print command for the Print Box. The corresponding resource is `printCmd` (XtRString).
- revision** Print revision number and exit.
- rfn** Specify the ruler font. The corresponding resource is `ruler-Font:<font name>`.
- ri** Keep the original size of images in the image box. The correspond-

	ing resource is <code>resizeIcon</code> (Boolean).
-sfn	Specify the script window font. The corresponding resource is <code>scriptFont:</code> .
-sfg	Specify the script window foreground. The corresponding resource is <code>scriptForeground:<color></code> .
-sbg	Specify the script window background. The corresponding resource is <code>scriptBackground:<color></code> .
-urc	XDrawMaker allocates an RGB color cube for the pixmap color management. The corresponding resource is <code>useRgbCube</code> (Boolean). The size of the RGB cube lies between 6 (216 colors) and 3 (27 colors), depending on the number of colors that can be allocated in the colormap. XDrawMaker creates the largest possible RGB cube, or none if it cannot allocate up to 27 colors. Image handling is faster and rendition is better with an RGB cube.
-verbose	Messages are printed during execution. The corresponding resource is <code>verbose</code> (Boolean).

2 Environment Variables

DMIMAGEDIR	Specifies the directory of images to be loaded for use in XDrawMaker.
DM_LICENSE_NAME	If you set this variable to <code>xdm2</code> , XDrawMaker will call license server version 3.0. If it is not found, XDrawMaker will then try license server version 2.0. If you do not set this variable, XDrawMaker will try <i>only</i> the license server version 2.0.
DRAWPATH	A variable used by XDrawMaker to define the directories searched to find all <code>.draw</code> files. <code>%D</code> is used as the substitution character for the <code>.draw</code> file name. For example, if your <code>.draw</code> files are in the directory <code>/dir/Appli/Draw</code> , set this variable to <code>/dir/Appli/Draw/%D</code> .
DRAWPIXMAPSPATH	This variable is used to define the directories

	searched to find bitmap files. %B is used as the substitution character for the bitmap file name. Equivalent to XBMLANGPATH but applied first.
DRAW_RGB_FILES	If set, this file is used to determine the correspondence of a color name and its RGB components. If not set, then the file <code>/usr/lib/X11/rgb.txt</code> is used.
FMEDITOR	Defines the text editor that will be called when the Custom Editor button is selected in one of the XDrawMaker editing windows. Specify <code>wx242</code> to use the Wx text editor from NSL.
NSLHOME	Used to access all the configuration files associated to XDrawMaker.
NSL_LICENSE_PORT	Used to specify the port number that the license server daemon should use instead of the default port number (5002).
XBMLANGPATH	Standard X variable used to define the directories searched to find bitmap files. %B is used as the substitution character for the bitmap file name.
XFILESEARCHPATH	Used to define the directories searched to find application resource files.

APPENDIX C: The XDrawMaker Menus

To open an XDrawMaker menu, click on its name in the menu bar. To select a menu item, click on it or enter the key combination that is listed to the right of the item. Items that are not yet implemented are dimmed. Some menu items are followed by an ellipsis. Clicking on one of these items will pop up a dialog box for you to enter further information, such as a filename. The XDrawMaker menus and options are as follows.

File This menu contains the XDrawMaker commands for opening, saving, and printing files. It has the following items:

- m **New** (*Ctrl-n*) clears the XDrawMaker desktop and all images but not colors you have set, so that you can begin a new Draw file. If the current file hasn't been saved, an Alert box will warn you.
- m **Open...** (*Ctrl-o*) pops up a dialog box for selecting an existing file to open, i.e. to load, clearing any existing objects on the desktop.
- m **Read...** (*Ctrl-r*) pops up a dialog box for choosing a file to be added to the current XDrawMaker window, i.e the new file is concatenated with any existing objects on the desktop.
- m **Save** (*Ctrl-s*) saves the current content of the desktop in the current file.
- m **Save As...** (*Ctrl-a*) pops up a dialog box for entering a new file name before saving your work. The file you enter becomes the current file. The objects in the desktop are saved to the file specified.
- m **Print...** pops up the Print Box which can be used to print or to save to a PostScript file for printing later.
- m **Export** opens a cascade menu with the choices below. Each of these options opens the File Selector to let you specify a file name:
 - **XFaceMaker File** Generate a .fm file. The file can be loaded in XFaceMaker for use in an application, or to generate the C code description of the draw file.
 - **PostScript File** Generate a .ps file. The file can be printed on a PostScript printer.
 - **Fax Group 4 File** Generates a .tif or .fax file. The file can be used as a CCITT Group 4 fax.
- m **Exit** (*Ctrl-e*) exits from XDrawMaker. If you have modified your file, an Alert box will warn you.

Edit This menu contains the following editing commands:

- m **Cut** (*Ctrl-x*) cuts the selected object or group, and places it into a temporary buffer.
- m **Copy** (*Ctrl-c*) copies the selected object or group into a temporary buffer.
- m **Paste** (*Ctrl-v*) pastes the contents of the temporary buffer at the designated point.
- m **Duplicate** (*Ctrl-d*) duplicates the selected object or group, attributing default names to duplicated objects.
- m **Delete** (*Ctrl-k*) deletes the selected object.
- m **Select All** (*Ctrl-/*) selects all objects on the desktop.
- m **Delete All** deletes all objects on the desktop.

View This menu contains the following viewing options:

- m **Hide** hides the selected object.
- m **Show** re-displays the selected object.
- m **Hide All** hides all objects on the desktop.
- m **Show All** displays all objects, including hidden objects.
- m **Grid** is a toggled item which displays the grid when set.
- m **Page Size** displays the specified page size on your desktop, as an outline delimiting the page on the desktop.
- m **Zoom In** enables you to enlarge an area of the desktop you define by drawing a rubber band box. The Zoom factor is such that the rubber band box is fit to the size of the XDrawMaker window. Note: as long as a Zoom factor is applied to the drawing, the Geometry Box is insensitive.
- m **Undo Zoom** undoes the last zoom.
- m **Reset Zoom** resets the enlarged area to its original size, i.e. undoes all the levels of Zoom.

Options This menu contains the following commands:

- m **Page Description...** pops up the Page Settings box for choosing options such as page size and orientation.
- m **Grid Settings...** pops up the Grid box for choosing a grid style, and for setting constraint values for rotating or scaling objects.
- m **Tree...** opens the Tree window which displays and lets you modify the names

of the objects in your drawing. The names are displayed as a tree structure, i.e. members of a group are displayed as children of the group.

m **Attribute Boxes...** opens a cascade menu with the following choices of editing boxes to pop up:

- **Geometry Box...** is used to specify an object's size, position, pre an post scale, rotation angle, center. You can also click on the corresponding Edit icon in the Main window to pop up this box. Note that, if a Zoom is currently applied on the drawing, the Geometry Box is insensitive,
- **Graphic Attributes...** is used to specify the graphic characteristics of an object such as color, stipple, fill style, etc. You can also click on the corresponding Edit icon in the Main window to pop up this box.
- **Dash Definition...** is used to specify a style for dashed lines. You can also click on the corresponding Edit icon in the Main window to pop up this box.
- **Arrow Box...** is used to specify a style for arrows. You can also click on the corresponding Edit icon in the Main window to pop up this box.
- **Slot Box...** is used to specify the size and image (if any) for slots. You can also click on the corresponding Edit icon in the Main window to pop up this box.

m **Object Boxes** opens a cascade menu with the following choices of editing boxes to pop up:

- **Group Edit Box...** pops up the Group Edit Box used to modify the list of objects in a group, to specify how geometric attributes are applied to members of the group, to change the order of points defining an object, to change the order of objects in the group, etc.. You can also click on the corresponding Edit icon in the Main window to pop up this box.
- **Text Options...** pops up the Text Options box for specifying the characteristics of text in a text object. This box lets you specify text segment characteristics such as font, foreground and background color, superscript/subscript, paragraph alignment, line and paragraph spacing, resize policy, hyper text, etc. You can also click on the corresponding Edit icon in the Main window to pop up this box.
- **Label Box...** pops up the Label Box for specifying the characteristics of text in a label object. This box lets you specify the label text, its Font, its alignment and position within the label's outline, how the label text should be transformed. Use this box to specify the number of colors allocated in anti-aliasing calculations in rotations of the label text. You can also click on the corresponding Edit icon in the Main window to pop up this box.

-
- **Image Box...** pops up the Image Selection box for loading or choosing which pixmap to load in an Image object in your draw file. The image types allowed are bitmap, xwd (XWindow Dump) gif, xpm3. Any images that are already loaded in XDrawMaker are displayed in this box. Use this box to specify whether the selected image object should be resized to fit the size of the image you specify, or should keep its own size. You can also click on the corresponding Edit icon in the Main window to pop up this box.
 - **Expert Box...** pops up the Expert box used to set resource values for resources not available in the XDrawMaker menus or editing boxes. Save your work before specifying resources in the Expert Box as conflicting resource settings may cause XDrawMaker to crash.
- m **Snap to Grid** is a toggled item which, when set, causes selected objects to align to grid points when they are moved or resized. Any new objects drawn on the desk top will snap to the grid points when this toggle is set.
- m **Double Buffering** is a toggled item. When set, objects are copied into an intermediate buffer. This eliminates “flashing” when the screen is refreshed.
- m **Resize Line Width** is a toggled item. When set, the width of all lines is resized when a drawing is scaled.
- m **Icon Bar** is a toggled item. When set, the Editing Box icons on the right hand side of the main window are displayed.

Transform This menu contains the following items:

- m **Rotate** changes the pointer to the rotation symbol (*exchange*) used to rotate objects. Rotate the object then click the *select* icon or click mouse **Button 3** to quit this mode.
- m **Pre Scale** changes the pointer into the scale symbol (*resize*) used to scale objects. Scale the object then then click the *select* icon or click mouse **Button 3** to quit this mode.
- m **Post Scale** changes the pointer into the scale symbol (*resize*) used to scale objects. Scale the object then click the *select* icon or click mouse **Button 3** to quit this mode.
- m **Define Center** changes the pointer into a small cross (*cross*) used to designate a new center point for an object. Move the cursor to where you want the center and click. A tiny cross is drawn to materialize the object’s new center.
- m **Reset Rotation** resets a selected object’s rotation to angle 0.0.
- m **Reset Pre Scale** resets a selected object to scale 1.0 for the scaling factor applied before rotation.

-
- m **Reset Post Scale** resets a selected object to scale 1.0 for the scaling factor applied after rotation.
 - m **Reset Center** resets the center of an object to its original position, i.e. the geometric center of the object or group.
 - m **Flip Horizontal** flips the selected object (provided it is a polygonal object, i.e. line, arc, spline, polygon, or segment) horizontally -- as in a mirror image.
 - m **Flip Vertical** flips the selected object (provided it is a polygonal object, i.e. line, arc, spline, polygon, or segment) vertically -- as in a mirror image.
 - m **Add Point** changes the pointer to a small cross for adding points to splines and polygons. Place the cursor in the position where you want the point to be added. A ghost image of the new segments this point will define is drawn to help you determine the point's correct position.
 - m **Remove Point** changes the pointer into a small cross for removing points from splines and polygons. Move the cursor over the point you want to delete and click.

Arrange This menu contains the following items:

- m **Bring to Front** makes the selected object the top object of a stack of overlapping objects.
- m **Send to Back** makes the selected object the bottom object of a stack of overlapping objects.
- m **Circulate Up** brings the selected object up one position in the stack of overlapping objects.
- m **Circulate Down** moves the selected object down one position in a stack of overlapping objects.
- m **Selection to** opens a cascade menu with the following choices:
 - **Group** makes a group of selected objects into a single object, or a “permanent group”.
 - **Path** connects a set of selected polygonal objects; i.e. arcs, splines, polygons, lines, and segments. Use the Group Editing Box to modify the order of objects in the Path, or the drawing direction of an object.
- m **Ungroup** disbands a selected “permanent group” into individual objects, or disconnects a set of connected polygonal objects.
- m **Align...** pops up the Alignment box for aligning a selected object on the grid.

Activate This menu has commands and items enabling you to activate your draw-

ings. It contains the following items:

- m ***Script*** pops up the Script Editing box used for writing scripts.
- m ***Run in DrawMaker Window*** This command executes the scripts in the XDrawMaker window. This is faster than the ***Launch a Separate Process*** command but if an error is encountered in a script, XDrawMaker may crash!
- m ***Launch a Separate Process*** This command spawns a new process which executes the scripts associated to objects.



Help This menu contains the following item:

- m ***Version ...*** Opens the box giving the version information of the product.
- m ***Contact NSL ...*** when clicked, connects your Netscape browser to NSL's www site.
- m ***HTML Documentation ...*** when clicked, and if the documentation has been installed on your disk, connects your Netscape browser to the product documentation in HTML format.

APPENDIX D: XDrawMaker Functions

This appendix describes the Draw functions that are available in XDrawMaker and the XDraw widget. These functions deal with object management (including geometric and graphic transformations), object creation, event management and script management.

The return values and parameter types are the types understood by the FACE language and are different from those known by a C compiler.

The functions listed here are a subset of the available functions.

For more information on resource values, or on structures, callbacks and their related resources, refer to the Draw widgets' manual pages.

The functions are divided into the following categories:

Arrow Functions	page 142
Bitmap Table Functions	page 146
Color Table Functions	page 154
Object Creation Functions	page 159
Event Functions	page 172
Font Table Functions	page 180
Geometric Transformation Functions	page 186
Graphic Transformation Functions	page 217
Input/Output Functions	page 239
Miscellaneous Functions	page 244
Object Management Functions	page 260
Pixmap Table Functions	page 317
Script Functions	page 324
Slot Functions	page 331
Text Functions	page 340

1 Arrow Functions

The functions in this section are used to specify arrow attributes.

NAME

XnslDrawSetArrowDimension

SYNOPSIS

```
None XnslDrawSetArrowDimension (w, o, width, height)
      Widget          w;
      Widget          o;
      Int             width;
      Int             height;
```

DESCRIPTION

XnslDrawSetArrowDimension sets the arrow size of object *o* in widget *w* to the given *width* and *height*. *width* and *height* are expressed in pixels.

Object *o* must belong to one of the following classes:

- XnslDrawArc
- XnslDrawLine
- XnslDrawPolygon
- XnslDrawSegment
- XnslDrawSpline

NAME

XnslDrawSetArrowDirection

SYNOPSIS

```
None      XnslDrawSetArrowDirection (w, o, dir)
          Widget      w;
          Widget      o;
          Int         dir;
```

DESCRIPTION

XnslDrawSetArrowDirection sets the direction of the arrow of object *o* in widget *w* to *dir*. Possible values for *dir* are:

- | | |
|-------------------------------|--|
| XnslDRAW_NO_ARROW | No arrow is displayed. |
| XnslDRAW_FROM_ARROW | The arrow is displayed at the first point of the widget. |
| XnslDRAW_TO_ARROW | The arrow is displayed at the last point of the widget. |
| XnslDRAW_FROM_TO_ARROW | Two arrows are displayed. |

Object *o* must belong to one of the following classes:

- XnslDrawArc
- XnslDrawLine
- XnslDrawPolygon
- XnslDrawSegment
- XnslDrawSpline

NAME

XnslDrawSetArrowStyle

SYNOPSIS

```
None      XnslDrawSetArrowStyle (w, o, style)
          Widget      w;
          Widget      o;
          Int          style;
```

DESCRIPTION

XnslDrawSetArrowStyle sets the arrow style of object *o* in widget *w* to the given *style*. Style can be one of the following values:

XnslDRAW_ARROW_FILLED	The arrow is a filled triangle.
XnslDRAW_ARROW_OPEN	The arrow is drawn as two lines.
XnslDRAW_ARROW_CLOSED	The arrow is drawn as a triangle.

The object passed to this function must be of one of the following classes:

- XnslDrawArc
- XnslDrawLine
- XnslDrawPolygon
- XnslDrawSegment
- XnslDrawSpline

2 Bitmap Table Functions

The functions in this section are used to work with bitmaps in the bitmap table and with stipples.

NAME

XnslDrawBitmapName

SYNOPSIS

```
String    XnslDrawBitmapName (w, index)
Widget    w;
Int       index;
```

DESCRIPTION

XnslDrawBitmapName returns the name of the bitmap at the given *index* in the internal bitmap table of widget *w*. This is the same name that the application has passed to **XnslDrawGetBitmap**.

NAME

XnslDrawBitmapPixmap

SYNOPSIS

```
Pixmap    XnslDrawBitmapPixmap(w, index)
Widget    w;
Int       index;
```

DESCRIPTION

XnslDrawBitmapPixmap retrieves the pixmap id of a bitmap in widget *w*, knowing its *index*.

NAME

XnslDrawBitmapStipple

SYNOPSIS

```
Pixmap XnslDrawBitmapStipple (w, i)
      Widget      w;
      Int         i;
```

DESCRIPTION

XnslDrawBitmapStipple returns the Pixmap whose index is *i* in the internal table.

NAME

XnslDrawClearBitmapTable

SYNOPSIS

```
None      XnslDrawClearBitmapTable (w, free)
Widget    w;
Int       free;
```

DESCRIPTION

XnslDrawClearBitmapTable frees all bitmaps in the table. If *free* is **True**, it also frees the memory for the table itself. The application should pass **False** for *free* if it expects to load new bitmaps in the table of widget *w*.

NAME

XnslDrawGetIndexBitmap

SYNOPSIS

```
Int  XnslDrawGetIndexBitmap (w, b)
Widget      w;
Pixmap      b;
```

DESCRIPTION

XnslDrawGetIndexBitmap looks for bitmap *b* in the internal bitmap table of widget *w*. If it finds the bitmap, it returns its index, otherwise it returns -1.

NAME

XnslDrawGetStipple

SYNOPSIS

```
Int  XnslDrawGetStipple(w, o)
      Widget      w;
      Widget      o;
```

DESCRIPTION

XnslDrawGetStipple returns the index corresponding to the Stipple attribute of object *o*.

NAME

XnslDrawGetStipplePixmap

SYNOPSIS

```
Pixmap    XnslDrawGetStipplePixmap (w, o)
Widget      w;
Widget      o;
```

DESCRIPTION

XnslDrawGetStipplePixmap returns the Pixmap corresponding to the stipple defined for widget *o*.

3 Color Table Functions

The functions in this section are used to work with colors in the color table of the widget.

NAME

XnslDrawClearColorTable

SYNOPSIS

```
None      XnslDrawClearColorTable (w, free)
Widget    w;
Int       free;
```

DESCRIPTION

XnslDrawClearColorTable frees all colors in the table. If *free* is **True**, it also frees the memory for the table itself. The application should pass **False** for *free* if it expects to load new colors in the color table of widget *w*.

NAME

XnslDrawColorName

SYNOPSIS

```
String    XnslDrawColorName (w, index)
Widget    w;
Int       index;
```

DESCRIPTION

XnslDrawColorName returns the color name for the color at the given *index* in the internal color table of widget *w*. If *index* is not within the table range, NULL is returned.

NAME

XnslDrawColorPixel

SYNOPSIS

```
Pixel      XnslDrawColorPixel (w, index)
Widget      w;
Int         index;
```

DESCRIPTION

XnslDrawColorPixel returns the pixel value of the color at the given *index* in the internal color table of widget *w*.

NAME

XnslDrawGetColor

SYNOPSIS

```
DrawPixel      XnslDrawGetColor (w, name)
Widget         w;
String         name;
```

DESCRIPTION

XnslDrawGetColor searches for a color with the given name *name* in the color table, and if it cannot find it, loads a new color into the color table of widget *w*. **XnslDrawGetColor** returns the index of the color in the internal color table. The name may either be a name in the RGB data base, or a color value beginning with a hash sign (#). If it cannot find the color and cannot load the given name, **XnslDrawGetColor** returns -1.

The index can be used for `XnslDrawSetForegroundColor()` or `XnslDrawSet-BackgroundIndex()`.

4 Object Creation Functions

The functions in this section should be used to create objects.

NAME

XnslDrawCreateArc

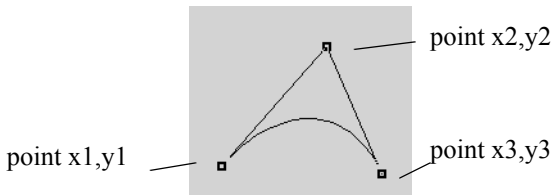
SYNOPSIS

```
Widget      XnslDrawCreateArc(w, coord, num_coord)
Widget      w;
Float       *coord;
Int         num_coord;
```

DESCRIPTION

This function creates an **XnslDrawArc** object in widget *w*. *coord* can be a pointer to a FACE Array of float values which are the coordinates of the arc defining points. You must pass six coordinates, i.e. three points, to create an **XnslDrawArc**.

Here is how an arc is specified:



Use **XtManageChild** to manage the object.

NAME

XnslDrawCreateDrawObject

SYNOPSIS

```
Widget    XnslDrawCreateDrawObject(name,
                                   class_name, parent,
                                   coordinates, n_coordinates)
String                                          name;
String                                          class_name;
Widget                                          parent;
XnslDrawPoint                                coordinates;
Int                                              n_coordinates;
```

DESCRIPTION

This function create an XnslDrawObject of name *name*, of class *class_name*, with *parent* as parent. The definition points *coordinates* and *n_coordinates* can be taken directly from the XnslDrawDoneCallbackStruct fields. The *class_name* can be one of the following strings:

- XnslDrawArc
- XnslDrawEllipse
- XnslDrawImage
- XnslDrawLabel
- XnslDrawLine
- XnslDrawPolygon
- XnslDrawRectangle
- XnslDrawSegment
- XnslDrawSpline
- XnslDrawText
- XnslDrawGroup
- XnslDrawPath

NAME

XnsIDrawCreateEllipse

SYNOPSIS

```
Widget XnsIDrawCreateEllipse(w, x, y, width, height)
      Widget      w;
      Float      x;
      Float      y;
      Float      width;
      Float      height;
```

DESCRIPTION

This function creates a **XnsIDrawEllipse** object in widget *w*. The ellipse is inscribed in a rectangle whose upper left hand corner is at coordinates *x, y* and whose extension is given by *width* and *height*.

NAME

XnslDrawCreateImage

SYNOPSIS

```
Widget XnslDrawCreateImage(w, x, y, width, height)
                        Widget      w;
                        Float      x;
                        Float      y;
                        Float      width;
                        Float      height;
```

DESCRIPTION

This function creates a **XnslDrawImage** object in widget *w*. The image is inscribed in a rectangle whose upper left hand corner is at coordinates *x*, *y* and whose extension is given by *width* and *height*.

NAME

XnslDrawCreateLabel

SYNOPSIS

```
Widget XnslDrawCreateLabel(w, x, y, width, height)
      Widget      w;
      Float      x;
      Float      y;
      Float      width;
      Float      height;
```

DESCRIPTION

This function creates a **XnslDrawLabel** object in widget *w*. The label is inscribed in a rectangle whose upper left hand corner is at coordinates *x*, *y* and whose extension is given by *width* and *height*.

NAME

XnslDrawCreateLine

SYNOPSIS

```
Widget  XnslDrawCreateLine(w, x1, y1, x2, y2)
                        Widget      w;
                        Float      x1;
                        Float      y1;
                        Float      x2;
                        Float      y2;
```

DESCRIPTION

This function creates a line object in widget *w*. The line connects point *x1,y1*, to point *x2,y2*.

NAME

XnslDrawCreatePolygon

SYNOPSIS

```
Widget XnslDrawCreatePolygon(w, coord, num_coord)
      Widget      w;
      Float      *coord;
      Int        num_coord;
```

DESCRIPTION

This function creates a polygonobject in widget *w*. *coord* can be a pointer to a FACE Array of float values which are the coordinates of the polygon vertices.

NAME

XnslDrawCreateRectangle

SYNOPSIS

```
Widget XnslDrawCreateRectangle(w, x, y, width,  
height)  
    Widget      w;  
    Float      x;  
    Float      y;  
    Float      width;  
    Float      height;
```

DESCRIPTION

This function creates a rectangle object in widget *w*. The rectangle's upper left corner is at point *x,y* and its extension is given by *width* and *height*.

NAME

XnslDrawCreateRegularPolygon

SYNOPSIS

```
Widget  XnslDrawCreateRegularPolygon(w, sides,  
                                     radius, cx, cy)
```

```
Widget      w;  
Int         sides;  
Float      radius;  
Float      cx;  
Float      cy;
```

DESCRIPTION

This function creates a regular polygon object in widget *w*. The polygon is inscribed in a circle whose radius and center are specified. *sides* is the number of sides of the polygon.

NAME

XnslDrawCreateSegment

SYNOPSIS

```
Widget XnslDrawCreateSegment(w, coordinates,  
                             num_coordinates)  
  
Widget      w;  
Float      *coordinates;  
Int        num_coordinates;
```

DESCRIPTION

This function creates a segment. object in widget *w*. *coordinates* can be a pointer to a FACE Array of float values which are the coordinates of the segment vertices. The number of coordinates you pass must be a multiple of 4: two coordinates per point, an even number of points.

NAME

XnslDrawCreateSpline

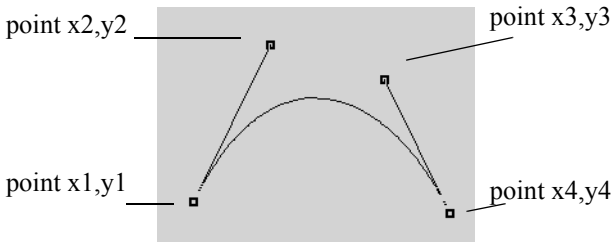
SYNOPSIS

```
Widget  XnslDrawCreateSpline(w, coordinates,  
                               num_coordinates)  
  
Widget      w;  
Float       *coordinates;  
Int         num_coordinates;
```

DESCRIPTION

This creates a spline object in widget *w*. *coordinates* can be a pointer to a FACE Array of float values which are the coordinates of the points that define the spline. The number of points passed must be $3*n + 1$, so that the size of the *coordinates* array, *num_coordinates* must be $2*(3*n + 1)$ as each point requires 2 coordinates, an x and a y value.

Here is how a spline segment is specified:



The last point of a segment (*x4,y4*) is also the first point (*x'1,y'1*) of the next segment, if there is one.

NAME

XnslDrawCreateText

SYNOPSIS

```
Widget    XnslDrawCreateText(w, x, y, width,
                                height)
                                Widget    w;
                                Float     x;
                                Float     y;
                                Float     width;
                                Float     height;
```

DESCRIPTION

This function creates a text object in widget *w*. The text's upper left corner is at point *x,y* and its extension is given by *width* and *height*.

5 Event Functions

The functions in this section should be used to get information related to X events in the widget.

In a FACE script the event (if the script has been launched from an action) is referred to as \$. If the event has no coordinate information or if the event parameter is NULL, these functions perform an XQueryPointer to locate the pointer.

NAME

XnslDrawEventDblX

SYNOPSIS

```
Float XnslDrawEventDblX(w, event)
      Widget      w;
      XEvent      *event;
```

DESCRIPTION

XnslDrawEventDblX returns the x value in the widget's coordinate system corresponding to the x pixel value in the event *event*.

NAME

XnslDrawEventDblY

SYNOPSIS

```
Float XnslDrawEventDblY(w, event)
      Widget      w;
      XEvent      *event;
```

DESCRIPTION

XnslDrawEventDblY returns the x value in the widget's coordinate system corresponding to the x pixel value in the event *event*.

NAME

XnsIDrawEventToAngle

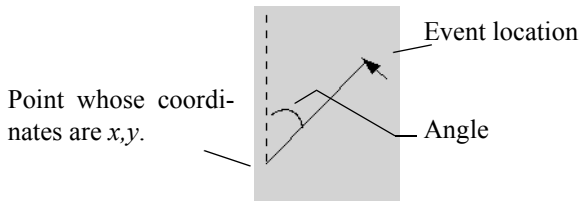
SYNOPSIS

```
Float      XnsIDrawEventToAngle (w, event, x, y)
Widget     w;
XButtonEvent event;
Float      x;
Float      y;
```

DESCRIPTION

XnsIDrawEventToAngle returns the angle formed by the line connecting the event position to the (x, y) coordinates and the upper vertical.

The unit of the resulting angle is given by the resource **XnsIDrawAngleUnits** of widget w . If the value of this resource is **XnsINO_ANGLE_UNITS** the unit is **radian**.



NAME

XnslDrawEventX

SYNOPSIS

```
Int  XnslDrawEventX (w, event)
      Widget          w;
      XEvent          event;
```

DESCRIPTION

XnslDrawEventX returns the x pixel value in the event *event*.

NAME

XnslDrawEventY

SYNOPSIS

```
Int  XnslDrawEventY (w, event)
      Widget          w;
      XEvent          event;
```

DESCRIPTION

XnslDrawEventY returns the y pixel value in the *event*.

NAME

XnslDrawGetKeyEventString

SYNOPSIS

```
String    XnslDrawGetKeyEventString (w, event)
          Widget      w;
          XKeyEvent    event;
```

DESCRIPTION

XnslDrawGetKeyEventString returns the string corresponding to the character typed if *event* is a key event.

NAME

XnslDrawSync

SYNOPSIS

```
None      XnslDrawSync(widget)
           Widget      w;
```

DESCRIPTION

This function synchronizes the display. (It calls XSync(XtDisplay(widget), False).

6 Font Table Functions

The functions in this section are used to work with fonts in the font table of the widget.

NAME

XnslDrawClearFontTable

SYNOPSIS

```
None XnslDrawClearFontTable (w, free)
      Widget                w;
      Int                   free;
```

DESCRIPTION

XnslDrawClearFontTable frees all fonts in the table. If *free* is **True**, it also frees the memory for the table itself. The application should pass **False** for *free* if it expects to load new fonts in the table.

NAME

XnslDrawFontName

SYNOPSIS

```
String    XnslDrawFontName (w, index)
Widget    w;
Int       index;
```

DESCRIPTION

XnslDrawFontName returns the font name of the font located at the given *index* in the internal font table of widget *w*.

NAME

XnslDrawFontStruct

SYNOPSIS

```
Pointer  XnslDrawFontStruct(w, index)
Widget      w;
Int        index;
```

DESCRIPTION

Returns the font structure associated to a font knowing its *index* in the font table of widget *w*.

NAME

XnslDrawGetFont

SYNOPSIS

```
Int  XnslDrawGetFont (w, name)
      Widget          w;
      String          name;
```

DESCRIPTION

XnslDrawGetFont searches for a font with the name *name* in the table, and if it cannot find it, loads a new font in widget *w*'s font table. It returns the index of the font in the table. If it cannot find the font, nor load it, **XnslDrawGetFont** returns -1.

NAME

XnslDrawGetFontIndex

SYNOPSIS

```
Int  XnslDrawGetFontIndex(w, pointer)
Widget  w;
Pointer pointer;
```

DESCRIPTION

Returns the index in widget *w*'s font table, corresponding to the font structure *pointer*.

7 Geometric Transformation Functions

In the geometric transformation functions described in this section, the following rules hold true.

- Angles are measured in the object unit (radian, degree, revolution).
- Distances are measured in the object unit.
- Coordinates are measured in the object unit.
- Scales are defined by dimensionless coefficients.

The “object unit” is the unit of measurement chosen for that object.

Object representation is divided in two parts: the set of definition points and the set of visible points. The visible points are computed using the geometric transformation fields of the object.

The transformations that can be applied to an object are:

- Center definition
- Rotation
- Scale
- Translation

These fields are used to build a matrix which is applied to the set of definition points to construct the visible representation of the object. Note that rotation and scale are relative to the Center of the object.

7.1 Center

The functions in this section are used to define or retrieve the coordinates of an object’s center.

NAME

XnsIDrawDefineCenter

SYNOPSIS

```
None      XnsIDrawDefineCenter (w, o, x, y)
Widget    w;
Widget    o;
Float     x;
Float     y;
```

DESCRIPTION

XnsIDrawDefineCenter defines the center of the given widget *o* to be at the point (*x*, *y*) in widget *w*'s coordinate system.

The unit for *x* and *y* is given by the resource **XnsIDrawUnits** of widget *o*. If the value of this resource is **XnsINO_UNITS** and the class of widget *o*'s parent is **XnsIDraw**, then unit is taken from the resource **XnsIDrawUnits** of widget *o*'s parent. If the value is also **XnsINO_UNITS**, the unit is **pixels**.

NAME

XnslDrawGetCenterX

SYNOPSIS

```
Float      XnslDrawGetCenterX (w, o)
           Widget      w;
           Widget      o;
```

DESCRIPTION

XnslDrawGetCenterX returns the x-coordinate of the widget's center.

The unit for x-coordinate is given by the resource **XnslNdrawUnits** of the widget *o*. If the value of this resource is **XnslNO_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawUnits** of widget *o*'s parent. If the value is also **XnslNO_UNITS**, the unit is **pixels**.

NAME

XnsIDrawGetCenterY

SYNOPSIS

```
Float      XnsIDrawGetCenterY (w, o)
Widget      w;
Widget      o;
```

DESCRIPTION

XnsIDrawGetCenterY returns the y-coordinate of the widget's center.

The unit for the y-coordinate is given by the resource **XnsINdrawUnits** of object *o*. If the value of this resource is **XnsINO_UNITS** and the class of widget *o*'s parent is **XnsIDraw**, then unit is taken from the resource **XnsINdrawUnits** of widget *o*'s parent. If the value is also **XnsINO_UNITS**, the unit is **pixels**.

NAME

XnslDrawUndefineCenter

SYNOPSIS

```
None      XnslDrawUndefineCenter (w, o)
          Widget      w;
          Widget      o;
```

DESCRIPTION

XnslDrawUndefineCenter resets the center of widget *o* to its default value, i.e. the geometric center of the object.

7.2 Miscellaneous Geometric Functions

The functions in this section are used to move objects to a new position.

NAME

XnslDrawCircTranslateObject

SYNOPSIS

```
None XnslDrawCircTranslateObject (w, o, cx, cy,  
                                   angle)
```

```
Widget      w;  
Widget      o;  
Float       cx;  
Float       cy;  
Float       angle;
```

DESCRIPTION

XnslDrawCircTranslateObject computes parameters for moving the widget along the arc of a circle whose center is at (cx, cy) and whose length is determined by the given *angle*. It performs this translation on the widget in its current state, taking into account its current translation and rotation.

This computes a translation given a reference point and an angle:

old center of object: (ocx, ocy)

new center of object : (ncx, ncy)

$$ncx = cx + \cos(angle) * (ocx - cx) - \sin(angle) * (ocy - cy)$$
$$ncy = cy + \sin(angle) * (ocx - cx) - \cos(angle) * (ocy - cy)$$

The translation is then:

$$tx = ncx - ocx$$
$$ty = ncy - ocy$$

NAME

XnslDrawMoveObject

SYNOPSIS

```
None XnslDrawMoveObject (w, o, dx, dy)
      Widget             w;
      Widget             o;
      Float              dx;
      Float              dy;
```

DESCRIPTION

XnslDrawMoveObject moves widget *o* by moving its center by the given distances in the x and y directions. The unit for *dx* and *dy* is given by the resource **XnslNdrawUnits** of widget *o*. If the value of this resource is **XnslNO_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawUnits** of widget *o*'s parent. If the value is also **XnslNO_UNITS**, the unit is **pixels**.

NAME

XnslDrawMoveObjectTo

SYNOPSIS

```
None XnslDrawMoveObjectTo (w, o, x, y)
      Widget                w;
      Widget                o;
      Int                   x;
      Int                   y;
```

DESCRIPTION

XnslDrawMoveObjectTo moves the center of widget *o* directly to the point at (*x*, *y*). The unit for *x* and *y* is given by the resource **XnslNdrawUnits** of the widget *o*. If the value of this resource is **XnslNO_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawUnits** of widget *o*'s parent. If the value is also **XnslNO_UNITS**, the unit is **pixels**.

7.3 Rotation Functions

The functions in this section control object rotation.

NAME

XnslDrawGetRotation

SYNOPSIS

```
Float      XnslDrawGetRotation (w, o)
Widget     w;
Widget     o;
```

DESCRIPTION

XnslDrawGetRotation returns the current (cumulative) rotation angle of widget *o*. The unit of the resulting angle is given by the resource **XnslNdrawAngleUnits** of widget *o*. If the value of this resource is **XnslNO_ANGLE_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawAngleUnits** of widget *o*'s parent. If the value is also **XnslNO_ANGLE_UNITS**, the unit is **radian**.

NAME

XnslDrawRotateObject

SYNOPSIS

```
None      XnslDrawRotateObject (w, o, angle)
Widget    w;
Widget    o;
Float     angle;
```

DESCRIPTION

XnslDrawRotateObject rotates widget *o* by the given *angle* relative to its current rotation. The unit of *angle* is given by the resource **XnslNdrawAngleUnits** of widget *o*. If the value of this resource is **XnslNO_ANGLE_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawAngleUnits** of widget *o*'s parent. If the value is also **XnslNO_ANGLE_UNITS**, the unit is **radian**.

NAME

XnslDrawSetRotation

SYNOPSIS

```
None      XnslDrawSetRotation (w, o, angle)
          Widget          w;
          Widget          o;
          Float           angle;
```

DESCRIPTION

XnslDrawSetRotation sets the rotation of widget *o* to the given *angle*.

The unit for *angle* is given by the resource **XnslNdrawAngleUnits** of widget *o*. If the value of this resource is **XnslNO_ANGLE_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawAngleUnits** of widget *o*'s parent. If the value is also **XnslNO_ANGLE_UNITS**, the unit is **radian**.

7.4 Scale functions

The functions in this section control object scaling.

Note: The Draw library provides two ways to scale objects: *prescale* and *postscale*. The difference between the two can be seen when an object is rotated. Prescale is applied *before* the rotation and postscale is applied *after* the rotation.

NAME

XnslDrawGetPostScaleX

SYNOPSIS

```
Float      XnslDrawGetPostScaleX (w, o)
Widget      w;
Widget      o;
```

DESCRIPTION

XnslDrawGetPostScaleX returns the current (cumulative) post scaling factor of widget *o* in the x direction.

NAME

XnslDrawGetPostScaleY

SYNOPSIS

```
Float      XnslDrawGetPostScaleY (w, o)
Widget      w;
Widget      o;
```

DESCRIPTION

XnslDrawGetPostScaleY returns the current (cumulative) post scaling factor of widget *o* for the y direction.

NAME

XnslDrawGetPreScaleX

SYNOPSIS

```
Float      XnslDrawGetPreScaleX (w, o)
Widget      w;
Widget      o;
```

DESCRIPTION

XnslDrawGetPreScaleX returns the current (cumulative) factor for pre scaling widget *o* in the x direction.

NAME

XnslDrawGetPreScaleY

SYNOPSIS

```
Float      XnslDrawGetPreScaleY (w, o)
Widget      w;
Widget      o;
```

DESCRIPTION

XnslDrawGetPreScaleY returns the current (cumulative) factor for pre scaling widget *o* in the y direction.

NAME

XnslDrawGetScaleX

SYNOPSIS

```
Float      XnslDrawGetScaleX (w, o)
Widget      w;
Widget      o;
```

DESCRIPTION

This function reads and returns the XScale of an object. This function is provided for backward compatibility, and reads the prescale. It is equivalent to the function **XnslDrawGetPreScaleX** which should be used instead.

NAME

XnslDrawGetScaleY

SYNOPSIS

```
Float XnslDrawGetScaleY (w, o)
    Widget      w;
    Widget      o;
```

DESCRIPTION

This function reads and returns the YScale of an object. This function is provided for backward compatibility, and reads the prescale. It is equivalent to the function **XnslDrawGetPreScaleY** which should be used instead.

NAME

XnslDrawPostScaleObject

SYNOPSIS

```
None      XnslDrawPostScaleObject (w, o, x, y)
Widget    w;
Widget    o;
Float     x;
Float     y;
```

DESCRIPTION

XnslDrawPostScaleObject post scales widget *o* by the given values in the x and y directions, in addition to any scaling already done to the widget.

NAME

XnslDrawPreScaleObject

SYNOPSIS

```
None      XnslDrawPreScaleObject (w, o, x, y)
          Widget      w;
          Widget      o;
          Float       x;
          Float       y;
```

DESCRIPTION

This function accesses the prescale parameters for the given widget *o*. **XnslDrawPreScaleObject** scales the widget by the given values: *x* in the x direction and *y* in the y direction, in addition to any scaling already done to the widget.

NAME

XnslDrawScaleObject

SYNOPSIS

```
None XnslDrawScaleObject (w, o, x, y)
      Widget              w;
      Widget              o;
      Float               x;
      Float               y;
```

DESCRIPTION

XnslDrawScaleObject scales widget *o* by the given values in the *x* and *y* directions, in addition to any scaling already done to the widget.

This function is provided for backward compatibility, and sets the prescale. It is equivalent to the function **XnslDrawPreScaleObject** which should be used instead.

NAME

XnslDrawSetPostScale

SYNOPSIS

```
None      XnslDrawSetPostScale (w, o, x, y)
Widget    w;
Widget    o;
Float     x;
Float     y;
```

DESCRIPTION

XnslDrawSetPostScale sets the post scale factor of widget *o* to *x*, *y*.

NAME

XnslDrawSetPreScale

SYNOPSIS

```
None      XnslDrawSetPreScale (w, o, x, y)
Widget    w;
Widget    o;
Float     x;
Float     y;
```

DESCRIPTION

XnslDrawSetPreScale sets the pre scale factor of widget *o* to *x*, *y*.

NAME

XnslDrawSetScale

SYNOPSIS

```
None XnslDrawSetScale (w, o, x, y)
      Widget           w;
      Widget           o;
      Float            x;
      Float            y;
```

DESCRIPTION

This function affects the Scale of an object. This function is provided for backward compatibility, and sets the prescale. It is replaced by the function XnslDrawSetPreScale which should be used instead.

7.5 Translation Functions

The functions in this section control the translation of objects.

NAME

XnslDrawGetTranslationX

SYNOPSIS

```
Float XnslDrawGetTranslationX (w, o)
    Widget      w;
    Widget      o;
```

DESCRIPTION

XnslDrawGetTranslationX returns the x-value of the current translation of object *o* in widget *w*. The unit for the result is given by the resource **XnslNdrawUnits** of widget *o*. If the value of this resource is **XnslNO_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawUnits** of widget *o*'s parent. If the value is also **XnslNO_UNITS**, the unit is **pixels**.

NAME

XnslDrawGetTranslationY

SYNOPSIS

```
Float XnslDrawGetTranslationY (w, o)
    Widget      w;
    Widget      o;
```

DESCRIPTION

XnslDrawGetTranslationY returns the y-value of the current translation of object *o* in widget *w*. The unit for the result is given by the resource **XnslNdrawUnits** of the widget *o*. If the value of this resource is **XnslNO_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawUnits** of widget *o*'s parent. If the value is also **XnslNO_UNITS**, the unit is **pixels**.

NAME

XnslDrawSetTranslation

SYNOPSIS

```
None XnslDrawSetTranslation (w, o, x, y)
      Widget          w;
      Widget          o;
      Float           x;
      Float           y;
```

DESCRIPTION

XnslDrawSetTranslation sets the translation parameters of widget *o* to the given *x*- and *y*-values. The unit for *x* and *y* is given by the resource **XnslNdrawUnits** of the widget *o*. If the value of this resource is **XnslNO_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawUnits** of widget *o*'s parent. If the value is also **XnslNO_UNITS**, the unit is **pixels**.

NAME

XnslDrawTranslateObject

SYNOPSIS

```
None      XnslDrawTranslateObject (w, o, x, y)
          Widget      w;
          Widget      o;
          Float       x;
          Float       y;
```

DESCRIPTION

XnslDrawTranslateObject translates the object by the given x- and y-values relative to its current translation. The unit for *x* and *y* is given by the resource **XnslNdrawUnits** of widget *o*. If the value of this resource is **XnslNO_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawUnits** of widget *o*'s parent. If the value is also **XnslNO_UNITS**, the unit is **pixels**.

8 Graphic Transformation Functions

The functions in this section concern bitmap, pixmap, font, and color management.

Note: In the Draw library, a set of tables is provided to store Pixmaps, bitmaps, colors, and fonts. These resources can be accessed through an integer which is an index in the corresponding table. An entry in the table is allocated by:

XnslDrawGetColor

XnslDrawGetFont

XnslDrawGetPixmap

XnslDrawGetBitmap

In each case, the name of the resource is passed and an index is returned, which can be used to set an object's resource. For example, to change the foreground of an object:

```
i = XnslDrawGetColor (parent, "red");
if (i == -1) /*color cannot be allocated */
    error();
else
    self : drawForeground = i;
```

NAME

XnslDrawGetBackground

SYNOPSIS

```
Int      XnslDrawGetBackground (w, o)
Widget   w;
Widget   o;
```

DESCRIPTION

This function returns the index corresponding to the background color of object *o* in the color table of widget *w*.

NAME

XnsldrawGetBackgroundPixel

SYNOPSIS

```
Pixel XnsldrawGetBackgroundPixel (w, o)
    Widget      w;
    Widget      o;
```

DESCRIPTION

This function returns the pixel corresponding to the background color of object *o* in widget *w*.

NAME

XnslDrawGetBitmap

SYNOPSIS

```
Int  XnslDrawGetBitmap (w, name)
      Widget             w;
      String             name;
```

DESCRIPTION

XnslDrawGetBitmap searches for the bitmap with the given path *name* in the internal bitmap table. If it finds the bitmap, it returns its index. If not, it tries to load the bitmap. If the process's environment has the variable **DRAWPIXMAPSPATH** set, it uses this path to determine where to search for the bitmap, otherwise, it tries the variable **XBMLANGPATH**.

Once **XnslDrawGetBitmap** has loaded the bitmap, it places it in the table and returns the new index. If it cannot load the bitmap, it returns -1.

NAME

XnslDrawGetForeground

SYNOPSIS

```
Int      XnslDrawGetForeground (w, o)
Widget   w;
Widget   o;
```

DESCRIPTION

This function returns the index of the Foreground color of object *o* in the color table of widget *w*.

NAME

XnsIDrawGetForegroundPixel

SYNOPSIS

```
Pixel XnsIDrawGetForegroundPixel (w, o)
    Widget      w;
    Widget      o;
```

DESCRIPTION

This function returns the pixel corresponding to the Foreground color of object *o*.

NAME

XnslDrawGetLineWidth

SYNOPSIS

```
Int XnslDrawGetLineWidth (w, o)
    Widget      w;
    Widget      o;
```

DESCRIPTION

XnslDrawGetLineWidth returns the line width of widget *o*.

NAME

XnslDrawSetBackground

SYNOPSIS

```
None      XnslDrawSetBackground (w, o, color_name)
Widget    w;
Widget    o;
String    color_name;
```

DESCRIPTION

XnslDrawSetBackground converts the color specified in *color_name* to a pixel value and sets the background color of object *o* to the resulting value.

NAME

XnslDrawSetBackgroundIndex

SYNOPSIS

```
None      XnslDrawSetBackgroundIndex (w, o, index)
Widget    w;
Widget    o;
DrawPixel index;
```

DESCRIPTION

XnslDrawSetBackgroundIndex sets the background color of object *o* to the given index value *index*. The *index* is a value returned by the **XnslDrawGetColor** function. It is the index of the color in the widget's (*w*) color table.

NAME

XnslDrawSetBackgroundPixel

SYNOPSIS

```
None XnslDrawSetBackgroundPixel (w, o, pixel)
      Widget          w;
      Widget          o;
      Pixel           pixel;
```

DESCRIPTION

XnslDrawSetBackgroundPixel sets the background color of widget *o* to the given pixel value *pixel*.

NAME

XnslDrawSetCloseMode

SYNOPSIS

```
None      XnslDrawSetCloseMode (w, o, mode)
Widget    w;
Widget    o;
Int       mode;
```

DESCRIPTION

XnslDrawSetCloseMode sets the close mode of the given widget *o* to the given mode *mode*. *mode* can be one of the following:

XnslDRAW_OPEN Do not close the widget.

XnslDRAW_CENTER_CLOSED Close through the widget's center
(Close the widget by joining its two end points to its center).

XnslDRAW_LINE_CLOSED Close the object by joining its two end
points.

XnslDRAW_OBJECT_CLOSED Close mode depends on the widget's
type.

This function works only for the following widget class:

- XnslDrawArc
- XnslDrawEllipse
- XnslDrawPath
- XnslDrawPolygon
- XnslDrawSpline

NAME

XnslDrawSetDashes

SYNOPSIS

```
None      XnslDrawSetDashes (w, o, dashes)
Widget    w;
Widget    o;
String     dashes;
```

DESCRIPTION

XnslDrawSetDashes sets the dashes of widget *o* to the values in the *dashes* string. The bytes have the same meaning as for an X graphics context. **XnslDrawSetDashes** only looks at the first nonzero bytes. If *dashes* is **NULL** or contains an empty array, **XnslDrawSetDashes** resets the widget's dashes attribute to its default value.

EXAMPLE

```
dashes = malloc(3);

sprintf(dashes, "%c%c", 2, 3);

XnslDrawSetDashes (parent, self, dashes);

free(dashes);
```

This example shows how to set a dash with 2 pixels on followed by 3 pixels off for a widget.

NAME

XnslDrawSetFillMode

SYNOPSIS

```
None      XnslDrawSetFillMode (w, o, mode)
Widget    w;
Widget    o;
Int       mode;
```

DESCRIPTION

XnslDrawSetFillMode fills widget *o* according to the given *mode*.

Values for *mode* are:

XnslDRAW_NON_FILLED

The widget is not filled.

XnslDRAW_FILLED

The widget is filled completely.

XnslDRAW_PCT_FILLED

The widget is filled according to the value of its XnslNdrawFillDir, and XnslNdrawPctFill resources.

NAME

XnslDrawSetForeground

SYNOPSIS

```
None      XnslDrawSetForeground (w, o, name)
Widget    w;
Widget    o;
String    name;
```

DESCRIPTION

XnslDrawSetForeground converts the color named in *name* to a pixel value and sets the foreground color of widget *o* to the resulting value.

NAME

XnslDrawSetForegroundIndex

SYNOPSIS

```
None XnslDrawSetForegroundIndex (w, o, index)
Widget      w;
Widget      o;
DrawPixel   index;
```

DESCRIPTION

XnslDrawSetForegroundIndex sets the foreground color of widget *o* to the given index value *index*. The *index* is a value returned by the **XnslDrawGetColor** function. It is an index in the widget's (*w*) color table.

NAME

XnslDrawSetForegroundPixel

SYNOPSIS

```
None      XnslDrawSetForegroundPixel (w, o, pixel)
Widget    w;
Widget    o;
Pixel     pixel;
```

DESCRIPTION

XnslDrawSetForegroundPixel sets the foreground color of widget *o* to the given pixel value *pixel*.

NAME

XnslDrawSetLineWidth

SYNOPSIS

```
None      XnslDrawSetLineWidth (w, o, width)
Widget    w;
Widget    o;
Int       width;
```

DESCRIPTION

XnslDrawSetLineWidth sets the line width of widget *o* to the given *width*. If *width* is zero or negative, **XnslDrawSetLineWidth** sets the widget's line width to its default value. Only objects that have lines are affected. Line widths are expressed in pixels.

NAME

XnslDrawSetPctFill

SYNOPSIS

```
None XnslDrawSetPctFill (w, o, percent, direction)
                        Widget      w;
                        Widget      o;
                        Int         percent;
                        Int         direction;
```

DESCRIPTION

XnslDrawSetPctFill provides the additional parameters needed if the fill mode of widget *o* is **XnslDRAW_PCT_FILLED**. *percent* indicates how much of the widget to fill, and *direction* specifies in what direction to fill it. *percent* must be between 0 and 100. Possible values for direction are:

- **XnslDRAW_FILL_BOTTOM**
- **XnslDRAW_FILL_TOP**
- **XnslDRAW_FILL_LEFT**
- **XnslDRAW_FILL_RIGHT**

NAME

XnslDrawSetStipple

SYNOPSIS

```
None      XnslDrawSetStipple (w, o, name)
Widget    w;
Widget    o;
String    name;
```

DESCRIPTION

XnslDrawSetStipple sets the stipple of widget *o* to the pixmap in the file named *name*. It uses the environment variable **XBMLANGPATH** to find the file if the full path is not provided.

NAME

XnslDrawSetStippleIndex

SYNOPSIS

```
None      XnslDrawSetStippleIndex (w, o, index)
Widget    w;
Widget    o;
String    index;
```

DESCRIPTION

XnslDrawSetStipple sets the stipple of widget *o* to the given index value *index*. The *index* is a value returned by the **XnslDrawGetBitmap** function..

NAME

XnsldrawSetStipplePixmap

SYNOPSIS

```
None      XnsldrawSetStipplePixmap (w, o, stipple)
Widget    w;
Widget    o;
Pixmap    stipple;
```

DESCRIPTION

XnsldrawSetStipplePixmap sets the pixmap stipple of widget *o* to *stipple*. The pixmap must have a depth of one.

NAME

XnslDrawSetTSOrigin

SYNOPSIS

```
None      XnslDrawSetTSOrigin (w, o, x, y)
          Widget          w;
          Widget          o;
          Int             x;
          Int             y;
```

DESCRIPTION

XnslDrawSetTSOrigin sets the stipple origin of widget *o* to the pixel at position (*x*, *y*). The unit for *x* and *y* is pixel.

9 Input/Output Functions

The functions in this section are available for saving and loading draw files.

NAME

XnslDrawChangeFile

SYNOPSIS

```
None      XnslDrawChangeFile (w, name)
          Widget      w;
          String      name;
```

DESCRIPTION

XnslDrawChangeFile deletes all the widgets contained in the XnslDraw widget, reads the widget description in the file with the given *name* and adds the widgets described to the XnslDraw widget *w*.

NAME

XnslDrawLoadFile

SYNOPSIS

```
None      XnslDrawLoadFile(w, file)  
Widget    w;  
String    file;
```

DESCRIPTION

This function loads the objects of the file *'file'* into an XnslDraw widget *w*.

It first deletes all the XnslDrawObject widgets contained in the XnslDraw widget *w*, reads the widgets of the file *file*, into *w* and resizes the widgets according to the value of the **XnslNdrawResizePolicy** resource. Use the functions **XnslDrawReadFile** or **XnslDrawChangeFile** rather than this one which is included only for backward compatibility.

NAME

XnslDrawReadFile

SYNOPSIS

```
None      XnslDrawReadFile (w, name)
Widget    w;
String    name;
```

DESCRIPTION

XnslDrawReadFile reads the widget description in the file with the given *name* and adds the widgets described to the XnslDraw widget *w*.

NAME

XnslDrawSaveWidget

SYNOPSIS

```
Int      XnslDrawSaveWidget (w, filename)
Widget   w;
String   filename;
```

DESCRIPTION

XnslDrawSaveWidget saves XnslDraw widget *w* in file *filename* according to the value of the resource **XnslNdrawOutputFormat**. This resource can have the following values.

XnslDRAW_DRAW

Saves the contents in the draw format.

XnslDRAW_FM

Saves the contents in the **XFaceMaker** format.

XnslDRAW_PS

Saves the contents as a **PostScript** program.

10 Miscellaneous Functions

The functions in this section are available to control redisplay, synchronizations, to evaluate mathematical functions, to convert widget contents to/from String representation, etc... .

NAME

XnslDrawComplete

SYNOPSIS

None XnslDrawComplete ()

DESCRIPTION

The function **XnslDrawComplete** terminates the use of the Draw library. If the library is protected by a license server (development version), calling this termination function releases the license token. Application programs may call this function safely when using the unprotected version of the library (run-time library).

NAME

XnslDrawCos

SYNOPSIS

```
Float    XnslDrawCos(a)
          Float    a;
```

DESCRIPTION

This function returns the cosine of angle a specified in radians.

NAME

XnslDrawDegToRad

SYNOPSIS

```
Float    XnslDrawDegToRad (a)
Float    a;
```

DESCRIPTION

XnslDrawDegToRad converts angle a from radians to degrees.

NAME

XnslDrawFlush

SYNOPSIS

```
None      XnslDrawFlush(w)
          Widget      w;
```

DESCRIPTION

XnslDrawFlush flushes the display event queue. It simply calls `XFlush(XtDisplay(w))`

NAME

XnslDrawGetCoordFromDrawPoint

SYNOPSIS

```
Float XnslDrawGetCoordFromDrawPoint
                                   (p, num, get_what)
XnslDrawPoint *p;
int num;
int get_what;
```

DESCRIPTION

XnslDrawGetCoordFromDrawPoint retrieves the *x* or *y* coordinates from an array of *XnslDrawPoints*. *p* specifies the *XnslDrawPoint* array. *num* specifies the number of the point in the list from which the coordinates are to be extracted. *get_what* specifies which coordinate to return. Passing 0 retrieves the *x* coordinate of the point. Passing 1 retrieves the *y* coordinate.

NAME

XnsldrawGetNthChild

SYNOPSIS

```
Widget XnsldrawGetNthChild (w, i)
    Widget      w;
    Int         i;
```

DESCRIPTION

XnsldrawGetNthChild retrieves the *n*th child of a composite widget *w*. NULL is returned if *i* is less than 0 or if *i* is greater than the number of children of widget *w*.

NAME

XnslDrawRadToDeg

SYNOPSIS

```
Float    XnslDrawRadToDeg (a)
          Float            a;
```

DESCRIPTION

XnslDrawRadToDeg converts angle a from radians to degrees.

NAME

XnslDrawRandom

SYNOPSIS

```
Int  XnslDrawRandom (n)
      Int             n;
```

DESCRIPTION

XnslDrawRandom returns a pseudorandom integer between 0 and n .

NAME

XnslDrawRedisplay

SYNOPSIS

```
None      XnslDrawRedisplay (w)
           Widget           w;
```

DESCRIPTION

XnslDrawRedisplay redraws the portions of the **XnslDraw** widget *w* that have changed since the last update.

NAME

XnslDrawRedisplayAll

SYNOPSIS

```
None      XnslDrawRedisplayAll (w)
          Widget      w;
```

DESCRIPTION

XnslDrawRedisplayAll redraws the entire widget, regardless of whether or not the display is up to date.

NAME

XnslDrawRefreshControl

SYNOPSIS

```
None      XnslDrawRefreshControl (w, mode)
Widget    w;
Int       mode;
```

DESCRIPTION

XnslDrawRefreshControl enables or disables the refresh operation undertaken by the Draw widget.

XnslDRAW_DISABLE	The widget stops calculating regions to be refreshed.
XnslDRAW_ENABLE	The widget calculates the regions to be refreshed.

NAME

XnslDrawSin

SYNOPSIS

```
Float    XnslDrawSin(a)
          Float    a;
```

DESCRIPTION

XnslDrawSin returns the sine of angle a specified in radians.

NAME

XnslDrawSleep

SYNOPSIS

```
None      XnslDrawSleep (n)
          Int          n;
```

DESCRIPTION

XnslDrawSleep sleeps for *n* microseconds. The actual time slept depends on the accuracy of the system clock. It uses the select system call to sleep.

NAME

XnslDrawStringToWidget

SYNOPSIS

```
Widget    XnslDrawStringToWidget(w, s, suff, managed)
          Widget        w;
          String        s;
          String        suff;
          Int           managed;
```

DESCRIPTION

XnslDrawStringToWidget converts string *s* to the corresponding widget. The widget is created with widget *w* as parent. The suffix *suff* is added to the name of the original object to create the name of the new object. This avoids automatic name generation which can be slow because of the need to check for duplicate names. An application using this function must provide a suffix which generates non-duplicate names, because no verification is performed when this function is used. *managed* indicates if the widget being created is to be managed.

NAME

XnslDrawWidgetToString

SYNOPSIS

```
String    XnslDrawWidgetToString (w)
Widget                                         w;
```

DESCRIPTION

XnslDrawWidgetToString outputs the string description of widget *w* according to the value of the resource **XnslNdrawOutputFormat**.

This resource can have the following values.

XnslDRAW_DRAW

Saves the contents in the draw format.

XnslDRAW_FM

Saves the contents in the **XFaceMaker** format.

XnslDRAW_PS

Saves the contents as a **PostScript** program.

The returned string must not be freed or modified.

11 General Object Management Functions

The functions listed here are used in various object management operations, such as retrieving an object's name, duplicating or deleting objects, getting/setting object points, etc.

NAME

XnslDrawAdjustObjects

SYNOPSIS

```
None      XnslDrawAdjustObjects (w)
Widget    w;
```

DESCRIPTION

XnslDrawAdjustObjects tells the XnslDraw widget *w* to look at its size and adjust the size of all of its resizable children accordingly. The size of the widget is the size of the window minus the margins defined for the widget.

This function is called when the widget's XnslNdrawResize Policy resource is either `Xnsl_DRAW_FULL` or `Xnsl_DRAW_UNIFORM` and when the widget is in **autoResize** mode and the size of the widget changes. This scales and translates the objects so that they fit within the margins defined for the widget. See the XnslDrawWidget documentation for more details

NAME

XnsIDrawAlignObjectToGrid

SYNOPSIS

```
None XnsIDrawAlignObjectToGrid (w, o, i)
      Widget          w;
      Widget          o;
      Int             i;
```

DESCRIPTION

XnsIDrawAlignObjectToGrid aligns a widget on the grid defined by an XnsIDraw widget *w*. The *i* parameter specifies which part of the object is to be aligned on the grid. Possible values are:

XnsIDRAW_TOP_LEFT

XnsIDRAW_TOP

XnsIDRAW_TOP_RIGHT

XnsIDRAW_RIGHT

XnsIDRAW_BOTTOM_RIGHT

XnsIDRAW_BOTTOM

XnsIDRAW_BOTTOM_LEFT

XnsIDRAW_LEFT

XnsIDRAW_CENTER

NAME

XnslDrawDeleteAllChildren

SYNOPSIS

```
None XnslDrawDeleteAllChildren (w)
      Widget      w;
```

DESCRIPTION

XnslDrawDeleteAllChildren destroys all the children contained in widget *w*.

NAME

XnslDrawDeleteAllObjects

SYNOPSIS

```
None      XnslDrawDeleteAllObjects (w)
          Widget      w;
```

DESCRIPTION

XnslDrawDeleteAllObjects destroys all the **XnslDrawObject** widgets contained in the **XnslDraw** widget *w*. All associated data is freed.

NAME

XnslDrawDuplicateObject

SYNOPSIS

```
Widget    XnslDrawDuplicateObject (w, o)
           Widget                    w;
           Widget                    o;
```

DESCRIPTION

XnslDrawDuplicateObject returns a copy of the **XnslDrawObject** *o*. All attributes are duplicated.

NAME

XnslDrawDupObject

SYNOPSIS

```
Widget XnslDrawDupObject (w, o, suff)
        Widget          w;
        Widget          o;
        String          suff;
```

DESCRIPTION

XnslDrawDupObject duplicates the widget *o*. The suffix *suff* is added to the name of the original object to create the name of the new object. This avoids automatic name generation which can be slow because of the need to check for duplicate names. An application using this function must provide a suffix which generates non-duplicate names, because no check is performed when this function is used.

NAME

XnslDrawFaceGetObjectPoints

SYNOPSIS

```
FaceArray XnslDrawFaceGetObjectPoints(w, o)
      Widget      w;
      Widget      o;
```

DESCRIPTION

This function returns a FACE array containing the coordinates of the object's definition points. The points are in the format used to define the object (e.g. for a rectangle, two points are returned. The first point is the coordinate of the top left corner of the rectangle. The second point contains the width and height of the rectangle)

EXAMPLE

```
/* list the definition points of an object*/
Float a[Int];
a = XnslDrawFaceGetObjectPoints(parent, self);
for(i = 0; i < size(a); i = i+2) {
    printf ("point %g %g\n", a[i], a[i + 1]);
}
destroy(a);
```

NAME

XnslDrawFaceGetObjectXPoints

SYNOPSIS

```
FaceArray      XnslDrawFaceGetObjectXPoints(w, o)
                Widget      w;
                Widget      o;
```

DESCRIPTION

This function returns a FACE array containing the coordinates of the object's visualization points. The points are a list of x,y coordinates in pixel units (e.g. for a rectangle, four points are returned, which contain the coordinates of the four corners).

EXAMPLE

```
/* list the display points of an object*/
Int a[Int];
a = XnslDrawFaceGetObjectXPoints(parent, self);
for(i = 0; i <size(a); i = i + 2) {
    printf("point %d %d\n", a[i], a[i + 1]);
}
destroy(a);
```

NAME

XnslDrawFaceSetObjectPoints

SYNOPSIS

```
FaceArray XnslDrawFaceSetObjectPoints(w, o, coord,
n_coords)
```

Widget	<i>w</i> ;
Widget	<i>o</i> ;
Pointer	<i>coord</i> ;
Int	<i>n_coords</i> ;

DESCRIPTION

This function uses the coordinates *coord* to build a list of points which is used to change the object's definition points.

EXAMPLE

```
/* list the definition points of an object*/
Float a[Int];
a = XnslDrawFaceGetObjectPoints(parent, self);
for(i = 0; i < size(a); i = i+2) {
    a[i] = NewValue();
}
XnslDrawFaceSetObjectPoints(parent, self, data(a),
                             size(a));
destroy(a);
```

NAME

XnsldrawFlipObject

SYNOPSIS

```
None XnsldrawFlipObject(w, obj, dir)
      Widget              w;
      Widget              obj;
      XnsldrawFlipDir     dir;
```

DESCRIPTION

XnsldrawFlipObject flips object *obj* according to the value of *dir*. Acceptable values for *dir* are:

Xnsldraw_FLIP_HORIZONTAL

mirror image the object with respect to the horizontal direction.

Xnsldraw_FLIP_VERTICAL

mirror image the object with respect to the vertical direction.

NAME

XnslDrawGetObjectName

SYNOPSIS

```
String    XnslDrawGetObjectName (w, o)
Widget    w;
Widget    o;
```

DESCRIPTION

XnslDrawGetObjectName returns the name¹ of the given widget *o*. **XnslDrawGetObjectName** returns a copy of the name in dynamically allocated data. The application is responsible for freeing this memory when it no longer needs it.

1. Since the Xt toolkit does not allow you to change the name of a widget, all XnslDrawObjects have an XnslNdrawName resource which can be changed at any time. At initialization time, the Xt name, (which can be retrieved by the XtName() function) and the XnslNdrawName are identical.

If you change the name of an XnslDrawObject by modifying the XnslNdrawName resource, the return value from the XtName() function and the XnslNdrawName resource have different values.

The following FACE statement will then fail because the FACE naming scheme uses XtName to locate widgets:

```
self : drawName = "NewName";
parent . NewName : drawForeground = red;
```

But this one is OK :

```
self : drawName = "NewName";
w = XnslDrawNameToOobject(parent, "NewName");
w : drawForeground = red;
```

In all the object name functions, we consider that the name of an object is the XnslNdrawName resource.

NAME

XnslDrawGetUserData

SYNOPSIS

```
Pointer   XnslDrawGetUserData(w, o)
          Widget           w;
          Widget           o;
```

DESCRIPTION

This function retrieves user data previously specified for an object by the XnslDrawSetUserData function. If no data is present in the object, NULL is returned.

NAME

XnslDrawHideObject

SYNOPSIS

```
None      XnslDrawHideObject (w, o)
Widget    w;
Widget    o;
```

DESCRIPTION

XnslDrawHideWidget makes widget *o* invisible. Note that **XtUnmanageChild** gives the same results.

NAME

XnslDrawNameToObject

SYNOPSIS

```
Widget      XnslDrawNameToObject (w, name)
            Widget      w;
            String      name;
```

DESCRIPTION

XnslDrawNameToObject returns the first widget with the given *name*² in *w*'s widget list. A child widget can be accessed by asking for its *name* in the form *#<child number>*. For example:

```
XnslDrawNameToObject (w, "#4")
returns the fourth child of the Draw widget.
```

2. Since the Xt toolkit does not allow you to change the name of a widget, all XnslDrawObjects have an XnslNdrawName resource which can be changed at any time. At initialization time, the Xt name, (which can be retrieved by the XtName() function) and the XnslNdrawName are identical.

If you change the name of an XnslDrawObject by modifying the XnslNdrawName resource, the return value from the XtName() function and the XnslNdrawName resource have different values.

The following FACE statement will then fail because the FACE naming scheme uses XtName to locate widgets:

```
self : drawName = "NewName";
parent . NewName : drawForeground = red;
```

But this one is OK :

```
self : drawName = "NewName";
w = XnslDrawNameToObject(parent, "NewName");
w : drawForeground = red;
```

NAME

XnslDrawObjectVisible

SYNOPSIS

```
Boolean   XnslDrawObjectVisible (w, o)
Widget    w;
Widget    o;
```

DESCRIPTION

This function returns the visibility status of object *o*. Returns 1 if visible, 0 if not. This function calls `XtIsManaged()` which can be used instead.

NAME

XnslDrawParent

SYNOPSIS

```
Widget    XnslDrawParent (o)
           Widget          o;
```

DESCRIPTION

XnslDrawParent returns the group to which widget *o* belongs. If the widget is not part of a group, **NULL** is returned.

NAME

XnslDrawRefNameToObject

SYNOPSIS

```
Widget    XnslDrawRefNameToObject (w, group, name)
Widget          w;
Widget          group;
String         name;
```

DESCRIPTION

XnslDrawRefNameToObject returns the first widget with the given *name*³ in the *group* list. A child widget can be accessed by asking for its *name* in the form *#<child number>*. For example:

```
XnslDrawRefNameToObject (w, group, "#4")
```

returns the fourth element of *group* *group*. This naming scheme allows the user to create independent groups which can be duplicated and allows the scripts associated with these groups to work regardless of the children's names.

3. Since the Xt toolkit does not allow you to change the name of a widget, all XnslDrawObjects have an XnslNdrawName resource which can be changed at any time. At initialization time, the Xt name, (which can be retrieved by the XtName() function) and the XnslNdrawName are identical. If you change the name of an XnslDrawObject by modifying the XnslNdrawName resource, the return value from the XtName() function and the XnslNdrawName resource have different values.

The following FACE statement will then fail because the FACE naming scheme uses XtName to locate widgets:

```
self : drawName = "NewName";
parent . NewName : drawForeground = red;
```

But this one is OK :

```
self : drawName = "NewName";
w = XnslDrawNameToObject(parent, "NewName");
w : drawForeground = red;
```

NAME

XnslDrawSetObjectName

SYNOPSIS

```
None      XnslDrawSetObjectName (w, o, new_name)
Widget    w;
Widget    o;
String    new_name;
```

DESCRIPTION

XnslDrawSetObjectName sets the name⁴ of widget *o* to the name in *new_name*. **XnslDrawSetObjectName** copies the string into allocated data, so the application may free the data upon its return.

4. Since the Xt toolkit does not allow you to change the name of a widget, all XnslDrawObjects have an XnslNdrawName resource which can be changed at any time. At initialization time, the Xt name, (which can be retrieved by the XtName() function) and the XnslNdrawName are identical.

If you change the name of an XnslDrawObject by modifying the XnslNdrawName resource, the return value from the XtName() function and the XnslNdrawName resource have different values.

The following FACE statement will then fail because the FACE naming scheme uses XtName to locate widgets:

```
self : drawName = "NewName";
parent . NewName : drawForeground = red;
```

But this one is OK :

```
self : drawName = "NewName";
w = XnslDrawNameToObject(parent, "NewName");
w : drawForeground = red;
```

NAME

XnslDrawSetUserData

SYNOPSIS

```
None      XnslDrawSetUserData(w, o, data)
Widget    w;
Widget    o;
Pointer    data;
```

DESCRIPTION

This function affects *data* to the user data of object *o*. This value is stored in a special field of the object and can be retrieved by the `XnslDrawGetUserData` function. It is not used internally by the widget.

NAME

XnslDrawShowObject

SYNOPSIS

```
None XnslDrawShowObject (w, o)
                        widget      w;
                        widget      o;
```

DESCRIPTION

XnslDrawShowWidget makes the given widget *o* visible. Note that **Xt-ManageChild** gives the same results.

11.1 Stack Management

The functions in this section are used to modify the object stack order.

NAME

XnslDrawBringToFront

SYNOPSIS

```
None XnslDrawBringToFront (w, o)
      Widget                w;
      Widget                o;
```

DESCRIPTION

XnslDrawBringToFront brings the *o* object's view to the top.

NAME

XnslDrawCirculateDown

SYNOPSIS

```
None XnslDrawCirculateDown (w, o)
      Widget                w;
      Widget                o;
```

DESCRIPTION

XnslDrawCirculateDown sends the *o* widget's view one step down.

NAME

XnslDrawCirculateUp

SYNOPSIS

```
None XnslDrawCirculateUp (w, o)
      Widget             w;
      Widget             o;
```

DESCRIPTION

XnslDrawCirculateUp brings the *o* widget's view one step up.

NAME

XnslDrawSendToBack

SYNOPSIS

```
None XnslDrawSendToBack (w, o)
      Widget             w;
      Widget             o;
```

DESCRIPTION

XnslDrawSendToBack brings widget *o*'s view to the back.

11.2 SelectionManagement

The functions in this section are used to look for selected object.

NAME

XnslDrawFirstSelected

SYNOPSIS

```
Widget XnslDrawFirstSelected (w)
      Widget          w;
```

DESCRIPTION

XnslDrawFirstSelected returns the first selected widget in widget *w*.

NAME

XnslDrawNextSelected

SYNOPSIS

```
Widget XnslDrawNextSelected (w, prev)
    Widget      w;
    Widget      prev;
```

DESCRIPTION

XnslDrawNexttSelected returns the next selected widget in widget *w*, after widget *prev*, or **NULL** if *prev* is the last widget in the selected widget list.

11.3 Object Type Checking Functions

The functions described in this section should be used to check if an object is of the specified type.

NAME

XnslIsDrawArc

SYNOPSIS

```
Int XnslIsDrawArc(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawArc checks if widget *o* is a widget of type arc.

NAME

XnslIsDrawEllipse

SYNOPSIS

```
Int XnslIsDrawEllipse(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawEllipse checks if widget *o* is a widget of type ellipse.

NAME

XnslIsDrawGroup

SYNOPSIS

```
Int XnslIsDrawGroup(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawGroup checks if widget *o* is a widget of type group.

NAME

XnslIsDrawImage

SYNOPSIS

```
Int XnslIsDrawImage(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawImage checks if widget *O* is a widget of type image.

NAME

XnslIsDrawLabel

SYNOPSIS

```
Int XnslIsDrawLabel(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawLabel checks if widget *O* is a widget of type label.

NAME

XnslIsDrawLine

SYNOPSIS

```
Int XnslIsDrawLine(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawLine checks if widget *O* is a widget of type line.

NAME

XnslIsDrawObject

SYNOPSIS

```
Int XnslIsDrawObject(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawObject checks if widget *O* is a widget of type object.

NAME

XnslIsDrawPath

SYNOPSIS

```
Int XnslIsDrawPath(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawPath checks if widget *O* is a widget of type path.

NAME

XnslIsDrawPolygon

SYNOPSIS

```
Int XnslIsDrawPolygon(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawPolygon checks if widget *O* is a widget of type polygon.

NAME

XnslIsDrawRectangle

SYNOPSIS

```
Int XnslIsDrawRectangle(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawRectangle checks if widget *o* is a widget of type rectangle.

NAME

XnslIsDrawSegment

SYNOPSIS

```
Int XnslIsDrawSegment(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawSegment checks if widget *o* is a widget of type segment.

NAME

XnslIsDrawSpline

SYNOPSIS

```
Int XnslIsDrawSpline(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawSpline checks if widget *O* is a widget of type spline.

NAME

XnslIsDrawText

SYNOPSIS

```
Int XnslIsDrawText(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawText checks if widget *o* is a widget of type text.

NAME

XnslIsDrawWidget

SYNOPSIS

```
Int XnslIsDrawWidget(o)
Widget    o;
```

DESCRIPTION

XnslIsDrawWidget checks if widget *O* is a widget of type widget.

11.4 Image Functions

The functions in this section should be used for to handle objects of type `image`.

NAME

XnslDrawGetImage

SYNOPSIS

```
Int      XnslDrawGetImage (w, o)
Widget   w;
Widget   o;
```

DESCRIPTION

XnslDrawGetImage returns the index into the pixmap table for the pixmap displayed by the widget *o*. The class of widget *o* must be **XnslDrawImage**

NAME

XnslDrawSetImage

SYNOPSIS

```
None      XnslDrawSetImage (w, o, i)
           Widget          w;
           Widget          o;
           Int              i;
```

DESCRIPTION

XnslDrawSetImage sets the pixmap displayed by tobject *o* to the pixmap whose index is *i* in the widget's pixmap table. The *index* is a value returned by the **XnslDrawGetPixmap** function. The class of widget *o* must be **XnslDrawImage**.

11.5 Group Functions

The functions in this section should be used to manage groups of objects.

NAME

XnslDrawAddInGroup

SYNOPSIS

```
None      XnslDrawAddInGroup (group, elem)
Widget    group;
Widget    elem;
```

DESCRIPTION

XnslDrawAddInGroup Adds the widget *elem* in group *group*.

NAME

XnslDrawRemoveFromGroup

SYNOPSIS

```
None      XnslDrawRemoveFromGroup (group, elem)
Widget    group;
Widget    elem;
```

DESCRIPTION

XnslDrawRemoveFromGroup removes widget *elem* from group *group*

NAME

XnslDrawUngroupGroup

SYNOPSIS

```
None      XnslDrawUngroupGroup (group)
          Widget      group;
```

DESCRIPTION

XnslDrawUngroupGroup dissociates the group *group*. The group is destroyed and the children become independent.

11.6 Ellipse Functions

The following functions are used for ellipse management. Angles are measured in object units; i.e. the unit of measurement chosen for the object.

NAME

XnslDrawGetEllipseEndAngle

SYNOPSIS

```
Float      XnslDrawGetEllipseEndAngle (w, o)
Widget      w;
Widget      o;
```

DESCRIPTION

XnslDrawGetEllipseEndAngle returns the angle representing the ending point of the arc. The class of the widget *o* must be **XnslDrawEllipse**. The unit of the resulting angle is given by the resource **XnslNdrawAngleUnits** of widget *o*.

If the value of this resource is **XnslNO_ANGLE_UNITS** and the class of widget *o*'s parent is **XnslDraw**, then unit is taken from the resource **XnslNdrawAngleUnits** of widget *o*'s parent.

If the value is also **XnslNO_ANGLE_UNITS**, the unit is **radian**.

NAME

XnsldrawGetEllipseStartAngle

SYNOPSIS

```
Float      XnsldrawGetEllipseStartAngle (w, o)
Widget      w;
Widget      o;
```

DESCRIPTION

XnsldrawGetEllipseStartAngle returns the angle representing the starting point of the arc drawn using the given widget. The class of widget *o* must be **XnsldrawEllipse**. The unit of the resulting angle is given by the resource **XnsldrawAngleUnits** of the widget *o*.

If the value of this resource is **XnsINO_ANGLE_UNITS** and the class of widget *o*'s parent is **Xnsldraw**, then unit is taken from the resource **XnsldrawAngleUnits** of widget *o*'s parent.

If the value is also **XnsINO_ANGLE_UNITS**, the unit is **radian**.

NAME

XnsIDrawSetEllipseAngles

SYNOPSIS

```
None XnsIDrawSetEllipseAngles (w, o, start, end)
Widget      w;
Widget      o;
Float       start;
Float       end;
```

DESCRIPTION

XnsIDrawSetEllipseAngles sets the angles of widget *o* to the values *start* and *end*. The class of the widget *o* must be **XnsIDrawEllipse**.

The unit for *start* and *end* is given by the resource **XnsINdrawAngleUnits** of the widget *o*. If the value of this resource is **XnsINO_ANGLE_UNITS** and the class of widget *o*'s parent is **XnsIDraw**, then unit is taken from the resource **XnsINdrawAngleUnits** of widget *o*'s parent.

If the value is also **XnsINO_ANGLE_UNITS**, the unit is **radian**.

NAME

XnsIDrawSetEllipseEndAngle

SYNOPSIS

```
None      XnsIDrawSetEllipseEndAngle (w, o, angle)
Widget    w;
Widget    o;
Float     angle;
```

DESCRIPTION

XnsIDrawSetEllipseEndAngle sets the ending angle of widget *o* to the value *angle*. The class of the widget *o* must be **XnsIDrawEllipse**.

The unit for *angle* is given by the resource **XnsINdrawAngleUnits** of widget *o*. If the value of this resource is **XnsINO_ANGLE_UNITS** and the class of widget *o*'s parent is **XnsIDraw**, then unit is taken from the resource **XnsINdrawAngleUnits** of widget *o*'s parent.

If the value is also **XnsINO_ANGLE_UNITS**, the unit is **radian**.

NAME

XnsIDrawSetEllipseStartAngle

SYNOPSIS

```
None XnsIDrawSetEllipseStartAngle (w, o, angle)
      Widget          w;
      Widget          o;
      Float           angle;
```

DESCRIPTION

XnsIDrawSetEllipseStartAngle sets the starting angle of widget *o* to the value *angle*. The class of widget *o* must be **XnsIDrawEllipse**.

The unit for *angle* is given by the resource **XnsIDrawAngleUnits** of widget *o*. If the value of this resource is **XnsINO_ANGLE_UNITS** and the class of the widget *o*'s parent is **XnsIDraw**, then unit is taken from the resource **XnsIDrawAngleUnits** of widget *o*'s parent.

If the value is also **XnsINO_ANGLE_UNITS**, the unit is **radian**.

12 Pixmap Table Functions

The functions in this section are used to work with pixmaps in the pixmap table.

NAME

XnsIDrawClearPixmapTable

SYNOPSIS

```
None XnsIDrawClearPixmapTable (w, free)
      Widget          w;
      Int             free;
```

DESCRIPTION

XnsIDrawClearPixmapTable frees all pixmaps in the table. If *free* is **True**, it also frees the memory for the table itself. The application should pass **False** for *free* if it expects to load new pixmaps in the table.

NAME

XnslDrawGetPixmap

SYNOPSIS

```
Int  XnslDrawGetPixmap (w, name, null)
      Widget           w;
      String           name;
      Pointer          null;
```

DESCRIPTION

XnslDrawGetPixmap searches for the pixmap with name *name* in the internal pixmap table. If it finds the pixmap, it returns its index. If not, it tries to load the pixmap. If the process's environment has the variable **DRAWPIXMAPSPATH** set, it uses this path to determine where to search for the pixmap, otherwise, it tries the variable **XBMLANGPATH**. Once **XnslDrawGetPixmap** has loaded the pixmap, it places it in the table and returns the new index. If it cannot load the pixmap, it returns -1.

Within XDrawMaker, the third argument must always be set to 0, as the FACE language does not allow you to retrieve values using pointers.

NAME

XnslDrawPixmapName

SYNOPSIS

```
String    XnslDrawPixmapName (w, index)
          Widget          w;
          Int             index;
```

DESCRIPTION

XnslDrawPixmapName returns the name of the pixmap at the given index in the internal pixmap table. This is the same name that the application had passed to **XnslDrawGetPixmap**.

NAME

XnslDrawPixmapPixmap

SYNOPSIS

```
Pixmap XnslDrawPixmapPixmap (w, index)
    Widget      w;
    Int         index;
```

DESCRIPTION

XnslDrawPixmapPixmap returns the pixmap ID of the pixmap at the given *index* in the internal pixmap table.

NAME

XnsIDrawPixmapType

SYNOPSIS

```
Int  XnsIDrawPixmapType (w, index)
      Widget             w;
      Int                index;
```

DESCRIPTION

XnsIDrawPixmapType returns the type of the pixmap at the given index *index* in the internal pixmap table. The type is one of the following:

XnsIDRAW_BITMAP_TYPE

An X bitmap file.

XnsIDRAW_XWD_TYPE

An X window dump file.

XnsIDRAW_GIF_TYPE

A Gif (Graphic Interchange Format) file.

XnsIDRAW_PIXMAP_TYPE

An X Pixmap file.

XnsIDRAW_XPM_TYPE

An Xpm3 file.

NAME

XnslDrawReleasePixmap

SYNOPSIS

```
Boolean  XnslDrawReleasePixmap(w, num)
        Widget      w;
        Int         num;
```

DESCRIPTION

XnslDrawReleasePixmap checks that the pixmap whose index is *num* is not being used before freeing the pixmap in the pixmap table and any colors allocated specifically for this pixmap. If the pixmap is in use, the function waits until it is no longer in use, then frees the pixmap.

Index *num* is subsequently free for use by a new pixmap allocated using **XnslDrawGetPixmap**.

The function returns False if the pixmap is not found and True otherwise.

13 Script Functions

The following functions are used to manage the invocation of FACE scripts. The following values are valid for the *location* argument:

- `draw_on_border`
- `draw_inside`
- `draw_outside`
- `draw_nowhere`
- `draw_everywhere`

If no location is specified, the triggering condition is *draw_on_border OR draw_inside*.

NAME

XnslDrawArmScript

SYNOPSIS

```
None      XnslDrawArmScript (w, o, time, action,
                                location, cd)

Widget     w;
Widget     o;
Int        time;
String     action;
String     location;
Pointer    cd;
```

DESCRIPTION

XnslDrawArmScript arranges to call the script for the given *action* and the given location *location* *time* milliseconds later. **XnslDrawArmScript** will set the call data for the script to *cd*. If *o* is not **NULL**, the function applies to the scripts of widget *o*. If *o* is **NULL**, the function refers to the script defined for the Widget *w*. If *action* is **NULL**, the function arms matching scripts regardless of the action. If *location* is **NULL** the function arms matching scripts regardless of the location.

EXAMPLE

```
XnslDrawArmScript (w, o, 1000, NULL, NULL, 0);
```

Will launch all the scripts of widget *o* one second later.

```
XnslDrawArmScript(w, o, 1000, "Button1Down", NULL, 0);
```

Will launch the scripts of widget *o* which were recorded with event **Button1Down**, regardless of the location of the event.

```
XnslDrawArmScript (w, NULL, 1000, NULL, NULL, 0);
```

Will launch all the scripts of widget *w* one second later.

NAME

XnslDrawCallScript

SYNOPSIS

```
None XnslDrawCallScript (w, o, action, location, cd)
    Widget          w;
    Widget          o;
    String          action;
    String          location;
    Pointer         cd;
```

DESCRIPTION

XnslDrawCallScript calls the script for the event *action* and the location *location*. **XnslDrawCallScript** sets the call data for the script to *cd*.

If *o* is not **NULL**, the function looks for matching scripts in the script list of object *o*..

If *o* is **NULL**, then the function looks for matching scripts in the script list of widget *w*.

If *action* is **NULL**, the function searches matching scripts regardless of the action.

If *location* is **NULL** the function searches for matching scripts regardless of the location.

EXAMPLE

```
XnslDrawCallScript (w, o, NULL, NULL, 0);
```

Will call the scripts for widget *o* regardless of the event and location.

```
XnslDrawCallScript(w, NULL, "Button2Down", NULL, 0);
```

Will call the scripts for widget *w* which were registered for event Button2Down regardless of the location.

NAME

XnslDrawCallScriptByName

SYNOPSIS

```
None XnslDrawCallScriptByName (w, action, where, cd)
    Widget          w;
    String          action;
    String          where;
    XtPointer       cd;
```

DESCRIPTION

XnslDrawCallScriptByName calls the scripts of all objects in widget *w* for the given event *action* and the location *where*. **XnslDrawCallScript-ByName** sets the call data for the script to *cd*.

If *action* is **NULL**, the function searches matching scripts regardless to the action.

If *where* is **NULL** the function searches for matching scripts regardless to the location.

NAME

XnslDrawDisableScripts

SYNOPSIS

```
None      XnslDrawDisableScripts (w, o)
Widget    w;
Widget    o;
```

DESCRIPTION

XnslDrawDisableScripts disables all the scripts of a widget. If widget *o* is not **NULL**, the scripts of this widget are disabled. If widget *o* is **NULL**, the scripts attached to widget *w* are disabled.

NAME

XnslDrawDisarmScript

SYNOPSIS

```
None XnslDrawDisarmScript (w, o, action, location)
Widget      w;
Widget      o;
String      action;
String      location;
```

DESCRIPTION

XnslDrawDisarmScript removes the time-out that calls the script for event *action* and location *location*. If *o* is **NULL**, then the function applies to widget *w*.

NAME

XnslDrawEnableScripts

SYNOPSIS

```
None      XnslDrawEnableScripts (w, o)
          Widget      w;
          Widget      o;
```

DESCRIPTION

XnslDrawEnableScripts enables the scripts of widget *o* if *o* is not **NULL** .
If widget *o* is **NULL**, the scripts attached to widget *w* are enabled.

14 Slot Functions

The functions in this section are used to control slots.

NAME

XnsldrawClearSlotSelection

SYNOPSIS

```
None XnsldrawClearSlotSelection (draw)
      widget          draw;
```

DESCRIPTION

XnsldrawClearSlotSelection deselects all slots of all widgets contained in the XDraw widget *draw*.

NAME

XnslDrawGetSlotPixmap

SYNOPSIS

```
Int XnslDrawGetSlotPixmap (draw, widget, i)
    Widget      draw;
    Widget      widget;
    Int          i;
```

DESCRIPTION

XnslDrawGetSlotPixmap returns the index of the pixmap associated to slot number *i* of object *widget* in XDraw widget *draw*.

NAME

XnslDrawGetSlotSize

SYNOPSIS

```
Int XnslDrawGetSlotSize (draw, widget, i)
    Widget      draw;
    Widget      widget;
    Int         i;
```

DESCRIPTION

XnslDrawGetSlotSize returns the size of slot number *i* of object *widget* in XDraw widget *draw*.

NAME

XnslDrawSelectObjectSlots

SYNOPSIS

```
None XnslDrawSelectObjectSlots (w, select)
      Widget          w;
      Int             select;
```

DESCRIPTION

XnslDrawSelectObjectSlots selects or deselects all slots of object *w* according to the value of *select*. If *select* is *1*, the slots are selected. If *select* is *0*, the slots are deselected.

NAME

XnslDrawSelectSlot

SYNOPSIS

```
None XnslDrawSelectSlot (w, i, select)
      Widget             w;
      Int                i;
      Int                select;
```

DESCRIPTION

XnslDrawSelectSlot selects or deselects slot *i* of object *w* according to the value of *select*. If *select* is 0, the slot is deselected. if *select* is 1, the slot is selected.

NAME

XnslDrawSetSlotPixmap

SYNOPSIS

```
None XnslDrawSetSlotPixmap (draw, w, i, pixmap)
      Widget      draw;
      Widget      w;
      Int         i;
      Int         pixmap;
```

DESCRIPTION

XnslDrawSetSlotPixmap specifies the pixmap whose index is *pixmap* in the pixmap table of widget *draw* as the pixmap for slot number *i* of object *w*.

NAME

XnsIDrawSetSlotSize

SYNOPSIS

```
None XnsIDrawSetSlotSize (draw, w, i, size)
      Widget      draw;
      Widget      w;
      Int         i;
      Int         size;
```

DESCRIPTION

XnsIDrawSetSlotSize sets the size of slot number *i* of object *w* in widget *draw* to *size*. The size is specified in pixels, it is the length of a side of the square materializing the slot.

NAME

XnslDrawToggleSelectSlot

SYNOPSIS

```
Int  XnslDrawToggleSelectSlot(draw, w, s)
      Widget      draw;
      Widget      w;
      Int          s;
```

DESCRIPTION

XnslDrawToggleSelectSlot toggles the selection status of slot *s* of object *w* in widget *draw*.

15 Text Functions

The functions in this section deal with the management of text objects.

Text positions are measured in character offsets from the first one, and their type is **XnsIDrawTextPosition**. DTIF is the Draw Text Interchange Format. This is a human readable format used to store graphic attributes for text.

15.1 Text Alignment Functions

The functions in this section concern text alignment. Alignment can have the following values:

- **XnsIDRAW_TEXT_ALIGN_LEFT**
- **XnsIDRAW_TEXT_ALIGN_RIGHT**
- **XnsIDRAW_TEXT_ALIGN_CENTER**
- **XnsIDRAW_TEXT_JUSTIFY**
- **XnsIDRAW_TEXT_JUSTIFY_FULL**

NAME

XnslDrawTextSetAlignment

SYNOPSIS

```
None      XnslDrawTextSetAlignment(wid, align)
Widget    wid;
DrawTextAlignment align;
```

DESCRIPTION

XnslDrawTextSetAlignment sets the default alignment and the alignment for all paragraphs to *align* for widget *wid*. The class of widget *wid* must be **XnslDrawText**. Possible values for *align* are:

XnslDRAW_TEXT_ALIGN_LEFT

Text is flushed on the left.

XnslDRAW_TEXT_ALIGN_RIGHT

Text is flushed on the right.

XnslDRAW_TEXT_ALIGN_CENTER

Text is centered

XnslDRAW_TEXT_JUSTIFY

Text is full justified except on the last line.

XnslDRAW_TEXT_JUSTIFY_FULL

Text is full justified.

NAME

XnslDrawTextSetJustif

SYNOPSIS

```
None      XnslDrawTextSetJustif(w, o, i)
Widget    w;
Widget    o;
Int       i;
```

DESCRIPTION

This function is obsolete and is replaced by **XnslDrawTextSetAlignment**.

The first argument is not used.

NAME

XnslDrawTextSetParAlignFromPos

SYNOPSIS

```
Boolean XnslDrawTextSetParAlignFromPos(wid, pos,
                                         align)
                                         Widget      wid;
                                         XnslDrawTextPosition pos;
                                         DrawTextAlignment  align;
```

DESCRIPTION

XnslDrawTextSetParAlignFromPos changes the alignment of the paragraph to which the position *pos* belongs in widget *wid*. Possible values for *align* are:

XnslDRAW_TEXT_ALIGN_LEFT

Text is flushed on the left.

XnslDRAW_TEXT_ALIGN_RIGHT

Text is flushed on the right.

XnslDRAW_TEXT_ALIGN_CENTER

Text is centered

XnslDRAW_TEXT_JUSTIFY

Text is full justified except on the last line.

XnslDRAW_TEXT_JUSTIFY_FULL

Text is full justified.

The class of widget *wid* must be **XnslDrawText**.

If *pos* is negative or too large, the function returns **False**.

NAME

XnslDrawTextSetParAlignment

SYNOPSIS

```
Boolean XnslDrawTextSetParAlignment(wid, align)
      Widget          wid;
      DrawTextAlignment align;
```

DESCRIPTION

This function changes the alignment of the current paragraph of widget *wid* to *align*. The return value of this function should be ignored. Possible values for *align* are:

XnslDRAW_TEXT_ALIGN_LEFT

Text is flushed on the left.

XnslDRAW_TEXT_ALIGN_RIGHT

Text is flushed on the right.

XnslDRAW_TEXT_ALIGN_CENTER

Text is centered

XnslDRAW_TEXT_JUSTIFY

Text is full justified except on the last line.

XnslDRAW_TEXT_JUSTIFY_FULL

Text is full justified.

The class of widget *wid* must be **XnslDrawText**.

15.2 Text Attribute Functions

The functions in this section are used to control the text's graphic attributes.

NAME

XnslDrawTextChangeAttributesAtPos

SYNOPSIS

```
None      XnslDrawTextChangeAttributesAtPos(wid,  
                                              app_mark, from, to)  
Widget                                         wid;  
XnslDrawTextMark                             app_mark;  
XnslDrawTextPosition                         from;  
XnslDrawTextPosition                         to;
```

DESCRIPTION

XnslDrawTextChangeAttributesAtPos specifies new attributes for a text segment. The class of widget *wid* must be **XnslDrawText**. *app_mark* is a pointer to the structure specifying the mark parameters. *from* is the position of the first character to which the new attributes will be applied. *to* is the position of the last character to which the new attributes will be applied.

NAME

XnslDrawTextChangeFont

SYNOPSIS

```
None XnslDrawTextChangeFont(w, o, font_name, start,  
                             end)
```

```
Widget      w;  
Widget      o;  
String      font_name;  
Int         start;  
Int         end;
```

DESCRIPTION

If widget *o* is an **XnslDrawLabel**, this function sets the resource **XnslNdraw-Font**. If the widget is an **XnslDrawText**, this function changes the font between *start* and *end* to the font whose name is *font_name*. The first argument is not used.

NAME

XnslDrawTextGiveStateAtPos

SYNOPSIS

XnslDrawTextMark XnslDrawTextGiveStateAtPos

```
(wid, pos)
    Widget      wid;
    XnslDrawTextPosition    pos;
```

DESCRIPTION

XnslDrawTextGiveStateAtPos returns a pointer to the structure specifying the text attributes at the position *pos* of widget *wid*. The returned structure is allocated. It should be freed when no longer needed. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextInsertMark

SYNOPSIS

```
Boolean   XnslDrawTextInsertMark(wid, mark)
Widget    wid;
XnslDrawTextMark  mark;
```

DESCRIPTION

XnslDrawTextInsertMark places the **XnslDrawTextMark** *mark* in widget *wid*. *mark* is a pointer to the structure specifying the text attributes. The mark is placed according to the **pos** field. If the position is negative or too large, the function returns **False**. The class of widget *wid* must be **XnslDrawText**.

15.3 Text File Functions

The functions in this section concern file input and output to the XnslDrawText widget.

NAME

XnslDrawTextInsertAsciiFile

SYNOPSIS

```
None XnslDrawTextInsertAsciiFile(wid, file_name)
      Widget      wid;
      String      file_name;
```

DESCRIPTION

XnslDrawTextInsertAsciiFile inserts the contents of the ASCII file *file_name* in widget *wid* at the insertion point. After insertion, the insertion point is moved to the end of the text inserted. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextInsertDtifFile

SYNOPSIS

```
Int XnslDrawTextInsertDtifFile(wid, file_name)
    Widget      wid;
    String      file_name;
```

DESCRIPTION

XnslDrawTextInsertDtifFile inserts the contents of the DTIF file *file_name* in widget *wid* at the insertion point. After insertion, the insertion point is moved to the end of the text inserted. If an error occurred the function returns -1. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextReadDtifFile

SYNOPSIS

```
Int      XnslDrawTextReadDtifFile(wid, file_name)
Widget   wid;
String   file_name;
```

DESCRIPTION

XnslDrawTextReadDtifFile loads the contents of the DTIF file *file_name* into widget *wid* at the curent insertion position. After insertion, the insertion point is moved to the end of the text inserted.

This function overrides some of the widget's resources: XnslNdrawTextAlignment, XnslNdrawTextHorizontalMargin, XnslNdrawTextParagraphSpacing, XnslNdrawTextReadOnly, XnslNdrawTextResizePolicy, XnslNdrawTextShowCursor, XnslNdrawTextVerticalMargin..

If an error occurred the function returns -1. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextSaveAsciiText

SYNOPSIS

```
Boolean XnslDrawTextSaveAsciiText(wid, file_name,  
                                   from, to)  
  
Widget      wid;  
String      file_name;  
XnslDrawTextPosition from;  
XnslDrawTextPosition to;
```

DESCRIPTION

XnslDrawTextSaveAsciiText saves the partial contents of widget *wid* in file *file_name* in ASCII format. *from* is the position of the first character saved, *to* is the position of the last character saved. If an error occurred the function returns **False**. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextSaveDtifText

SYNOPSIS

```
Boolean XnslDrawTextSaveDtifText(wid, file_name)
      Widget      wid;
      String      file_name;
```

DESCRIPTION

XnslDrawTextSaveDtifText saves the contents of widget *wid* in the file *file_name*, in DTIF format. If an error occurred the function returns **False**. The class of widget *wid* must be **XnslDrawText**.

15.4 Text Insertion Functions

The functions in this section are used to get or insert text in a text object.

NAME

XnslDrawTextGetAsciiText

SYNOPSIS

```
String XnslDrawTextGetAsciiText (wid, from, to)
Widget                                wid;
XnslDrawTextPosition                 from;
XnslDrawTextPosition                 to;
```

DESCRIPTION

XnslDrawTextGetAsciiText returns a text segment in ASCII format. The text returned is the segment enclosed between positions *from* and *to*. It is allocated and it is a null terminated string. The class of widget *wid* must be **XnslDrawText**.

EXAMPLE

Read and print the entire content of text object *self*:

```
last = XnslDrawTextGetLastPosition(self);
s = XnslDrawTextGetAsciiText
(self, XnslDrawTextPosition(0), last);
printf ("text: %s\n", s);
free(s);
```

NAME

XnslDrawTextGetString

SYNOPSIS

```
String    XnslDrawTextGetString (wid, o)
Widget    wid;
Widget    o;
```

DESCRIPTION

This function returns in ASCII format the contents of widget *o* which can be either an **XnslDrawText**, or an **XnslDrawLabel**.

This function is obsolete and has been replaced by **XnslDrawTextGetAsciiText** to read the ASCII content of an **XnslDrawText** object.

To read the contents of an **XnslDrawLabel** object , read its **drawLabelString** resource.

EXAMPLE

Read and print the label text of **XnslDrawLabel** *w*

```
printf(" resource: %s\n",w:drawLabelString);
```

NAME

XnslDrawTextInsertAsciiAtPos

SYNOPSIS

```
None XnslDrawTextInsertAsciiAtPos(wid, str, pos)
      Widget                        wid;
      String                       str;
      XnslDrawTextPosition         pos;
```

DESCRIPTION

XnslDrawTextInsertAsciiAtPos inserts text *str* in ASCII format in widget *wid* at position *pos*. The insertion point is moved to the end of the text inserted. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextInsertAsciiText

SYNOPSIS

```
None      XnslDrawTextInsertAsciiText(wid, str)
Widget    wid;
String    str;
```

DESCRIPTION

XnslDrawTextInsertAsciiText inserts text *str* in ASCII format in widget *wid* at the current insertion position. The insertion point is moved to the end of the text inserted. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextInsertDtifText

SYNOPSIS

```
None      XnslDrawTextInsertDtifText(wid, str)
Widget    wid;
String    str;
```

DESCRIPTION

XnslDrawTextInsertDtifText inserts text *str* in DTIF format in widget *wid* at the current insertion position. The insertion point is moved to the end of the text inserted. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextInsertDtifTextAtPos

SYNOPSIS

```
None XnslDrawTextInsertDtifTextAtPos(wid, str, pos)
      Widget                               wid;
      String                              str;
      XnslDrawTextPosition                pos;
```

DESCRIPTION

XnslDrawTextInsertDtifTextAtPos inserts text *str* in DTIF format in widget *wid* at position *pos*. The insertion point is moved to the end of the text inserted. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextInsertString

SYNOPSIS

```
None      XnslDrawTextInsertString(w, o, s)
          Widget      w;
          Widget      o;
          String      s;
```

DESCRIPTION

If widget *o* is an **XnslDrawText**, this function inserts the contents of the NULL terminated string *s* in the widget *o*. If widget *o* is an **XnslDrawLabel**, this function replaces its contents with *s*.

This function is obsolete and has been replaced by **XnslDrawTextInsertAsciiText** to specify the ASCII content of an **XnslDrawText** object.

To specify the contents of an **XnslDrawLabel** object , set its **drawLabelString** resource.

15.5 Text Killer Functions

The functions in this section are used to delete text in text objects.

NAME

XnslDrawTextClearText

SYNOPSIS

```
None      XnslDrawTextClearText (wid)
          Widget      wid;
```

DESCRIPTION

XnslDrawTextClearText deletes all text from widget *wid*. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextEraseBuffer

SYNOPSIS

```
None      XnslDrawTextEraseBuffer(w, o)
Widget    w;
Widget    o;
```

DESCRIPTION

This function deletes all text from object *o* in widget *w*. The class of widget *o* may be **XnslDrawText**, **XnslDrawTextWidget** or **XnslDrawLabel**.

NAME

XnslDrawTextRuboutChar

SYNOPSIS

```
None XnslDrawTextRuboutChar(wid, count)
      Widget                wid;
      Int                   count;
```

DESCRIPTION

XnslDrawTextRuboutChar deletes *count* characters in widget *wid*, to the right of the current insertion position. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextRuboutCharAtPos

SYNOPSIS

```
None XnslDrawTextRuboutCharAtPos(wid, pos, count)
      Widget                        wid;
      XnslDrawTextPosition        pos;
      Int                          count;
```

DESCRIPTION

XnslDrawTextRuboutCharAtPos deletes *count* characters in widget *wid*, starting at a given position *pos*. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextYank

SYNOPSIS

```
Boolean   XnslDrawTextYank (wid)
Widget    wid;
```

DESCRIPTION

XnslDrawTextYank pastes the current selection in widget *wid*, at the current insertion position. The insertion point is moved to the end of the text inserted. The class of widget *wid* must be **XnslDrawText**.

15.6 Text Position Functions

The functions in this section can be used to specify text insertion positions.

NAME

XnslDrawTextGetInputPosition

SYNOPSIS

```
Int      XnslDrawTextGetInputPosition(w, o)
Widget   w;
Widget   o;
```

DESCRIPTION

This function is obsolete and is replaced by the function **XnslDrawTextGetPosition**. The first argument is not used.

NAME

XnslDrawTextGetLastPosition

SYNOPSIS

```
XnslDrawTextPosition XnslDrawTextGetLastPosition
                        (wid)
Widget                wid;
```

DESCRIPTION

XnslDrawTextGetLastPosition returns the position of the last character in widget *wid*. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextGetPosition

SYNOPSIS

```
XnslDrawTextPosition XnslDrawTextGetPosition(wid)
Widget               wid;
```

DESCRIPTION

XnslDrawTextGetPosition returns the current position of the insertion point in widget *wid*. The class of widget *wid* must be **XnslDrawText**.

NAME

XnslDrawTextSetInputPosition

SYNOPSIS

```
None      XnslDrawTextSetInputPosition(w, o, pos)
Widget    w;
Widget    o;
Int       pos;
```

DESCRIPTION

This function is obsolete and is replaced by the function **XnslDrawTextSet-Position**. The first argument is not used.

NAME

XnslDrawTextSetPosition

SYNOPSIS

```
None      XnslDrawTextSetPosition (wid, pos)
Widget    wid;
XnslDrawTextPosition pos;
```

DESCRIPTION

XnslDrawTextSetPosition places the insertion point of widget *wid* at the position specified by *pos*. *pos* is an offset, in characters from the beginning of the widget, of the new position for the insertion point. The beginning of the widget is at offset 0. The class of widget *wid* must be **XnslDrawText**.

Symbols

\$ 107

.ps files 29

A

action

- button 79

- Destroy 79

- KeyPress 79

- KeyRelease 79

- Load 79

- MouseMoved 79

- Resize 79

- translation 80

actions 23, 80

Actions menu 79

Activate menu 77, 139

- Script Editing box 23

add point 68

AddGrab 121

aligning

- objects 75

- text 63

angles

- defining 51

- units of measurement 9

animating objects 22

Any 90, 128

application functions 108

- declaring 108

ApplicationShell

- in FACE 95

Apply button 11

arc 36

Arrange menu 139

Array 128
arrays 111
 clearing 116
 hash array 111
 in FACE 111
 in structures 119
 linear 111
arrow functions 142
arrows 52
assignments 97, 102
atoi 124
auto 92
B
Background Color icon 44, 49
backUp 131
-backup 131
bitmap table functions 146
borders 49
button action 79
C
call_data 107
Callback 121
callback 124
callbacks
 in FACE 92, 119
CallCallbacks 121
cancel actions 14, 67
center
 defining 51
 resetting 69
center functions 186
cftoi 124
chord 50
Circulate Down 20

Circulate Up 20

citof 124

clear 125

click 4

client_data 121

Close icon 43

close mode 49

color allocation 66

color table functions 154

color value 44

-colors 131

colors 131

- background 49

- dashed lines 53

- foreground 48

combined files

- saving 31

command line options 131

-compile 131

ConfigureWidget 121

configuring

- command line options 131

- XDrawMaker 131

connecting objects

- links and slots 17

- paths 40

constants 90

- defined 94

- defining 100

- floating-point 109

- OSF/Motif 98

control structures 102

ConvertString 101, 121

copying

- files 28
- objects 71
- core 51
- CursorPosition** 128
- custom 131
- custom button 4
- Custom Editor 22, 133
- custom page size 10
- cut 71
- D
- dash 53
- data** 116, 125
- default values 9
- define (keyword) 94
- defined constant 94
- defining a center 69
- delete all 71
- deleting
 - objects 71
 - slots 18
- demo mode 7
- demos 1
- deselecting objects 20
- desktop 9
- destroy** 113, 125
- Destroy action 79
- DestroyWidget** 121
- dm command 8
- DM_LICENSE_NAME 132
- DMIMAGEDIR 132
- document
 - page size 10
 - setting up 10
- drag 4

- draw files 14
 - combining 31
 - copying 28
 - current file name 8
 - default name 28
 - generating 29
 - grids in 12
 - objects in 14
 - opening 31
 - printing 30
 - saving 28
- Draw widget 22
- draw_everywhere 80
- draw_inside 80
- draw_nowhere 80
- draw_on_border 80
- draw_outside 80
- DRAW_RGB_FILES 133
- drawing
 - adding graphic attributes 34
 - arcs 36
 - ellipses 37
 - geometric objects 36
 - lines 36
 - objects 34
 - on desktop 33
 - polygons 37
 - rectangles 37
 - segments 38
 - splines 37
- drawing objects 14
- drawing size 11
- DRAWPATH 132
- DRAWPIXMAPSPATH 132, 220, 319

- dtol 125
- dumpcore 131
- duplicate 71
- E
- Edit icons 8
- Edit menu 136
- editing scripts 22
- editor
 - choosing 22
 - FMEDITOR variable 22
 - Wx 22
- ellipse 37
 - handles 20
 - icon 37
 - start and end handles 37
- ellipse functions 311
- else 103
- Empty icon 43
- empty object 49
- environment variables 132
- Escape key 14, 67
- Eval scripts 105
- eval_string 125
- event functions 172
- Event mask 78
- events 23, 80
- Expert box 55
- extern (keyword) 86, 92
- F
- FACE 85
 - application functions 108
 - arithmetic expression 86
 - assignment 97
 - assignments 102

- basic objects 88
- built-in functions 121
- comments 86
- constants 90
- Eval 105
- function calls 104
- function declaration 108
- function definition 106
- include files 87
- interpreter 85
- keywords 86
- object types 88
- reserved names 95
- special characters 86
- special variables 93
- statements 102
- type casters 100
- variables 91

FACE functions 104, 106

fast zoom 72

fastZoom 131

Fax Group 4 29

File menu 135

File Selection box 28, 31

files

- .draw 14
- .fm 29
- combining 31
- Fax Group 4 29
- generating 29
- new 31
- opening 31
- PostScript 29
- printing 30

- saving 28
- saving as .ps 30
- XDrawMaker 14
- XFaceMaker 29
- filesSee also draw files
- Fill Icon 43
- fill mode 49
- fill pattern 45, 49
- first 115, 125
- Float 128
- floating-Point variables
 - in FACE 109
- fm files
 - generating 29
- FMEDITOR 78, 133
- font table functions 180
- fonts 56
- for 103
- Foreground Color icon 44, 48
- Free 122
- free 125
- ftoi 125
- function (keyword) 106, 108
- function calls 102, 108
- functions
 - application 108
 - arrow 142
 - bitmap table 146
 - center management 186
 - color table management 154
 - ellipse 311
 - event 172
 - font table 180
 - geometric transformation 186

- global 108
- graphic transformation 217
- group 307
- image management 304
- in FACE 106
- load 239
- mathematical 244
- object creation 159
- object management 260
- pixmap table 317
- redisplay 244
- rotation 195
- save 239
- scale 199
- script 324
- slot management 331
- string_to_widget 244
- synchronization 244
- text 340
- text alignment 340
- text attributes 345
- text file management 350
- text insertion 356
- text killer 364
- text position 370
- translation 212
- type checking 289
- various geometric 191
- widget_to_string 244

Functions menu 23

-fz 72, 131

G

- gadgets
 - properties 99

- generating files 29
 - Fax Group 4 files 29
 - fm files 29
 - PostScript files 29
- Geometric icons 36
- geometric transformation functions 186
- global (keyword) 92
- global functions 108
- Graphic icons 8, 33, 43
- graphic icons 8, 33
- graphic transformation functions 217
- Grid Configuration box 12, 73
- grids 72
 - changing 73
 - hide/show 72
 - setting up 12
 - snap to grid 13, 74
 - viewing 12
- group management functions 307
- grouping objects 21
- groups 21
 - inherited attributes 48
 - moving 21
 - permanent 21
 - selecting 21
 - temporary 21
- H
- handles 20, 34
 - start/end for ellipse 37
- help 131
- Help menu 140
- I
- icon bar 9, 34
- icons

- Background Color 44, 49
- Close 43
- deselecting 36
- Ellipse 37
- Empty 43
- Fill 43
- Foreground Color 44, 48
- Graphics 34
- Image 65
- Keep Tool 35
- Line 36
- Line selector 45
- objects 15, 34
- Open 43
- Patterns 45, 49
- Polygon 37
- Rectangle 37
- Segment 38
- Select 35
- selecting 34
- Spline 37
- if-else 103
- Image** 128
- Image dialog box 65
- Image icon 65
- image management functions 304
- imageDirectory 131
- images
 - in slots 54
 - inserting 65
 - loading 65
 - resizing 65
- imgdir 131
- include (keyword) 87

-indicator 8, 131
indicator 131
installing XDM 7
Int 128
interpreter 85
IsManaged 122
IsSensitive 122
itoa 125
itod 126
itof 126
itof 126
J
justification of text 60
K
Keep Tool icon 35
keyboard 123
KeyPress action 79
KeyRelease action 79
keywords 86
L
Label icon 63
landscape 11
launching XDrawMaker 8
line 36
Line icon 36
Line Selector icon 45
lines
 arrows 52
 dashes 53
Link and Slot icons 8, 33
Link icons 18
linking objects 17
Load action 79
load functions 239

load indicator 8
local 91
location
 draw_everywhere 80
 draw_inside 80
 draw_nowhere 80
 draw_on_border 80
 draw_outside 80
LowerWidget 122
M
main window 9
Malloc 122
malloc 126
ManageChild 122
MapWidget 122
margins 11
mathematical functions 244
measurement 9
menu bar 9
menus 135
MenuWidget 129
MouseMoved action 79
MoveWidget 122
moving objects 21, 67
moving slots 18
mwm 7
N
NameToWidget 122
naming objects 42
new_array 113, 126
new_c_array 113, 126
new_struct 119, 126
new_table 113, 126
NewString 123

next 115, 126

None 90

NSL_LICENSE_PORT 133

NSLHOME 133

O

object creation functions 159

Object icons 8, 33

object management functions 260

objects 14

- adding color 44

- aligning on grids 75

- angle 51

- animating 22, 77

- arc 36

- center 51

- closing 43

- connecting with paths 40

- core 51

- deselecting 20

- drawing 14

- filling 43

- groups 21

- labels 63

- linking 17, 18

- modifying 34

- moving 21, 67

- naming 16, 41

- opening 43

- post scale 51

- pre scale 51

- resizing 68

- scaling 70

- segment 38

- selecting 20

- slots 17
 - viewing when hidden 20
 - zooming 72
- OK button 11
- olwm 7
- Open icon 43
- opening files 31
- Options menu 136
- osfCancel 14, 67
- outline box 4
- ownCmap 131
- owncmap 131
- P
- Page Description box 10
- page margins 11
- page size 10
 - custom 11
 - defining 10
 - landscape 11
 - no page 10
 - portrait 11
 - size limits 11
 - units of measure 9
- parent (keyword) 95
- paste 71
- paths 40
 - between objects 40
- Patterns icon 45, 49
- pct filled 49
- pie close mode 50
- Pixmap** 129
- pixmap table functions 317
- pixmap
 - background 49

- in slots 54
- Pointer** 129
- polygon 37
 - arrows 52
- Popdown** 123
- popdown** 126
- Popup** 123
- portrait 11
- post scale 51, 70
- PostScript
 - files 29
 - printing draw files 30
- pre scale 51, 70
- press 4
- Print Box 30
- printCmd 30, 131
- printcmd 30, 131
- printf** 126, 127
- printing files 30
- program
 - starting 8
- properties 98
- R**
- RaiseWidget** 123
- random** 127
- Read command 28
- rectangle 37
- redisplay functions 244
- release mouse button 4
- remove point 68
- RemoveGrab** 123
- Reset button 9
- reset center 69
- Resize action 79

Resize Policy 83
resizeIcon 132
ResizeWidget 123
resizing objects 68
resources 131
 accessing 97
 defined in Expert box 55
 specifying 55
return 127
return (keyword) 106
-revision 131
-rfn 131
RGB box 44, 48
-ri 131
root (keyword) 95
rotate 69
rotation functions 195
rubber band box 4, 20
rulerFont 131
Run command 82
S
Save As command 28
Save command 28
save functions 239
saving files 28
 combined 28
 copies 28
-sbg 132
scale functions 199
scaling 70
script 104
 editing 78
script (keyword) 104
Script Editing box 77

script management functions 324

scriptBackground 132

scriptFont 132

scriptForeground 132

scripts 80

- editing 22

- format 79

- named 104

- pointers 80

- running 82

- shared 104

- testing 82

- writing 78, 80

segment 38

select button 4

Select icon 35

selecting objects 20

- in Tree 20

self (keyword) 95

SetKeyboardFocus 123

SetSensitive 123

SetValues 124

-sfg 132

-sfn 132

shift-click 20

showing objects 20

size 127

size 115

Slot box 53

slot functions 331

Slot icons 17

slots 17

- deleting 18

- inserting pixmaps 54

- linking 18
- moving 18
- resizing 54
- Slot box 54
- Snap to Grid 13, 21, 74
- spline 37
 - arrows 52
- sprintf 127
- sscanf 127
- starting XDrawMaker 8
- statements 102
- stipple 49
- strcmp 127
- strcpy 127
- String 129
- string_to_widget functions 244
- strlen 127
- strncmp 127
- structures 117
 - in face 117
- synchronization functions 244
- T
- technical support 5
- testing 77
- text 56
 - aligning 63
 - fonts 62
 - justifying 60
 - selecting 56
- text alignment functions 340
- text attribute functions 345
- text editor 78
 - choosing 78, 133
- Wx 78

text file management functions 350

text insertion functions 356

text killer functions 364

text management functions 340

text position functions 370

oplevel (keyword) 95

TopLevelShell 95

Transform menu 138

TransientShell 95

translation action 80

translation functions 212

Tree window 41

 naming objects 16

twm 7

type (keyword) 100

type casters 100, 128

type checking 89, 100

type checking functions 289

type conversion 101

U

undo 71

ungroup 21

unit 9

UnmanageChild 124

UnmapWidget 124

-urc 132

useRgbCube 132

V

variables 91

 global 92

 local 91

 scope of in FACE 107

various geometric functions 191

-verbose 132

verbose 132
View menu 136
W
Warning 124
while 103
while (keyword) 103
Widget 129
widget_to_string functions 244
widgets 95
 naming 95
 resources 97
Window 129
window manager 7
Wx text editor 22, 78, 133
X
X properties 98
 for gadgets 99
XBMLANGPATH 133, 220, 319
XDrawMaker Exec window 77
XDrawMaker functions 23, 80
XFaceMaker files 29
XFILESEARCHPATH 133
Xm.h 88
XmBackgroundPixmap 129
XnsIDrawAddInGroup 308
XnsIDrawAdjustObjects 261
XnsIDrawAlignObjectToGrid 262
XnsIDrawArmScript 325
XnsIDrawBitmapName 147
XnsIDrawBitmapPixmap 148
XnsIDrawBitmapStipple 149
XnsIDrawBringToFront 282
XnsIDrawCallScript 326
XnsIDrawCallScriptByName 327

XnslDrawChangeFile 240
XnslDrawCircTranslateObject 192
XnslDrawCirculateDown 283
XnslDrawCirculateUp 284
XnslDrawClearBitmapTable 150
XnslDrawClearColorTable 155
XnslDrawClearFontTable 181
XnslDrawClearPixmapTable 318
XnslDrawClearSlotSelection 332
XnslDrawColorName 156
XnslDrawColorPixel 157
XnslDrawComplete 245
XnslDrawCos 246
XnslDrawCreateArc 160
XnslDrawCreateDrawObject 161
XnslDrawCreateEllipse 162
XnslDrawCreateImage 163
XnslDrawCreateLabel 164
XnslDrawCreateLine 165
XnslDrawCreatePolygon 166
XnslDrawCreateRectangle 167
XnslDrawCreateRegularPolygon 168
XnslDrawCreateSegment 169
XnslDrawCreateSpline 170
XnslDrawCreateText 171
XnslDrawDefineCenter 187
XnslDrawDegToRad 247
XnslDrawDeleteAllChildren 263
XnslDrawDeleteAllObjects 264
XnslDrawDisableScripts 328
XnslDrawDisarmScript 329
XnslDrawDuplicateObject 265
XnslDrawDupObject 266
XnslDrawEnableScripts 330

XnslDrawEventDbIX 173
XnslDrawEventDbIY 174
XnslDrawEventToAngle 175
XnslDrawEventX 176
XnslDrawEventY 177
XnslDrawFaceGetObjectPoints 267
XnslDrawFaceGetObjectXPoints 268
XnslDrawFaceSetObjectPoints 269
XnslDrawFirstSelected 287
XnslDrawFlipObject 270
XnslDrawFlush 248
XnslDrawFontName 182
XnslDrawFontStruct 183
XnslDrawGetBackground 218
XnslDrawGetBackgroundPixel 219
XnslDrawGetBitmap 220
XnslDrawGetCenterX 188
XnslDrawGetCenterY 189
XnslDrawGetColor 158
XnslDrawGetCoordFromDrawPoint 249
XnslDrawGetEllipseEndAngle 312
XnslDrawGetEllipseStartAngle 313
XnslDrawGetFont 184
XnslDrawGetFontIndex 185
XnslDrawGetForeground 221
XnslDrawGetForegroundPixel 222
XnslDrawGetImage 305
XnslDrawGetIndexBitmap 151
XnslDrawGetKeyEventString 178
XnslDrawGetLineWidth 223
XnslDrawGetNthChild 250
XnslDrawGetObjectName 271
XnslDrawGetPixmap 319
XnslDrawGetPostScaleX 200

XnslDrawGetPostScaleY 201
XnslDrawGetPreScaleX 202
XnslDrawGetPreScaleY 203
XnslDrawGetRotation 196
XnslDrawGetSlotPixmap 333
XnslDrawGetSlotSize 334
XnslDrawGetStipple 152
XnslDrawGetStipplePixmap 153
XnslDrawGetTranslationX 213
XnslDrawGetTranslationY 214
XnslDrawGetUserData 272
XnslDrawHideObject 273
XnslDrawLoadFile 241
XnslDrawMoveObject 193
XnslDrawMoveObjectTo 194
XnslDrawNameToObject 274
XnslDrawNextSelected 288
XnslDrawObjectVisible 275
XnslDrawParent 276
XnslDrawPixmapName 320
XnslDrawPixmapPixmap 321
XnslDrawPixmapType 322
XnslDrawPostScaleObject 206
XnslDrawPreScaleObject 207
XnslDrawRadToDeg 251
XnslDrawRandom 252
XnslDrawReadFile 242
XnslDrawRedisplay 253
XnslDrawRedisplayAll 254
XnslDrawRefNameToObject 277
XnslDrawRefreshControl 255
XnslDrawReleasePixmap 323
XnslDrawRemoveFromGroup 309
XnslDrawRotateObject 197

XnslDrawSaveWidget 243
XnslDrawSelectObjectSlots 335
XnslDrawSelectSlot 336
XnslDrawSendToBack 285
XnslDrawSetArrowDimension 143
XnslDrawSetArrowDirection 144
XnslDrawSetArrowStyle 145
XnslDrawSetBackground 224
XnslDrawSetBackgroundIndex 225
XnslDrawSetBackgroundPixel 226
XnslDrawSetCloseMode 227
XnslDrawSetDashes 228
XnslDrawSetEllipseAngles 314
XnslDrawSetEllipseEndAngle 315
XnslDrawSetEllipseStartAngle 316
XnslDrawSetFillMode 229
XnslDrawSetForeground 230
XnslDrawSetForegroundIndex 231
XnslDrawSetForegroundPixel 232
XnslDrawSetImage 306
XnslDrawSetLineWidth 233
XnslDrawSetObjectName 278
XnslDrawSetPctFill 234
XnslDrawSetPostScale 209
XnslDrawSetPreScale 210
XnslDrawSetRotation 198
XnslDrawSetSlotPixmap 337
XnslDrawSetSlotSize 338
XnslDrawSetStipple 235, 236
XnslDrawSetStipplePixmap 237
XnslDrawSetTranslation 215
XnslDrawSetTSOrigin 238
XnslDrawSetUserData 279
XnslDrawShowObject 280

XnslDrawSin 256
XnslDrawSleep 257
XnslDrawStringToWidget 258
XnslDrawSync 179
XnslDrawTextChangeAttributesAtPos 346
XnslDrawTextChangeFont 347
XnslDrawTextClearText 365
XnslDrawTextEraseBuffer 366
XnslDrawTextGetAsciiText 357
XnslDrawTextGetInputPosition 371
XnslDrawTextGetLastPosition 372
XnslDrawTextGetPosition 373
XnslDrawTextGetString 358
XnslDrawTextGiveStateAtPos 348
XnslDrawTextInsertAsciiAtPos 359
XnslDrawTextInsertAsciiFile 351
XnslDrawTextInsertAsciiText 360
XnslDrawTextInsertDtifFile 352
XnslDrawTextInsertDtifText 361
XnslDrawTextInsertDtifTextAtPos 362
XnslDrawTextInsertMark 349
XnslDrawTextReadDtifFile 353
XnslDrawTextRuboutChar 367
XnslDrawTextRuboutCharAtPos 368
XnslDrawTextSaveAsciiText 354
XnslDrawTextSaveDtifText 355
XnslDrawTextSetAlignment 341
XnslDrawTextSetParAlignFromPos 343
XnslDrawTextSetParAlignment 344
XnslDrawTextSetPosition 375
XnslDrawTextYank 369
XnslDrawToggleSelectSlot 339
XnslDrawTranslateObject 216
XnslDrawUndefineCenter 190

XnslDrawUngroupGroup 310
XnslDrawWidgetToString 259
XnslsDrawArc 290
XnslsDrawEllipse 291
XnslsDrawGroup 292
XnslsDrawImage 293
XnslsDrawLabel 294
XnslsDrawLine 295
XnslsDrawObject 296
XnslsDrawPath 297
XnslsDrawPolygon 298
XnslsDrawRectangle 299
XnslsDrawSegment 300
XnslsDrawSpline 301
XnslsDrawText 302
XnslsDrawWidget 303
XnslNdrawAngleUnits 312
XtGetValues 97
XtSetValues 97
Z
zoom 72